

Succinct Non-interactive Arguments of Proximity

Liyan Chen¹, Zhengzhong Jin², and Daniel Wichs^{2,3}

¹MIT

²Northeastern University

³NTT Research

February 4, 2026

Abstract

We study succinct non-interactive arguments of proximity (SNAP), which allow a prover to convince a verifier that a statement is true through a short message. Moreover, the verifier reads only a sublinear number of bits of the statement, and soundness is required to hold against polynomial-time adversaries when the statement is ε -far from any true statements. SNAPs can be seen as the natural analog of property testing in the context of succinct non-interactive arguments (SNARGs).

We obtain both positive and negative results for SNAPs.

- **Adaptive SNAPs for P and NP:** For any $\varepsilon \in (0, 1)$, we construct the first adaptively sound SNAPs for P with ε -proximity based on standard assumptions: LWE or subexponential DDH or DLIN over bilinear maps. Our proof size, verifier's query complexity, and verification time are $n^{1/2+o(1)} \cdot \text{poly}(\lambda)$, where n is the length of the statement and λ is the security parameter. By additionally assuming sub-exponentially secure indistinguishability obfuscation, we upgrade this result to SNAPs for NP with essentially the same parameters. Previously, we only had non-adaptively sound SNAPs for P in the designated verifier setting with $O(n^{1-\delta})$ proof size, query complexity, and verification time for some constant $\delta > 0$.
- **Lower Bound:** We show that our parameters in the adaptive soundness setting are nearly optimal, up to an $n^{o(1)} \cdot \text{poly}(\lambda)$ factor: in any adaptive SNAP for P, the product of proof size and verifier query complexity must be $\Omega(n)$. Our lower bound is unconditional.
- **Fully Succinct Non-adaptive SNAPs for NP:** For any constant $\varepsilon \in (0, 1)$, we construct the first non-adaptively sound SNAPs for NP with ε -proximity, based on learning with errors and indistinguishability obfuscation. The proof size, verifier's query complexity, and verification time in our constructions are fixed polynomials in the security parameter. We also show that restricting such SNAPs to just P would already imply non-adaptively sound SNARGs for NP.

Central to our SNAP constructions is a new notion of *commitment of proximity*, which enables sublinear-time verification of the commitment. To derive our unconditional lower bound, we adopt and generalize theorems from oracle-presampling techniques in the random oracle literature. Both techniques may be of independent interest.

Contents

1	Introduction	1
1.1	Our Results	2
1.2	Key Tool: Commitment of Proximity	5
1.3	More Related Work	6
2	Technical Overview	6
2.1	Impossibility Result for Adaptively Sound SNAP	6
2.2	SNAP for P with Tight Verification Efficiency	9
2.3	Fully Succinct Non-Adaptive SNAP	14
2.4	Negative Result for Non-Adaptive SNAP for P	15
3	Preliminaries	16
3.1	Error Correcting Codes	16
3.2	Batch Arguments (BARGs)	16
3.3	Hash Family with Local Opening	18
3.4	Somewhere Extractable Hash (SEH) Family	19
3.5	RAM SNARGs	21
3.6	Flexible RAM SNARGs with Partial Input Soundness	22
3.7	Indistinguishability Obfuscation for Circuits	24
3.8	SNARG for NP	25
4	Commitment of Proximity	26
4.1	Preliminary: Unbalance Bipartite Expander	26
4.2	Definitions	28
4.3	Construction	28
5	Succinct Non-interactive Arguments of Proximity (SNAPs)	30
5.1	Definitions	31
5.2	Main Theorem Statement	33
5.3	Additional Preliminary: Secure Sketch	34
5.4	Construction	35
5.5	Analysis	38
5.5.1	Completeness and Efficiency	38
5.5.2	Adaptive Instance Extractability: Overview and Basic Conventions	39
5.5.3	Adaptive Instance Extractability From Correctness of Bottom Layers	40
5.5.4	Correctness of Digests by Induction	41
5.6	Generic Upgrade to SNAPs for NP	44
6	Unconditional Impossibility Result	47
6.1	Additional Preliminaries	47

6.1.1	Information Theoretical Tools	47
6.1.2	Secret Sharing	48
6.2	Efficient Oracle Pre-sampling Lemma	49
6.3	Attack By Pre-sampling	51
7	Lower Bound for Non-adaptively Secure SNAP	53
7.1	Construction of SNARG for NP from SNAP for P	53
7.2	Analysis	54
8	Fully Succinct SNAP with Non-adaptive Soundness	56
8.1	Preliminary: Somewhere Statistical Correlation Intractable Hash Function	56
8.2	Construction	57
8.3	Analysis	58
A	A Non-uniform Efficient Oracle Pre-sampling Lemma	65

1 Introduction

A Succinct Non-Interactive Argument (SNARG) allows a prover to convince a verifier that a statement is true by providing a short, one-message, computationally sound proof. SNARGs operate in the common reference string (CRS) model, where a trusted setup algorithm generates a CRS, and shares it with the prover and the verifier. A key feature of SNARGs is to provide fast verification time, ideally only linear in the statement size, which makes SNARGs versatile in numerous cryptographic applications, such as delegation of computation and blockchains [GKR15; BCG+14]. Fast verification makes SNARGs interesting even for languages in P, where deciding the language may take super-linear time. There has been a lot of recent progress on constructing SNARGs, and we have adaptively secure SNARGs for P under many standard assumptions including LWE, the DLIN assumption on bilinear maps, a combination of QR and DDH, or the sub-exponential hardness of DDH [KVZ21; CJJ22; HJKS22; WW22; KLVW23; CGJJZ23]. We also have (adaptively secure) SNARGs for NP assuming (sub-exponentially secure) indistinguishability obfuscation and one-way functions [SW14; WW24b; WZ24; WW24a]. However, a major downside of SNARGs for P or NP is that the verification time must be at least linear in the length of the statement, since the verifier must inherently read the statement being verified. In many modern applications in the big data era, the size of the statement can be huge, making even linear-time verification insufficient.

As a motivating example, consider the case of Elon Musk acquiring the social media company Twitter in 2022 [ABDY24]. He claimed that 5% of Twitter’s daily active users were spambots and stated that he would put the acquisition on hold until Twitter could refute this claim. This later led to a legal battle between Twitter and Musk. If Twitter could prove that its social network meets certain “healthy” standards in social network analysis, then significant public resources in the lawsuits could potentially be saved. However, it is impractical to use SNARGs here, because the verifier needs to read the entire statement — which is the entire social network. We emphasize that big datasets are not limited to social network analysis [KSY20; ABDY24], and there are many other examples as well, such as statistical analysis on massive medical data [KR15].

To handle those extremely large statements, in this work, we study SNARGs that are also arguments of proximity [KR15]. In a Succinct Non-interactive Argument of Proximity (SNAP), the prover has full access to the statement, while the verifier is only given oracle access to the statement and is only allowed to query some small (sublinear) number of its bits. The prover still sends a single short proof to the verifier to convince it that the statement is true. We consider SNAPs in the CRS model. To enable sublinear verification, the soundness is relaxed as being “approximate”. Namely, no efficient adversary should be able to produce a valid proof for any false statement that is ε -far from the language, meaning that it differs from every true statement in at least an ε -fraction of bits.

Approximate soundness can be easily upgraded to the standard (non-approximate) soundness, by encoding the statement with an error-correcting code for correcting ε -fraction of Hamming errors. Hence, SNAPs for P or NP automatically yield SNARGs for P or NP, respectively. However, approximate soundness is already sufficient for many real-world applications, such as statistics over massive datasets, or the aforementioned social network examples. In fact, relying on the approximation to achieve sublinear running time has been a fundamental idea in property testing [RS96; GGR98]. Hence SNAPs are more versatile and allow for many applications where SNARGs do not suffice.

As with SNARGs, we can consider SNAPs with *adaptive soundness*, where the adversarial prover can choose the false statement to prove adaptively after seeing the CRS, or *non-adaptive (selective)*

soundness where the adversarial prover must choose the false statement ahead of time before seeing the CRS.

We ask the following fundamental question about SNAPs:

For what parameters and under what assumptions can we build adaptively or non-adaptively sound SNAPs for P or NP?

When it comes to SNARGs, adaptive soundness can be somewhat more difficult to achieve and argue than non-adaptive soundness, but we now have constructions that get adaptive soundness for both P and NP under essentially the same assumptions and with essentially the same efficiency as what is known for non-adaptive soundness. In contrast, when it comes to SNAPs, adaptive soundness presents some inherent difficulties that make it fundamentally more difficult than non-adaptive soundness. For example, a simple observation shows that adaptively sound publicly verifiable SNAPs cannot exist with a deterministic verifier. This is because an adversary can always choose a true statement x' and use the CRS to honestly derive a SNAP proof π for it. Then it can check which small subset of the bits of x' the deterministic verifier will read when verifying π . Finally, it can flip every other bit of x' to adaptively derive some false statement x that is far from the language, but the proof π will still verify for x . Therefore, when it comes to adaptively sound SNAPs, we will need to consider a randomized verifier. As we will see, even with a randomized verifier, we provide inherent lower bounds on the parameters of adaptively sound SNAPs that don't apply to non-adaptively sound SNAPs.

Prior Work. Previously, [KR15] constructed non-adaptively sound SNAPs for P in the designated verifier setting, where the verifier needs a secret key to verify the proof, assuming sub-exponentially secure fully homomorphic encryption. Their proof size and verifier query complexity for statement size n are both $O(n^{1-\gamma})$ for some (unspecified) constant $\gamma > 0$.¹

As a complement of their construction, they also showed that adaptively sound SNAPs for P, with $o(\sqrt{n})$ proof length and $o(\sqrt{n})$ verifier query complexity for statement size n cannot be proven secure under any falsifiable assumptions via black-box reductions, by leveraging the Gentry-Wichs black-box separation for adaptively sound SNARGs for NP [GW11]. We note that this barrier should not be seen as evidence that such SNAPs for P do not exist. Indeed, adaptively sound SNARGs for NP are believed to exist, and we have constructions in the random oracle model [Mic00], or under various non-falsifiable assumptions [Gro10; BCC+17; BCIOP13]. The recent works of [WW24b; WZ24; WW24a] even showed how to use complexity leveraging to circumvent the Gentry-Wichs barrier, yielding adaptively sound SNARGs for NP that are provably secure via a black-box use of well-founded falsifiable assumptions (i.e., ones that imply iO), at the cost of having a long CRS and relying on sub-exponential hardness. Therefore, the above barrier leaves such possibilities open for SNAPs as well.

1.1 Our Results

Our results offer a detailed characterization of what is possible for SNAPs under what assumptions. Throughout this section, we use n to denote the length of the statement. The first sub-question we ask is:

¹We believe their SNAP could be improved to be adaptive by plugging in the recent adaptive no-signaling PCP of [BHK17]. Perhaps one may have also hoped to upgrade their result to the public verifiability setting by leveraging recent techniques developed in the context of publicly verifiable SNARGs. However, we found numerous obstacles towards doing so, and our eventual construction takes a completely different approach.

Question. Do there exist adaptively sound SNAPs with both $o(\sqrt{n})$ proof size and $o(\sqrt{n})$ verifier query complexity, under *any* assumptions?

Our answer to this question turns out to be negative. We show that it is *unconditionally* impossible to simultaneously beat the $\sqrt{n}$ bound for the proof size and the query complexity, even for some languages within P.

Theorem 1.1 (Informal). For some sufficiently small constant $\varepsilon > 0$, SNAPs for P with ε -proximity, proof size $p(n)$, and verifier's query complexity $q(n)$ have the following *unconditional* lower bound:

$$p(n) \cdot q(n) = \Omega(n).$$

We remark that our lower bound also extends to the setting of a large ε that is close to $1/2$, with the caveat that the lower bound becomes $p(n) \cdot q(n) = \Omega(n/\log n)$.

To obtain our results, we draw on a surprising connection to the oracle pre-sampling lemma [Unr07] previously used in the context of the auxiliary-input random oracle model, and we improve on the lemma by showing that the pre-sampled bits can be computed efficiently. We prove a uniform version as Lemma 6.12, and a non-uniform version as Lemma A.1. Although the uniform version suffices for our SNAP impossibility result, we hope the non-uniform version could be of independent interest. See Section 6 for the full proof of the impossibility result.

We then turn to the positive side, and ask the second sub-question:

Question. Can we build adaptively sound SNAPs with $\tilde{O}(\sqrt{n})$ proof size and $\tilde{O}(\sqrt{n})$ query complexity?

Note that a SNAP with $\tilde{O}(\sqrt{n})$ proof size and $\tilde{O}(\sqrt{n})$ query complexity can only have $\Omega(\sqrt{n})$ verification time. This is in contrast to the setting of SNARGs, where the verification time is usually polylogarithmic in the length of the NP witness. Nevertheless, SNAPs with verification time as large as $\tilde{O}(\sqrt{n})$ can remain of practical interest, as SNARGs with square-root verification time have also been shown to be concretely efficient [AHIV17; GLSTW23].

We provide a positive answer to this sub-question for both P and NP, beginning with SNAPs for P, as stated in the following theorem.

Theorem 1.2 (Adaptive SNAP for P, Informal). For any constant $\varepsilon > 0$, there exists an adaptively sound SNAP for P with ε -proximity assuming rate-1 somewhere extractable hash and somewhere extractable batch argument systems, with the following parameters. Let λ be the security parameter.

- The CRS length is $\text{poly}(\lambda)$.
- The proof length is $n^{1/2+o(1)} \cdot \text{poly}(\lambda)$.
- The verifier's query complexity is $n^{1/2+o(1)} \cdot \text{poly}(\lambda)$.
- The verifier running time is $n^{1/2+o(1)} \cdot \text{poly}(\lambda)$.

In particular, such SNAP for P exists assuming either learning with errors (LWE), a combination of quadratic residuosity (QR) and decisional Diffie-Hellman (DDH), the sub-exponential hardness of DDH, or the DLIN assumption on bilinear groups. $\diamond$

For detailed construction and proof, see Section 5.²

Our results achieve the optimal proof length and verifier’s query complexity up to an $n^{o(1)}$ factor, given the impossibility result in Theorem 1.1. Compared to [KR15], our construction offers several improvements. First, it provides adaptive soundness, while [KR15] only achieves non-adaptive soundness. Second, our SNAPs are publicly verifiable, whereas [KR15] requires a designated verifier. The public verification allows anyone to verify the proof, and it has the advantage that the CRS is automatically reusable for different statements. Third, it achieves smaller proof length and query complexity compared to the $O(n^{1-\gamma})$ proof length and query complexity in [KR15].

We further extend our results to construct SNAPs for NP with similarly near-optimal proof length and query complexity, under the additional assumption of adaptive SNARG for NP, which can itself be realized from well-founded cryptographic assumptions [WW24a; JLS24; JLS22].

Theorem 1.3 (Adaptive SNAP for NP, Informal). For any constant $\varepsilon > 0$, there exists an adaptive SNAP for NP with ε -proximity, assuming the same assumption as in Theorem 1.2, and adaptively sound SNARGs for NP. In particular, assuming the same assumption as in Theorem 1.2, sub-exponential security of one-way functions, and sub-exponential security of iO, the SNAP has $\text{poly}(n, \lambda)$ prover CRS size, $\text{poly}(\lambda)$ verifier CRS size, $n^{1/2+o(1)} \cdot \text{poly}(\lambda)$ proof length, $n^{1/2+o(1)} \cdot \text{poly}(\lambda)$ query complexity by the verifier, and $n^{1/2+o(1)} \cdot \text{poly}(\lambda)$ verification time. $\diamond$

For detailed theorem statement and proof, see Theorem 5.6 and Section 5.6.

We then turn to the non-adaptive soundness setting, where the cheating prover needs to select the statement in advance, before seeing the CRS. Non-adaptive soundness may be sufficient in many applications, where the statement cannot be maliciously chosen depending on the CRS. For instance, in the social network example mentioned above, the social network is almost fixed before the generation of the CRS. Neither our unconditional impossibility nor the Gentry-Wichs style impossibility in [KR15] extends to the non-adaptively sound setting. Hence, we ask the following third sub-question:

Question. Can we build non-adaptively sound SNAPs with much smaller proof length and verifier’s query complexity?

We answer this question positively for NP, with the following theorem.

Theorem 1.4 (Fully Succinct Non-adaptive SNAP for NP, Informal). For any constant $\varepsilon > 0$, there exists a non-adaptively sound SNAP for NP with ε -proximity and $\text{poly}(\lambda)$ proof length, $\text{poly}(\lambda)$ verifier’s query complexity, and $\text{poly}(\lambda)$ verification time, assuming learning with errors, sub-exponentially secure OWF, and sub-exponentially secure iO, which in turn can be constructed from well-studied assumptions. $\diamond$

For detailed proof of Theorem 1.4, see Section 8.

By restricting the language to P, can we construct non-adaptively sound SNAPs with $o(\sqrt{n})$ proof length and query complexity, without relying on heavy machinery such as iO? We further show that achieving such a result would imply an improved construction of SNARGs for NP, which currently remains unknown. This is formalized in the following theorem.

²For simplicity, we state all our informal theorems in the introduction for a constant $\varepsilon > 0$, but all our results also generalize to sub-constant ε . See the formal theorem statements for the general versions.

Theorem 1.5 (Lower-bound for Non-adaptive SNAP for P, Informal). If there exist non-adaptively sound SNAPs for P with proof length $p(n)$ and query complexity $q(n)$ and

$$p(n) \cdot q(n) = o(n),$$

then there exist non-adaptively sound SNARGs for NP (with imperfect completeness), without making any additional assumptions. $\diamond$

Our lower bound for non-adaptively sound SNAP for P can also be viewed as a new path to build SNARGs for NP. For the detailed proof, see Section 7.

1.2 Key Tool: Commitment of Proximity

As a key tool to obtain our SNAP constructions, we developed a new type of commitment scheme that we call *commitment of proximity*, which may be of independent interest. At a high level, it is a commitment scheme that supports sublinear-time verification of the opening and ensures that the opened value is approximately close to the committed one. Specifically, a commitment of proximity consists of the following algorithms.

- **Commitment.** Given a random commitment key ck , the commitment algorithm computes a succinct commitment c from a message $x \in \{0, 1\}^n$.
- **Verification.** The verification algorithm takes a commitment c and a commitment key ck as input, and is given oracle access to a message x . The verification algorithm only queries a *sublinear* number of bits in x , and can decide to either accept or reject the message.

Crucially, we require the following security of commitment of proximity.

Binding of Proximity with Distance Parameter k . Given the commitment key ck , no efficient adversary can find two messages x, x' with Hamming distance greater than k that would cause the verification algorithm to accept when given the correctly generated commitment c of the message x' and oracle access to the other message x . We refer to k as the *distance parameter*.

Crucial to our SNAP constructions is a near-optimal construction of commitment of proximity for any distance parameter as shown in the following theorem. We believe this theorem may be of independent interest.

Theorem 1.6 (Near-Optimal Commitment of Proximity, Informal). Assuming the existence of collision-resistant hash functions, there exists a commitment of proximity scheme with the following parameters. Let λ be the security parameter, $n = n(\lambda)$ be the message length, and $k = k(\lambda)$ be the distance parameter, then we have:

- Commitment size is $k \cdot n^{o(1)} \cdot \text{poly}(\lambda)$.
- Verifier's query complexity is $n/k \cdot n^{o(1)} \cdot \text{poly}(\lambda)$.
- Verification time is $(k + n/k) \cdot n^{o(1)} \cdot \text{poly}(\lambda)$.

In particular, for $k = \sqrt{n}$ all above efficiency measures become $\sqrt{n} \cdot n^{o(1)} \cdot \text{poly}(\lambda)$. $\diamond$

We emphasize that our impossibility Theorem 1.1 extends to the commitment of proximity, and hence the parameters in our construction are near-optimal up to an $n^{o(1)} \cdot \text{poly}(\lambda)$ factor. For the detailed construction and proof, see Section 4.

1.3 More Related Work

Holographic SNARGs. Holographic SNARGs [BFLS91; Kiy23] enable a prover to convince a verifier of a statement’s truthfulness via a short message, where the verifier is a sublinear-time algorithm with oracle access to an encoding of the statement. Crucially, the encoding is guaranteed to be correct when defining both completeness and soundness. Unlike holographic SNARGs, SNAPs do not rely on any guaranteed encoding of the statement, which makes them more realistic in settings such as the aforementioned social network applications, where encoding the entire network may not be feasible. If the encoding is an error-correcting code, then any SNAP also functions as a holographic SNARG, though the reverse implication may not hold.

Interactive Proofs of Proximity. Interactive proofs of proximity [EKR04; RVW13] allow a prover to convince a sublinear-time verifier that a statement is true. These proof systems are defined in the plain model (without a trusted setup) and require soundness to hold against unbounded provers for any statement that is ε -far from the true statements, with interaction allowed over multiple rounds. In contrast, SNAPs differ in that the proof consists of a single, short message without further interaction, with soundness guaranteed only against polynomial-time adversaries, and requiring a trusted setup. Hence, our SNAP constructions rely on cryptographic assumptions, but this suffices for many practical applications. Interactive proofs of proximity has been studied extensively (to name a few, [GGR15; GR17; CG18; GLR18]). In particular, [GR15] also studied the non-interactive setting (for proofs with statistical security), but did not consider general languages such as P, NP . Other variants, such as interactive oracle proofs of proximity [BCGRS17; RRR16] and probabilistically checkable proofs of proximity [BGHSV06], are also defined with the unbounded soundness setting and in the plain model. However, they also require the prover to send either long proof strings or functions. Prior work [BRV18] also considered interactive arguments of proximity in the plain model, achieving soundness against polynomial-time bounded cheating provers and additionally achieving statistical zero-knowledge. However, their protocol requires multiple (constant) rounds of interaction, whereas ours is non-interactive (single-round) but works in the CRS model.

2 Technical Overview

We outline our technical approach. This section is divided into four parts, addressing each main result.

2.1 Impossibility Result for Adaptively Sound SNAP

We show the impossibility of adaptively sound SNAPs for P with both $o(\sqrt{n})$ proof size and $o(\sqrt{n})$ verifier’s query complexity. To prove this impossibility result, we construct a specific language in P and then give a generic attack that efficiently breaks the adaptive soundness of any candidate SNAP scheme as above for this language.

Hard language for SNAPs from Secret Sharing. We start by finding a language in P for which it is impossible to construct a SNAP as above.

Let’s first consider the toy case where the length of the SNAP proof is zero. In this case, the verifier needs to verify that the instance is close to the language by only querying a few locations.

To carry out a successful attack, it suffices to find some instance with two properties: (a) it is far from the language, (b) but it looks like a yes instance given only a few queries.

The two properties we need are directly offered by secret sharing. Consider an n -party threshold secret sharing scheme with a boolean message and boolean shares, where $t = \Omega(n)$ shares provide privacy and $r = \Omega(n)$ shares suffice for reconstruction. Define the language $\mathcal{L}$ as consisting of instances $x = (s_1, \dots, s_m)$ that are a valid secret sharing of the bit 0, where s_i is the share of party i . Now let x' be a random secret sharing of 1. Showing that x' satisfies properties (a) and (b) above follows from the threshold reconstruction and privacy of the secret sharing:

- Threshold reconstruction implies that x' is far from the language. Since otherwise, if x' agrees with some $x \in \mathcal{L}$ on r shares, then these shares would incorrectly reconstruct to 0.
- Privacy implies that the verifier would mistake x' for some yes instance. As the verifier can only read a few locations of x' (i.e., only has access to the shares of a few parties), privacy ensures that the verifier cannot distinguish whether the shared message is 0 or 1.

However, the above argument no longer holds if we take into account the SNAP proof. The problem is that the verifier can read the SNAP proof which depends on the entire instance. In particular, the SNAP proof can easily tell the verifier what the shared bit is.

Instead, our attack works by starting with a random true instance x and a SNAP proof π for x . We show that the SNAP proof only reveals a small number of bits of x . In particular, given π , we can flip many of the bits of x to derive some x' far from the language such that one cannot distinguish between seeing π and getting a bounded number of oracle queries to x vs x' . Therefore, also the SNAP verifier cannot distinguish and will accept the proof π as a valid proof for x' which is far from the language, breaking security of the SNAP. We show this indistinguishability property by applying the *oracle pre-sampling technique* (we would also call it “bit-fixing”), which was previous used to study the auxiliary-input random-oracle model (AI-ROM).

Bounding Information of SNAP Proof by Oracle Pre-sampling. To explain the oracle pre-sampling technique, we take a quick detour into the AI-ROM. The model AI-ROM arises naturally when analyzing *non-uniform* adversaries in the random oracle model (ROM). In AI-ROM, the adversary has access to a random oracle plus an auxiliary input that depends on the entire random oracle and may not be efficiently computable. A notable technical difficulty in AI-ROM is that we cannot apply lazy sampling to analyze security of the random oracle, as the auxiliary input arbitrarily depends on the entire oracle. To overcome this difficulty, the oracle pre-sampling technique (or “bit-fixing”) was introduced in [Unr07] and later improved in [BN13; CDGS18]. The technique is, given an oracle O and its corresponding auxiliary input z , we can fix a few bits on O and resample the rest uniformly at random. It is proved that the new oracle O' together with the original auxiliary input z , is indistinguishable from (O, z) against adversaries with bounded query complexity.

We illustrate how we would use oracle pre-sampling technique by making an analogue between our setting of adaptively secure SNAPs and AI-ROM: the verifier corresponds to the AI-ROM distinguisher, the instance x corresponds to the random oracle (note that the verifier reads x exactly via oracle access), and the SNAP proof corresponds to the auxiliary input that depends on the entire random oracle. By this analogue, the oracle pre-sampling technique translates to the idea that, given a yes instance x and its corresponding SNAP proof π (for now we ignore the crs), it is possible to fix a few bits in x i.e. the shares of a few parties, and then resample the rest of the shares

uniformly at random, such that the verifier would accept π as a proof to the new instance x' . We interpret the result as the proof π only delivers information about a few bits in x . Therefore, even with the SNAP proof it is still possible to modify the “unfixed” bits of x to obtain some x' far from the language.

A Generic Attack via Efficient Oracle Pre-sampling. We apply the oracle pre-sampling technique to construct an attack to the candidate SNAP. The attacker first samples a random yes instance x and computes its corresponding SNAP proof π , then uses the oracle pre-sampling technique to fix a few bits in x i.e. sharings of a few parties and resample the rest to derive x' . However, the attacker does not resample the rest bits as a fresh secret sharing of 0 faithfully, but instead resamples them as a fresh secret sharing of 1. This change cannot be distinguished via a bounded number of queries to x' and these bits are independent of π . Overall, this ensures that x' is far from the language but the SNAP verifier nevertheless accepts π as a valid proof for x' .

To make this attack run in polynomial time, we have to make the oracle pre-sampling procedure *efficient*: there should be a poly-time algorithm that computes the set of fixed bits in x from (x, π) . To our knowledge, this special property was not addressed by previous papers on AI-ROM. We overcome this problem by proving efficient oracle pre-sampling lemmas. We prove two version of efficient oracle pre-sampling, one is the uniform version, where the algorithm to compute the fixed bits is uniform, but bit-fixing can only fool a particular adversary; the other is the non-uniform version, where the algorithm to compute fixed bits is non-uniform, but bit-fixing can fool any bounded-size adversary. Although the uniform version suffices for our SNAP impossibility result (since we only need to fool a particular algorithm - namely, the SNAP verifier), we also prove the non-uniform version in Appendix A as efficient oracle pre-sampling could be of independent interest. We also note that our oracle pre-sampling lemmas do not achieve the optimal parameter as in [CDGS18], but this does not affect the application to SNAPs. We leave proving efficient oracle pre-sampling lemmas with *tight parameter* as an interesting open problem of independent interest.

At last, we address a few technical concerns about our attack, as well as the parameters:

- **The Common Reference String.** In the above overview about our attack, we ignore the SNAP common reference string. In spirit the crs is not hard to tackle with, as it is completely independent of the instance. We show how to easily augment the oracle pre-sampling lemma to takes the crs into account.
- **Random Oracle with Arbitrary Distribution.** The results in AI-ROM originally only considers uniformly random oracles, but in our case the oracle is distributed according to secret sharings of 0. Fortunately, this issue was already addressed by [BN13] which extends oracle pre-sampling lemma in [Unr07] to arbitrary oracle distributions. Our efficient oracle pre-sampling lemmas are also proved for any oracle distributions, by utilizing the proof technique in [BN13].
- **The Attack Parameters.** To apply the oracle pre-sampling technique, the number of fixed bits roughly equals to the product of the length of auxiliary input, and the query complexity of the distinguisher. Translating to the context of SNAPs, our attack only works if the product of proof length and verifier query complexity is sub-linear in instance length. In particular, our attack rules out adaptive SNAPs with $o(\sqrt{n})$ proof length and $o(\sqrt{n})$ verifier’s query complexity.

An additional feature of this attack is that it relativizes, therefore the impossibility result also holds in the random oracle model.

2.2 SNAP for P with Tight Verification Efficiency

Warm-up: Adaptive SNAP from SNARKs. As a first step, we show how to build adaptively sound SNAPs with $O(\sqrt{n})$ verification time from SNARKs, which can be built based on knowledge-type assumptions [BCC+17]. We will later replace SNARKs with standard assumptions.

In the adaptively sound setting, the prover chooses the instance x for the verifier. Hence, the main challenge is ensuring that the instance “in the prover’s mind” and the verifier’s instance x are almost the same (i.e., ε -close). For example, a malicious prover may generate a proof π for some different instance $x' \in \mathcal{L}$, and then use π as a cheating proof for an x far from $\mathcal{L}$. A natural attempt to prevent such attacks is to have the prover hash its instance x' and use a SNARK to prove that $x' \in \mathcal{L}$ and that the hash is correctly computed from x' . However, the verifier must then read its own instance x and recompute the hash to ensure consistency, which takes at least linear time.

To achieve sublinear verification, we modify this approach by having the prover divide the instance evenly into $\sqrt{n}$ blocks, and hashing each block individually to obtain $\sqrt{n}$ hash values. The verifier randomly selects $O(1/\varepsilon)$ blocks, reads the corresponding blocks of its own instance x , and verifies the consistency of the hash values for those blocks. To see why this approach gives us the desired ε -closeness between x' and x , if there is an ε -fraction of Hamming errors between x' and x , then at least ε -fraction of the hash values are inconsistent. Then the verifier can detect them with high probability.

In summary, the full SNAP construction proceeds as follows.

- **Prover.** The prover divides its instance into $\sqrt{n}$ blocks where each block consists of $\sqrt{n}$ bits. It then commits each block individually by applying a collision resistant hash function. Then the prover generates a SNARK, using $\sqrt{n}$ hash values as the instance, and proves that there exists an $x' \in \mathcal{L}$ and the hash values are computed correctly from x' .

The SNAP proof contains $\sqrt{n}$ short hash values and the SNARK proof.

- **Verifier.** The verifier takes those $\sqrt{n}$ short hash values. It randomly selects $\Theta(1/\varepsilon)$ blocks, then reads all locations contained in the selected blocks on its instance x , and checks if the corresponding hash values are correct. The verifier can repeat this several times to reduce the soundness error. Finally, it verifies the SNARK proof using the hash values.

To prove the soundness of the above construction, we first extract an $x' \in \mathcal{L}$ from the SNARK. Then the above randomized check of the verifier ensures that $\Delta(x', x) \leq \varepsilon n$, which implies approximate soundness.

The verification time for this construction is $O_\lambda(\sqrt{n}/\varepsilon)$, which is tight according to our impossibility result. This construction answers the feasibility of SNAPs with tight verification efficiency, and would also serve as the starting point for our further constructions.

The above construction of SNAP can be divided into a hash-by-block mechanism and a SNARK component. Recall that, the hash-by-block part ensures the x' “in the prover’s mind” can have at most εn Hamming errors compared to the verifier’s instance x . Looking ahead, we will first improve this εn parameter to $O(\sqrt{n})$ using a better hash-by-block mechanism. We then encode the $O(\sqrt{n})$ differing bits between x and x' as part of the SNAP proof. This improvement is crucial to replacing the SNARK component with a construction based on standard assumptions.

Key Abstraction: Commitment of Proximity. We abstract out the hash-by-block mechanism above as one potential instantiation of a *commitment of proximity*, a new tool that we introduced and discussed in Section 1.2.

The previous commit-by-block construction can be interpreted as a commitment of proximity with $O(\sqrt{n})$ verification efficiency for distance parameter εn with a constant $\varepsilon > 0$. Although the $O(\sqrt{n})$ verifier’s query complexity is already tight by the impossibility result, there seems to be no barrier towards a smaller distance parameter, i.e. $k = o(n)$. Note that with $O(\sqrt{n})$ verifier’s query complexity, the distance parameter should be at least $k = \Omega(\sqrt{n})$, since an adversary could flip $\Theta(\sqrt{n})$ random bits of x and the verifier is unlikely to notice this tampering within $O(\sqrt{n})$ queries.

Building Commitments of Proximity with Optimal Distance. To improve the distance parameter, instead of using non-overlapping blocks, we randomly assign $\sqrt{n}$ indices of the instance for each of $\tilde{O}(\sqrt{n})$ hash values. Namely, we compute each hash value by choosing a random subset of $\sqrt{n}$ indices and hashing the bits of the instance in this subset. Then we modify the verification of this commitment by having the verifier randomly pick λ hash values, re-compute those hash values from its own instance x , and check if the re-computed values are consistent with the picked ones. We can show that this construction achieves the optimal $O(\sqrt{n})$ distance parameter.

The reason is that, we can view the above random assignment procedure as sampling a random bipartite graph, where each index of the instance corresponds to a left vertex, and each hash value corresponds to a right vertex. Then for each $\sqrt{n}$ coordinates that x and x' differ, we can view those coordinates as a subset of the left vertices. By a standard probabilistic argument, we can show that (with high probability) every such subset of $\sqrt{n}$ left vertices neighbors a constant fraction of the right vertices. Moreover, by the collision-resistant property of the hash function, each hash value being a neighbor of this subset must be different for x and x' . Therefore, if the verification algorithm randomly checks λ hash values, it can detect the inconsistency of the hash values with high probability.

However, the commitment key now needs to include the description of this bipartite graph, and hence its size grows at least linearly in n . This would result in a large verification time, since the verifier needs to at least read this long key. Hence, we need a succinct description of the bipartite graph while achieving the same properties we need from the random graph. From the above argument for the randomized construction, the property we need is exactly the *expansion property*. Namely, we require that for any subset of $\sqrt{n}$ left vertices, it neighbors at least a constant fraction of the right vertices. As we have an explicit construction of such expander graphs with almost tight parameters³ [KT22], this gives us a commitment of proximity for distance parameter $k = \sqrt{n}$ with $O(\sqrt{n})$ -time verification. More generally, for any instance length n and distance parameter k , we have a commitment of proximity with commitment size $O(k)$, verification query complexity $O(n/k)$, and verification time $O(k + n/k)$. For more details, see Section 4.

Replacing SNARKs with RAM delegation. So far, we still need SNARKs to extract an instance x' “in the prover’s mind”. From this point forward, we focus on replacing SNARKs with a construction based on standard assumptions.

To avoid knowledge extractions, our next idea is to leverage the fact that x and x' can only differ by at most $O(\sqrt{n})$ bits guaranteed by the aforementioned commitment of proximity with optimal

³Actually, up to a factor of $n^{o(1)}$. We note that such a factor is minor as $n^{o(1)}$ is smaller than the security parameter, and we will omit it in the technical overview section.

distance. Specifically, we make the following modifications to our SNAP construction.

- The prover additionally sends a *syndrome* of its instance x' allowing for correcting $O(\sqrt{n})$ Hamming errors, as part of the SNAP proof. Recall that syndrome decoding of error correcting codes allows a receiver to efficiently recover the original message from a sender by using the syndrome and a corrupted message. By using appropriate error correcting codes, the size of such a syndrome can be $\tilde{O}(\sqrt{n})$.
- Instead of using SNARKs, the prover uses a RAM delegation for P to prove that the instance is in the language. Recall that a RAM delegation scheme allows a verifier to delegate a RAM program to the prover. Crucially, the verifier only needs a short Merkle hash digest of the initial memory to verify the proof. Hence, we have the prover Merkle hash the instance to get a short digest, and include this short digest as part of the SNAP proof.
- The prover further attaches a RAM delegation for P proof that convinces the verifier of the “consistency” between the commitment of proximity, the syndrome, and the Merkle hash digest. Namely, it proves that the commitment of proximity and the syndrome are indeed computed correctly from the same underlying instance contained in the Merkle hash digest.

It almost seems that the above approach may work. The RAM SNARGs for P ensures that, if we can somehow extract a value x' that is a preimage of the Merkle hash, then the claimed statements about x' must be true: namely, x' is in the language, and the commitment of proximity and the syndrome are computed correctly from x' . Moreover, the commitment of proximity ensures that the distance between x' and the verifier’s instance x is bounded by $O(\sqrt{n})$. Therefore, we can extract x' from the verifier’s instance x and the received syndrome using syndrome decoding.

On close inspection, however, the above suffers from a circularity issue – it relies on the ability to extract x' to argue about the soundness of the RAM SNARG, but then relies on the soundness of the RAM SNARG to argue that we can correctly extract x' . To break the circularity, we need to move to a more complex construction described below. The main intuition is to apply the above argument piecemeal, where in different hybrids we argue that we correctly extract different components of x' from x via syndrome decoding until we are able to argue that we extract it in full.

Proving Consistency in a Toy Case: 2-Layer Merkle Tree. To illustrate our main idea of getting around the above issue, let’s first consider a simplified construction, where we only build the Merkle hash tree up to depth 2 when we Merkle hash the instance. As we will see, we can modify the above SNAP construction to achieve a “slightly” succinct proof of length $n/2 \cdot (1 + o(1))$ while the verifier’s query complexity remains $O(\sqrt{n})$.

Specifically, we modify our SNAP construction as follows.

- **Prover.** The prover first divides his instance into chunks of length λ^2 , and only builds the Merkle hash tree of depth 2 over the chunks. Namely, we hash every two adjacent chunks into one hash value. Crucially, we use somewhere extractable hash (SEH) to instantiate this hash function, where the SEH is set to be extractable for either all left children, or all right children. Since rate-1 SEH hash values can achieve additive overhead of $O(\lambda)$ [DGKV22; PP22], the total length of the hash values is $n/2 \cdot (1 + o(1))$.

Let L (resp. R) be the concatenated strings of all left (resp. right) children. Then the prover computes commitments of proximity c_L, c_R of distance parameter $\sqrt{n}$ for both L and R , re-

spectively. We also compute syndromes s_L, s_R for L and R , respectively, for correcting $\sqrt{n}$ Hamming errors.

Then we apply another RAM delegation scheme, and generate “consistency” proofs π_L, π_R for the values c_L, s_L and c_R, s_R , respectively. Namely, we view L (resp. R) as the input to a RAM program that computes c_L, s_L (resp. c_R, s_R), and view the SEH hash values as the Merkle hash of the input. We can do this because the SEH hash here has local openings as small as $O(\lambda^2)$ bits.

The SNAP proof consists of all the SEH hash values, the syndromes s_L, s_R , the commitments of proximity c_L, c_R , and the consistency proofs π_L, π_R . Finally, the prover sends a RAM delegation proof $\pi_{\mathcal{L}}$ proving that the instance is in $\mathcal{L}$ using the SEH hash values.

- **Verifier.** The verifier first verifies the consistency proof π_L, π_R as the normal RAM delegation, using the SEH hash values sent in the SNAP proof. Then it verifies the commitment of proximity for L and R respectively. Finally, it verifies the RAM delegation proof $\pi_{\mathcal{L}}$.

Now we can show that, if the overall proof verifies, then we can extract some value x' “in the prover’s mind” from the verifier’s x and the syndromes, such that any local openings of the SEH that the prover can give must be consistent with x' . We do so in a piecemeal fashion. First, we set the SEH to be extractable on the left children, and extract a string L' from the SEH. The RAM delegation π_L then ensures that s_L is a correctly computed syndrome of L' , and c_L is a correctly computed commitment of proximity of L' . Since the commitment of proximity ensures that L' and L (the corresponding bits of the verifier’s x) differ by $O(\sqrt{n})$ bits, we know that L' must also be the outcome of syndrome decoding L using the syndrome s_L . This means that the local opening to any position in left children must be consistent with the syndrome decoding result L' by the statistical binding property of SEH. Our next key observation is that the consistency between local openings of the SEH and the syndrome decoded value L' does not rely on the SEH trapdoor. Therefore, even if we switch SEH to be extractable for the *right children*, the consistency between the local openings and the syndrome decoding for the *left children* still holds! Then we can continue to prove the consistency between the local openings and the syndrome decoding for the right children, using the same strategy as above for the left children. Therefore, we can extract out the overall value x' “in the prover’s mind” by applying syndrome decoding to recover L', R' from the L, R components of the verifier’s x respectively. Furthermore, x' is close to x . Lastly, the property that any local opening of the SEH that the prover can give has to be consistent with x' also ensures that the proof $\pi_{\mathcal{L}}$ then guarantees $x' \in \mathcal{L}$. Together, these properties guarantee the approximate soundness of the overall SNAP.

However, this toy case construction only achieves $n/2 \cdot (1 + o(1))$ SNAP proof size. We need to further shrink the proof length.

Reducing Proof Length by Building a Merkle Tree. The current construction can be interpreted as a method to avoid sending the instance x' “in prover’s mind” in the clear, as we send the SEH hash values of length $n/2 \cdot (1 + o(1))$ and recover x' from those SEH hash values in the security reduction. Indeed, we achieve this by first extracting the left children and then switching the SEH key to extract the right children in the above reduction. Therefore, a natural idea to further shrink the SNAP proof size is to avoid sending those SEH hash values in the clear. Instead, we can apply the same construction once again to these SEH hash values. That is, we build a new 2-layer Merkle tree on top of “bottom” SEH hash values using a new rate-1 SEH, and only send the “top” SEH hash values of length $n/4 \cdot (1 + o(1))$ in the SNAP proof. Now the SNAP proof consists of the top layer

of hash values, and “top components” $c_L^1, s_L^1, c_R^1, s_R^1, \pi_L^1, \pi_R^1$ computed the same way as previously but thinking of the bottom SEH hash values as the data and the top SEH hash values as its hash, and the “bottom components” $c_L^0, s_L^0, c_R^0, s_R^0, \pi_L^0, \pi_R^0$ and $\pi_{\mathcal{L}}$ computed on the data x and the bottom SEH hash values as its hash.

The reader may immediately notice that this idea does not work directly (even syntactically), since the RAM delegation proofs π_L^0, π_R^0 are given with respect to the bottom layer SEH hash values. Without those bottom layer SEH hash values in the SNAP proof, the verifier cannot verify these proofs! To resolve this issue, we further Merkle hash the bottom layer SEH hash values to rt_L^1, rt_R^1 using a collision-resistant hash function, where rt_L^1 (resp. rt_R^1) is the Merkle hash of the left (resp. right) children. Then we add the Merkle hash roots rt_L^1, rt_R^1 as part of the bottom components in the SNAP proof. Now we can locally open any position in the instance by opening two levels of Merkle trees: we first locally open the Merkle trees for rt_L^1 or rt_R^1 , and then we locally open the bottom layer SEH hash values. In this way, we fix this issue by generating the proofs π_L^0, π_R^0 using this 2-level local opening with respect to the roots rt_L^1, rt_R^1 respectively. Then their verification only needs the short values rt_L^1, rt_R^1 , which we can include with the proof.

Another issue with the current idea is that the verifier cannot verify the commitments of proximity in the top components c_L^1, c_R^1 , as it does not have direct oracle access to the underlying data, which consists of the bottom SEH hash values. However, since each of the bottom SEH hash values only depends on a few bits of the data, the verifier can simulate such oracle queries to these hash values via oracle queries to the data (at the cost of an appropriate blow-up in the number of queries).

To prove the soundness of this construction, we only need to show that the consistency proofs π_L^1, π_R^1 in the top components ensure that any local opening to the top layer SEH hash values in the rate-1 SEH Merkle tree must be consistent with the syndrome decoding result. To see this, we first extract the bottom layer SEH hash values from the top components of the proof using the same strategy as in the toy case, and argue that they must be consistent with the syndrome decoding of the bottom layer of SEH hash values derived from the verifier’s instance x . Now, since the reduction learns the bottom layer SEH hash values from the syndrome decoding, the situation is reduced to the previous toy case. Hence, we can further extract from the bottom layer SEH and argue that the extracted values must be consistent with the syndrome decoding of the instance x . In this way, we have proven that any local opening to the top layer SEH hash values must be consistent with the syndrome decoded x' .

The above idea can achieve $n/4 \cdot (1 + o(1))$ SNAP proof length. To further shrink the SNAP proof length, we can build a full Merkle tree using the rate-1 SEH, and this construction achieves the optimal SNAP proof size.

Summary of the Full Construction. We give a summary of the full SNAP construction as follows.

- **Prover.** The prover builds a Merkle tree using rate-1 SEH, where the instance x forms the leaves. We only build the tree until the number of hash values is less than $\sqrt{n}$.

At each level i of the Merkle tree, the prover splits the SEH hash values at level i as left children and right children, and denotes them as L_i, R_i , respectively. Then the prover computes a commitment of proximity c_L^i for L_i with distance parameter $\sqrt{n}/2$, and a syndrome s_L^i for L_i for $O(\sqrt{n})$ Hamming errors, and a Merkle hash rt_L^i of L_i using a collision-resistant hash function. The prover then attaches a RAM delegation π_L^i to prove that c_L^i, s_L^i, rt_L^i are computed

correctly from L_i with respect to rt_L^{i+1}, rt_R^{i+1} . The prover does the same thing for the right child and obtains $c_R^i, s_R^i, rt_R^i, \pi_R^i$.

The SNAP proof contains the top layer of the SEH hash values, $(c_b^i, s_b^i, rt_b^i, \pi_b^i)$ for all levels and $b \in \{L, R\}$, and finally a RAM delegation proof $\pi_{\mathcal{L}}$ claiming that the instance is in the language $\mathcal{L}$ using the top layer SEH hash values as the digest.

- **Verifier.** The verifier checks the correctness of all RAM delegation proofs $\{\pi_b^i\}$ and $\pi_{\mathcal{L}}$, with respect to the digests indicated in the prover algorithm. It also runs commitment of proximity verification for all c_b^i , where every oracle access to the intermediate SEH hash values is emulated by querying all the leaves under that SEH hash value, and recompute the intermediate SEH hash value on its own.

During the security reduction, the entire Merkle tree is recovered starting from the top layer given in the proof, by applying syndrome decoding on each layer computed from the verifier's instance x . Each time, given layer $i + 1$ of the Merkle tree is recovered correctly, the security proof shows layer i is also correctly recovered with respect to its hashes rt_L^i, rt_R^i . At the end, the leafs of the Merkle tree, i.e. the instance x' "in prover's mind" is recovered and is guaranteed to be close to the verifier's instance x . Then by soundness of $\pi_{\mathcal{L}}$ we conclude that $x' \in \mathcal{L}$, which guarantees the overall soundness of the SNAP.

We point out a technical issue on verifier's query complexity: to verify a commitment of proximity at an intermediate layer of the SEH Merkle tree, the verifier needs to simulate the oracle access to the corresponding layer of the SEH Merkle tree. However, the verifier only has oracle access to the instance, i.e., leaves of the Merkle tree. Therefore, to read one bit in a layer of s tree nodes, the verifier needs to make $O(n/s)$ queries to the leaf layer and recompute the subtree. We achieve $O(\sqrt{n})$ query complexity by setting the distance parameters to be $\sqrt{n}/2$, for commitments of proximity of *all* layers. Therefore, to verify a commitment of proximity of a layer of size s , it requires $O(s/\sqrt{n})$ queries to the SEH hash values on that layer, which can be simulated by $O((s/\sqrt{n}) \cdot n/s) = O(\sqrt{n})$ queries to the instance x .

Generic Upgrade to adaptive SNAP for NP. Our adaptive SNAP for P can be used to transform any adaptive SNARG for NP into an adaptive SNAP for NP. The transformation is neat: we delegate the SNARG verification to the prover, and ask the prover to additionally send a SNAP proof for it. The soundness of this transform relies on our ability to efficiently extract x' "in prover's mind" from the instance x and SNAP proof π in the soundness reduction. Then we can apply the soundness of the adaptive SNARG for NP and the fact that x' is accepted by the SNARG verifier. We conclude that x' is in the NP language, and therefore x is close to that language.

We note that if the SNARG for NP is fully succinct, then the resulting SNAP for NP is tight in terms of verification efficiency.

2.3 Fully Succinct Non-Adaptive SNAP

We show how to construct non-adaptive SNAPs for NP with εn distance where both proof size and query complexity are some fixed polynomials $\text{poly}(\lambda)$ independent of n .

Take a language $\mathcal{L} \in \text{NP}$ with a corresponding NP relation $\mathcal{R}_{\mathcal{L}}$. Our SNAP for $\mathcal{L}$ will have a special type of correlation-intractable hash function (CIH) h in the CRS. The prover takes the statement $x \in \mathcal{L}$ and computes $S = h(x)$, where S is interpreted as a small (security parameter)

size subset of locations inside x . The prover gives out the values $(S, y = x_S)$ and then uses an adaptively secure SNARG for NP to construct a proof π that there exists some $(x, w) \in \mathcal{R}_{\mathcal{L}}$ such that $h(x) = S$ and $x_S = y$. The full SNAP proof consists of (S, y, π) . The SNAP verifier with oracle access to some statement x' first runs the SNARG verifier to check the proof π for the statement (S, y) and then also checks that $x'_S = y$ by making $|S|$ queries to x' .

Suppose a non-adaptive prover picks some x' that is ε -far from the language and is able to give a valid SNAP proof for x' . We rely on a special form of statistical correlation-intractability that allows us to indistinguishably change the description of h in the CRS to depend on x' and ensure that for any x that is ε -far from x' , if $S = h(x)$ then $x_S \neq x'_S$.⁴ Note that this step crucially relies on soundness being non-adaptive. With the above change, we ensure that the only way that the prover can succeed in convincing the SNAP verifier is by giving a SNARG proof π for a false statement (S, y) . This is because the SNAP verifier checks $y = x'_S$ and hence for any x such that $h(x) = S$ and $x_S = y = x'_S$ it must be that x is within distance less than ε of x' and hence not in $\mathcal{L}$.

2.4 Negative Result for Non-Adaptive SNAP for P

We showed how to construct fully succinct non-adaptive SNAPs for NP by using SNARGs for NP. Could we construct such SNAPs for just P under weaker assumptions that don't imply SNARGs for NP? Unfortunately, we show this is impossible since such SNAPs for P imply non-trivial SNARGs for NP. Namely, we prove that there exists a constant $\varepsilon > 0$ for which any non-adaptive SNAP for P with distance εn , query complexity q and communication complexity c such that $qc = o(n)$ would imply a non-trivial non-adaptive SNARG for NP where the proof length is sublinear in witness length.⁵ Therefore, we cannot beat $\sqrt{n}$ verification efficiency for non-adaptive SNAPs for P, without relying on assumptions that imply non-trivial non-adaptive SNARG for NP.

Our proof is inspired by the proof of the negative result of [KR15]. However, their result only achieves SNARGs for NP with *average-case completeness* for random statements sampled from some a priori fixed distribution. We extend their ideas to build SNARGs for general NP with the standard notion of completeness for worst-case statements.

Take a language $\mathcal{L} \in \text{NP}$ with a corresponding NP relation $\mathcal{R}_{\mathcal{L}}$. To give a SNARG proof for some statement $x \in \mathcal{L}$ using witness w , the high level idea is to give a SNAP proof for the language $\mathcal{L}' \in \text{P}$ consisting of statements $x' = (\tilde{x}, \tilde{w})$, where $\tilde{x}$ is an encoded versions of x and $\tilde{w}$ is an encoded version of w such that $(x, w) \in \mathcal{R}_{\mathcal{L}}$. We specify what the encodings are.

To guarantee *soundness*, we just need the encoding $\tilde{x} = \text{ECC}(x)$ to be an error-correcting code with relative distance 2ε and set the size $|\tilde{x}| \geq |\tilde{w}|$. This ensures that, if $x \notin \mathcal{L}$ is a false statement, then any value $x' = (\tilde{x}, \cdot)$ where $\tilde{x} = \text{ECC}(x)$ is at an absolute distance $2\varepsilon|\tilde{x}|$ away from the language $\mathcal{L}'$, which translates to relative distance $2\varepsilon|\tilde{x}|/(|\tilde{x}| + |\tilde{w}|) \geq \varepsilon$.

The more difficult issue is *completeness*. To verify the SNAP proof, a SNAP verifier needs oracle access to $x' = (\tilde{x}, \tilde{w})$, but the SNARG verifier only knows the statement x . While the SNARG verifier can compute $\tilde{x} = \text{ECC}(x)$ itself, it does not know w and hence cannot compute $\tilde{w}$. To handle this, we ensure that only a small number of random locations of the encoding $\tilde{w}$ depend on w and the rest are fixed independently of w . Intuitively, this should ensure that the SNAP verifier that only queries a small number of locations in x' is unlikely to query any of the “special locations” in $\tilde{w}$ that

⁴This can be seen as a special instance of statistical CI for efficient product relations, which was constructed by [HLR21] assuming statistical CI for efficiently enumerable relations, which was in turn constructed by a recent work [HW24] assuming LWE and iO.

⁵But the CRS and the verification time can be large.

depend on w , and hence the SNARG verifier can run it and answer its queries accurately without knowing w . But an additional difficulty is that the locations queried by the SNAP verifier depend on the SNAP proof, which in turn depends on $\tilde{w}$ and hence may reveal something about the special locations. Therefore, we need the extra property that, even if we see all of $\tilde{w}$, we cannot tell which special locations depend on w . It turns out that there is an easy way to achieve this.

We define encodings of w as additive secret sharing of w : $\tilde{w} = (s_1, \dots, s_m)$ such that $\oplus_{i \in [m]} s_i = w$. Initially, we place m uniformly random and independent values s_i in the crs of the SNARG (To make crs shorter, in fact, we only need to put a short seed in crs and generate all of s_i by a PRG). The SNARG prover picks a random location $j \leftarrow [m]$ and modifies s_j to $s'_j = w \oplus (\oplus_{i \neq j} s_i)$ to derive the encoding $\tilde{w} = (s_1, \dots, s'_j, \dots, s_m)$ of the witness w . It then creates a SNAP proof π for the statement $x' = (\tilde{x}, \tilde{w}) \in \mathcal{L}'$. Notice that only the random location j of $\tilde{w}$ depends on w , and the rest are fixed by the crs. Moreover, j is perfectly hidden even given $\tilde{w}$. Therefore, the SNAP proof (which depends on $\tilde{w}$ but not on the SNARG crs) cannot reveal anything about j . The SNARG verifier runs the SNAP verifier and uses x to answer its queries inside $\tilde{x}$, while using the crs values $s_1, \dots, s_m$ to answer its queries inside $\tilde{w}$. The only time a completeness error occurs is if the SNAP verifier makes a query inside s_j , but this only happens with a small probability. We can amplify the completeness error to negligible by running many copies of the above scheme and accepting if any of them verify.

3 Preliminaries

Notations. We denote the set of all positive integers up to n as $[n] = \{1, \dots, n\}$. Throughout this paper, unless specified, all polynomials we consider are positive polynomials. For any finite set S , $x \leftarrow S$ denotes a uniformly random element x from the set S . Similarly, for any distribution $\mathcal{D}$, $x \leftarrow \mathcal{D}$ denotes an element x drawn from the distribution $\mathcal{D}$.

For any two strings $x, y \in \{0, 1\}^*$ of the same length, we denote their Hamming distance as $\Delta(x, y)$. For any language $\mathcal{L}$ and string x , we write $\Delta(x, \mathcal{L}) = \min_{y \in \mathcal{L} \cap \{0, 1\}^{|x|}} \Delta(x, y)$.

For any two distributions $\mathcal{D}_1, \mathcal{D}_2$, we denote their statistical distance as $\Delta(\mathcal{D}_1, \mathcal{D}_2) := \frac{1}{2} \sum_x |\Pr_{\mathcal{D}_1}[x] - \Pr_{\mathcal{D}_2}[x]|$.

3.1 Error Correcting Codes

An (n, m, d) -code C is an ECC (for hamming errors) with codeword length n , message length m . The hamming distance between every pair of codewords in C is at least d .

In this paper, we utilize the fact that there exists constant $\varepsilon > 0$, such that for some constant C , a binary $(n = C \cdot m, m, \varepsilon n)$ -code with explicit construction exists for all $m \in \mathbb{N}$.

3.2 Batch Arguments (BARGs)

The following preliminaries on seBARGs are due to [BBKLP23].

A batch argument system BARG for an NP language L enables proving that k NP statements are true with communication cost that is polylogarithmic in k . There are many BARG variants which are known to be existentially equivalent under mild computational assumptions (see, e.g., [CJJ22; KVZ21; KLVW23]). In this work, for simplicity in our constructions, we make use of an argument system for what we call “batch index Turing machine SAT” (BatchTMSAT), defined below.

Definition 3.1. The language BatchIndexTMSAT consists of instances of the form $x = (M, z, k, T)$, where:

- M is the description of a Turing machine.
- z is an input string (to M)
- k is a batch size, and
- T is a running time.

An instance $x = (M, z, k, T)$ is in BatchIndexTMSAT if for all $i \leq i \leq k$, there exists a string w_i such that $M(z, i, w_i)$ accepts within T steps. $\diamond$

We sometimes use the notation $\mathcal{R}(x, i, w_i)$ to denote the relation with instance (x, i) and corresponding witness w_i .

Syntax. A (publicly verifiable and non-interactive) somewhere extractable batch argument system seBARG for BatchIndexTMSAT consists of the following polynomial time algorithms:

$\mathcal{G}(\lambda, 1^n, 1^m, k, i^*) \rightarrow (\text{crs}, \text{td})$. This is a probabilistic polynomial-time algorithm that takes as input a security parameter λ , input length 1^n , witness length 1^m , a parameter k , and an index $i^* \in [k]$. It outputs a common reference string crs along with a trapdoor td .

$\mathcal{P}(\text{crs}, M, z, 1^T, w_1, \dots, w_k) \rightarrow \pi$. This deterministic polynomial-time algorithm takes as input a crs , Turing machine M , input z , runtime 1^T , and k witnesses $w_1, \dots, w_k$. It outputs a proof π .

$\mathcal{V}(\text{crs}, x, \pi) \rightarrow 0/1$. This deterministic polynomial-time algorithm takes as input a crs , instance $x = (M, z, k, T)$, and a proof π . It outputs a bit (1 to accept, 0 to reject).

$\text{Extract}(\text{td}, \pi) \rightarrow w^*$. This deterministic polynomial-time algorithm takes as input a trapdoor td and a proof π . It outputs a single witness w^* .

Definition 3.2 (seBARG). A somewhere-extractable batch argument system $\text{seBARG} = (\mathcal{G}, \mathcal{P}, \mathcal{V}, \text{Extract})$ for BatchIndexTMSAT is required to satisfy the following properties:

Completeness. For any $\lambda \in \mathbb{N}$, any $k(\lambda), n(\lambda), m(\lambda), T(\lambda) \leq 2^\lambda$, any instance $x = (M, z, k, T) \in \text{BatchIndexTMSAT}$ with $|M| + |z| = n$, any corresponding witnesses $w_1, \dots, w_k \in \{0, 1\}^m$ and any index $i^* \in [k]$,

$$\Pr \left[\mathcal{V}(\text{crs}, x, \pi) = 1 : \begin{array}{l} (\text{crs}, \text{td}) \leftarrow \mathcal{G}(\lambda, 1^n, 1^m, i^*), \\ \pi \leftarrow \mathcal{P}(\text{crs}, M, z, 1^T, w_1, \dots, w_k) \end{array} \right] = 1.$$

Efficiency. In the completeness experiment, above, $|\text{crs}| + |\pi| \leq m \cdot \text{poly}(\lambda, \log(knT))$. The running time of the verifier is at most $\text{poly}(|\text{crs}| + |\pi|) + \text{poly}(\lambda) \cdot |x|$.

Index hiding. For any poly-size adversary $\mathcal{A}$ and any polynomials $k(\lambda), n(\lambda), m(\lambda)$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$ and every pair of indices $i_0, i_1 \in [k]$,

$$\Pr \left[\mathcal{A}(\text{crs}) = b : \begin{array}{l} b \leftarrow \{0, 1\}, \\ (\text{crs}, \text{td}) \leftarrow \mathcal{G}(\lambda, 1^n, 1^m, i_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

Somewhere argument of knowledge. For any poly-size adversary $\mathcal{A}$ and any polynomials $k(\lambda), n(\lambda), m(\lambda), T(\lambda)$ there exists a negligible function $\text{negl}(\cdot)$ such that for any index $i^* \in [k]$ and for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \mathcal{V}(\text{crs}, x, \pi) = 1 \\ \wedge (x, i^*, w^*) \notin \mathcal{R} \end{array} : \begin{array}{l} (\text{crs}, \text{td}) \leftarrow \mathcal{G}(\lambda, 1^n, 1^m, i^*) \\ (M, z, \pi) = \mathcal{A}(\text{crs}) \\ w^* \leftarrow \text{Extract}(\text{td}, \pi) \end{array} \right] \leq \text{negl}(\lambda). \quad \diamond$$

Remark 1. We say that a seBARG scheme is T^* -secure, for $T^* = T^*(\lambda)$, if the index hiding property and the somewhere argument of knowledge property hold w.r.t. a $\text{poly}(T^*)$ -size adversary (as opposed to a $\text{poly}(\lambda)$ -size), and the advantage probability is $\text{negl}(T^*)$ (as opposed to $\text{negl}(\lambda)$). In addition, the index hiding property and the somewhere argument of knowledge property are required to hold even if the parameters $k(\lambda), n(\lambda), m(\lambda), T(\lambda)$ are polynomial in $T^*(\lambda)$. We refer to these properties as T^* -index-hiding and T^* -somewhere-argument-of-knowledge, respectively. $\diamond$

Throughout this paper, when we refer to a BARG or seBARG, we will implicitly mean a seBARG for BatchIndexTMSAT.

Remark 2. Given an seBARG, one can naturally extend the definition of the key generation algorithm $\mathcal{G}$ to take as input an index set $I \subset [k]$, as opposed to a single index. $\mathcal{G}(1^\lambda, 1^n, 1^m, I)$ will simply run $\mathcal{G}(1^\lambda, 1^n, 1^m, i)$ for every $i \in I$. The prover algorithm $\mathcal{P}$, given a crs that encodes the $|I|$ indices, will simply generate $|I|$ proofs (one for each crs), and the verifier will check these $|I|$ proofs independently. In this situation, the seBARG trapdoor $\text{td} = (\text{td}_1, \dots, \text{td}_{|I|})$ is written in $|I|$ components, and the scheme inherits a strong form of index hiding: given seBARG.crs, the j th index i_j remains hidden even given all $|I| - 1$ other trapdoors $\text{td}_{i_j'}$.

Theorem 3.3 ([CJJ22; WW22; HJKS22; KLVW23]). There exists an seBARG for BatchIndexTMSAT assuming LWE or DLIN or (subexponential DDH and QR). $\diamond$

Throughout this paper, when we refer to a BARG we implicitly mean a BARG for BatchIndexTMSAT.

3.3 Hash Family with Local Opening

We adapt this section from [KLVW23].

In this section we recall the definition of a hash family with local opening [Mer88].

Syntax. A hash family (HT) with succinct local opening consists of the following algorithms:

$\mathcal{G}(1^\lambda, N) \rightarrow \text{hk}$. This is a probabilistic algorithm that takes as input the security parameter 1^λ in unary and the input length N and outputs a key $\text{hk} \in \{0, 1\}^{\text{poly}(\log N, \lambda)}$. hk is referred to as the hash key.

$\text{Hash}(\text{hk}, x) \rightarrow h$. This is a deterministic poly-time algorithm that takes as input a hash key hk and an input $x \in \{0, 1\}^N$ and outputs a hash value $h \in \{0, 1\}^{\text{poly}(\lambda)}$.

$\text{Open}(\text{hk}, x, j) \rightarrow (b, \rho)$. This deterministic algorithm takes as input hash key hk generated by $\mathcal{G}(1^\lambda, N)$, string $x \in \{0, 1\}^N$ and index $j \in [N]$. It outputs a bit $b \in \{0, 1\}$ and an opening $\rho \in \{0, 1\}^{\text{poly}(\lambda)}$.

$\text{Verify}(\text{hk}, h, j, b, \rho) \rightarrow 0/1$. This deterministic algorithm takes as input hash key hk generated by $\mathcal{G}(1^\lambda, N)$, hash value h , index $j \in [N]$, bit $b \in \{0, 1\}$ and opening ρ . It outputs a verdict bit indicating whether the opening is valid or not.

Definition 3.4 (Properties of HT). A HT family $(\mathcal{G}, \text{Hash}, \text{Open}, \text{Verify})$ is required to satisfy the following properties.

Succinctness. The runtime of $\mathcal{G}(1^\lambda, N)$ is bounded by $\text{poly}(\lambda)$, and the runtime of Verify (and hence the size of h and (b, ρ)) is at most $\text{poly}(\lambda)$.

Opening completeness. For any $\lambda \in \mathbb{N}$, any $N \leq 2^\lambda$, any index $j \in [N]$, and any $x \in \{0, 1\}^N$,

$$\Pr \left[\begin{array}{l} b = x_j \\ \wedge \text{Verify}(\text{hk}, h, j, b, \rho) = 1 \end{array} : \begin{array}{l} \text{hk} \leftarrow \mathcal{G}(1^\lambda, N), \\ h = \text{Hash}(\text{hk}, x), \\ (b, \rho) = \text{Open}(\text{hk}, x, j) \end{array} \right] = 1.$$

Collision resistance w.r.t. opening. For any poly-size adversary $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{hk}, h, j, 0, \rho_0) = 1 \\ \wedge \text{Verify}(\text{hk}, h, j, 1, \rho_1) = 1 \end{array} : \begin{array}{l} \text{hk} \leftarrow \mathcal{G}(1^\lambda, N), \\ (h, j, \rho_0, \rho_1) = \mathcal{A}(\text{hk}) \end{array} \right] \leq \text{negl}(\lambda). \quad \diamond$$

Theorem 3.5 ([Mer88]). Assuming the existence of a collision resistant hash family there exists a hash family with succinct local opening. $\diamond$

3.4 Somewhere Extractable Hash (SEH) Family

In what follows we recall the definition of a somewhere extractable hash (SEH) family based on prior works [HW15; OPWW15].

Syntax. A somewhere extractable hash (SEH) family consists of the following polynomial time algorithms:

$\mathcal{G}(1^\lambda, N, I) \rightarrow (\text{hk}, \text{td})$. This probabilistic algorithm takes as input security parameter 1^λ , message length N , and subset $I \subseteq [N]$. It outputs hash key $\text{hk} \in \{0, 1\}^{|I| \cdot \text{poly}(\lambda)}$ along with trapdoor td .

$\text{Hash}(\text{hk}, x) \rightarrow h$. This deterministic algorithm takes as input hash key hk generated by $\mathcal{G}(1^\lambda, N, I)$ and string $x \in \{0, 1\}^N$. It outputs a hash value $h \in \{0, 1\}^{|I| \cdot \text{poly}(\lambda)}$.

$\text{Open}(\text{hk}, x, j) \rightarrow (b, \rho)$. This deterministic algorithm takes as input hash key hk generated by $\mathcal{G}(1^\lambda, N, I)$, string $x \in \{0, 1\}^N$ and index $j \in [N]$. It outputs a bit $b \in \{0, 1\}$ and an opening $\rho \in \{0, 1\}^{\leq |I| \cdot \text{poly}(\lambda)}$.

$\text{Verify}(\text{hk}, h, j, b, \rho) \rightarrow 0/1$. This deterministic algorithm takes as input hash key hk generated by $\mathcal{G}(1^\lambda, N, I)$, hash value h , index $j \in [N]$, bit $b \in \{0, 1\}$ and opening ρ . It outputs a verdict bit indicating whether the opening is valid or not.

$\text{Extract}(\text{td}, h, j) \rightarrow u$. This deterministic algorithm takes as input trapdoor td generated by $\mathcal{G}(1^\lambda, N, I)$, hash value h , and index $j \in [I]$. It outputs a bit u .

We sometimes use the notation $\text{Extract}(\text{td}, h) = (\text{Extract}(\text{td}, h, j))_{j \in [I]}$.

Definition 3.6 (Somewhere extractable hash family). A somewhere extractable hash family $\text{SEH} = (\mathcal{G}, \text{Hash}, \text{Open}, \text{Verify}, \text{Extract})$ is required to satisfy the following properties:

Efficiency. The size of the hash key hk and the hash value h is at most $|I| \cdot \text{poly}(\lambda)$.

Index hiding. For any poly-size adversary $\mathcal{A}$, any polynomial $N = N(\lambda)$, and any $I_0, I_1 \subseteq [N]$ such that $|I_0| = |I_1|$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\mathcal{A}(\text{hk}) = b : \begin{array}{l} b \leftarrow \{0, 1\} \\ (\text{hk}, \text{td}) \leftarrow \mathcal{G}(1^\lambda, N, I_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

Extraction correctness. For any $N = N(\lambda) \leq 2^\lambda$, any subset $I \subseteq [N]$, and any $x \in \{0, 1\}^N$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[(x_i)_{i \in I} \neq \text{Extract}(\text{td}, h, I) : \begin{array}{l} (\text{hk}, \text{td}) \leftarrow \mathcal{G}(1^\lambda, N, I) \\ h \leftarrow \text{Hash}(\text{hk}, x) \end{array} \right] \leq \text{negl}(\lambda).$$

Opening completeness. For any $\lambda \in \mathbb{N}$, any $N \leq 2^\lambda$, any subset $I \subseteq [N]$, any index $j \in [N]$, and any $x \in \{0, 1\}^N$,

$$\Pr \left[\begin{array}{l} b = x_j \\ \wedge \text{Verify}(\text{hk}, h, j, b, \rho) = 1 \end{array} : \begin{array}{l} (\text{hk}, \text{td}) \leftarrow \mathcal{G}(1^\lambda, N, I), \\ h = \text{Hash}(\text{hk}, x), \\ (b, \rho) = \text{Open}(\text{hk}, x, j) \end{array} \right] = 1.$$

Somewhere statistical binding w.r.t. opening. For any polynomial $N = N(\lambda)$, any subset $I \subseteq [N]$, and any (all-powerful) adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{hk}, h, i^*, b, \rho) = 1 \\ \wedge i^* \in I \wedge b \neq b_{i^*} \end{array} : \begin{array}{l} (\text{hk}, \text{td}) \leftarrow \mathcal{G}(1^\lambda, N, I), \\ (h, i^*, b, \rho) \leftarrow \mathcal{A}(\text{hk}), \\ (b_i)_{i \in I} = \text{Extract}(\text{td}, h) \end{array} \right] \leq \text{negl}(\lambda). \quad \diamond$$

Remark 3. Note that the index hiding property and the somewhere statistically binding w.r.t. opening property of a SEH hash family, implies the following property:

Computational binding w.r.t opening. For any polynomial $N = N(\lambda)$, any $I \subseteq [N]$, and any poly-size adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{hk}, h, i, 0, \rho_0) = 1 \\ \wedge \text{Verify}(\text{hk}, h, i, 1, \rho_1) = 1 \end{array} : \begin{array}{l} (\text{hk}, \text{td}) \leftarrow \mathcal{G}(1^\lambda, N, I), \\ (i, h, \rho_0, \rho_1) \leftarrow \mathcal{A}(\text{hk}) \end{array} \right] \leq \text{negl}(\lambda). \quad \diamond$$

Remark 4 (Rate-1). We say an SEH is a rate-1 SEH, if the SEH further achieves the following efficiency on the hash value size: $|h| \leq |I| + \text{poly}(\lambda)$. $\diamond$

Lemma 3.7. There is a somewhere extractable hash family exists under LWE or DDH or DLIN . $\diamond$

Theorem 3.8 (Rate-1 SEH implied by [DGKV22]). Assuming the hardness of the LWE assumption, there exists a rate-1 somewhere extractable hash family. $\diamond$

3.5 RAM SNARGs

In this section we define RAM SNARGs.

A RAM machine $\mathcal{R}$ is modeled as a deterministic machine with random access to memory of size 2ℓ bits and a local state of size S . At every step, the machine reads or writes a single memory bit and updates its state. For convenience, we think of the input to the RAM machine as a pair $x = (x_{\text{exp}}, x_{\text{imp}})$, where x_{imp} is large and is stored in the random access memory, and x_{exp} is a short explicit input.

Syntax. A RAM SNARG for machine $\mathcal{R}$ is associated with a hash family with local opening

$$\text{HT} = (\text{HT}.\mathcal{G}, \text{HT}.\text{Hash}, \text{HT}.\text{Open}, \text{HT}.\text{Verify}),$$

and consists of the following polynomial time algorithms (relative to HT):

$\mathcal{G}(1^\lambda, T) \rightarrow \text{crs}$. This is a probabilistic algorithm that takes as input a security parameter 1^λ and a time bound T . It outputs a common reference string crs . Note that crs should contain hk generated by $\text{HT}.\mathcal{G}(1^\lambda)$.

$\text{Digest}(\text{hk}, x_{\text{imp}}) \rightarrow \text{digest}$. This is a deterministic algorithm that takes as input a hk and a string x_{imp} . It outputs a digest digest of size $\text{poly}(\lambda)$. This algorithm is fixed by the choice of HT: it computes $\text{digest} = \text{HT}.\text{Hash}(\text{hk}, x_{\text{imp}})$.

$\mathcal{P}(\text{crs}, \text{hk}, \text{vk}, (x_{\text{exp}}, x_{\text{imp}})) \rightarrow \pi$. This is a deterministic algorithm that takes as input a crs , hash keys hk, vk and a pair $x_{\text{exp}}, x_{\text{imp}}$ which consists of a (long) input x_{imp} and a (short) input x_{exp} . It outputs a proof π .

$\mathcal{V}(\text{crs}, \text{vk}, \text{digest}, x_{\text{exp}}, \pi) \rightarrow \{0, 1\}$. This is a deterministic algorithm that takes as input a crs , a hash verification key vk , a digest digest of the long input, a short input x_{exp} and a proof π . It outputs a bit (1 to accept, 0 to reject).

Definition 3.9 (RAM SNARG). A RAM SNARG for machine $\mathcal{R}$ is required to satisfy the following properties:

Completeness. For any $\lambda, n \in \mathbb{N}$ such that $n \leq T \leq 2^\lambda$ and any $x = (x_{\text{exp}}, x_{\text{imp}} \in \{0, 1\}^n)$ such that $\mathcal{R}(x)$ halts within T time steps, we have that

$$\Pr \left[\begin{array}{l} \mathcal{V}(\text{crs}, \text{vk}, \text{digest}_{x_{\text{imp}}}, x_{\text{exp}}, \pi) = 1 \\ \wedge \mathcal{R}(x) = 1 \end{array} : \begin{array}{l} \text{crs} \leftarrow \mathcal{G}(1^\lambda, T) \\ (\text{hk}, \text{vk}) \leftarrow \text{HT}.\mathcal{G}(1^\lambda, n) \\ \pi = \mathcal{P}(\text{crs}, \text{hk}, \text{vk}, x) \\ \text{digest}_{x_{\text{imp}}} = \text{Digest}(\text{hk}, x_{\text{imp}}) \end{array} \right] = 1.$$

Efficiency. In the completeness experiment above, the length of crs and the proof π as well as running time of $\mathcal{V}$ is at most $|x_{\text{exp}}|\text{poly}(\lambda)$.

Soundness. For any poly-size adversary $\mathcal{A}$ and any polynomial $T = T(\lambda)$, $n = n(\lambda)$ there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \mathcal{V}(\text{crs}, \text{vk}, \text{digest}, x_{\text{exp}}, \pi) = 1 \\ \wedge \text{Digest}(\text{hk}, x_{\text{imp}}) = \text{digest} \\ \wedge \mathcal{R}(x_{\text{exp}}, x_{\text{imp}}) = 0 \end{array} : \begin{array}{l} \text{crs} \leftarrow \mathcal{G}(1^\lambda, T) \\ (\text{hk}, \text{vk}) \leftarrow \text{HT}.\mathcal{G}(1^\lambda, n) \\ (\text{digest}, x_{\text{exp}}, x_{\text{imp}}, \pi) \leftarrow \mathcal{A}(\text{crs}, \text{hk}, \text{vk}) \end{array} \right] \leq \text{negl}(\lambda). \quad \diamond$$

Remark 5. On efficiency, we require *linear* dependency on $|x_{\text{exp}}|$, instead of polynomial dependency as previous literature. This requirement can be made without loss of generality, as we can take the whole x as the implicit input, and computing a digest of x_{exp} supporting local opening only takes $|x_{\text{exp}}|\text{poly}(\lambda)$ time (assuming CRHF). $\diamond$

Theorem 3.10 ([CJJ22]). For any hash family with succinct local opening, there exists a RAM SNARG associated with it assuming the existence of seBARG. $\diamond$

3.6 Flexible RAM SNARGs with Partial Input Soundness

In this section, we define flexible RAM SNARG with partial input soundness [KLVW23]. We first define the syntax of fully-local SEH hash family.

Syntax. A fully-local SEH (flSEH) hash family associated with extractable sets $S = \{S_N\}_{N \in \mathbb{N}}$ consists of the following algorithms:

$\mathcal{G}(1^\lambda, N, I) \rightarrow (\text{hk}, \text{td})$. This probabilistic algorithm takes as input security parameter 1^λ , message length N , and subset $I \in S_N$. It outputs hash key $\text{hk} \in \{0, 1\}^{\text{poly}(\lambda)}$ along with trapdoor td ⁶.

$\text{Hash}(\text{hk}, x) \rightarrow h$. This deterministic algorithm takes as input hash key hk generated by $\mathcal{G}(1^\lambda, N, I)$ and string $x \in \{0, 1\}^N$. It outputs a hash value $h \in \{0, 1\}^{|I| \cdot \text{poly}(\lambda)}$.

$\text{Digest}(\text{hk}, h) \rightarrow \text{rt}$. This deterministic poly-time algorithm that takes as input hk and a hash value h , and outputs a (short) digest rt of hk of length $\text{poly}(\lambda)$.

$\text{Open}(\text{hk}, x, j) \rightarrow (b, \rho)$. This deterministic algorithm takes as input hash key hk generated by $\mathcal{G}(1^\lambda, N, I)$, string $x \in \{0, 1\}^N$ and index $j \in [N]$. It outputs a bit $b \in \{0, 1\}$ and an opening $\rho \in \{0, 1\}^{\text{poly}(\lambda)}$.

$\text{Verify}(\text{hk}, \text{rt}, j, b, \rho) \rightarrow 0/1$. This deterministic algorithm takes as input hash key hk generated by $\mathcal{G}(1^\lambda, N, I)$, a (short) rt , index $j \in [N]$, bit $b \in \{0, 1\}$ and opening ρ . It outputs a verdict bit indicating whether the opening is valid or not.

$\text{Extract}(\text{td}, h, j) \rightarrow u$. This deterministic algorithm takes as input trapdoor td generated by $\mathcal{G}(1^\lambda, N, I)$, hash value h , and index $j \in [I]$. It outputs a bit u .

Remark 6. We often abuse notation and let

$$\text{Hash}(\text{hk}, x) = \text{rt},$$

where $\text{rt} = \text{Digest}(\text{hk}, v = \text{Hash}(\text{hk}, x))$. $\diamond$

Definition 3.11 (flSEH). A flSEH hash family is required to satisfy the following properties:

Efficiency. The running time of $\text{Verify}(\text{hk}, \text{rt}, j, b, \rho)$ is $\text{poly}(\lambda)$. The running time of $\mathcal{G}$ and the length of a hash value h is $|I| \cdot \text{poly}(\lambda)$. Checking if $I \in S_N$ can be done in $\text{poly}(\lambda, N)$ time.

⁶Different to [KLVW23], we only consider succinct hk in this paper (rather than an additional verification key). This is achievable when S_N only contains a few index sets.

Opening completeness. For any $\lambda \in \mathbb{N}$, any $N \leq 2^\lambda$, any $I \in S_N$, any index $j \in [N]$, and any $x \in \{0, 1\}^N$,

$$\Pr \left[\begin{array}{l} b = x_j \\ \wedge \text{Verify}(\text{hk}, \text{h}, j, b, \rho) = 1 \end{array} : \begin{array}{l} \text{hk} \leftarrow \mathcal{G}(1^\lambda, N, I), \\ \text{h} = \text{Hash}(\text{hk}, x), \\ (b, \rho) = \text{Open}(\text{hk}, x, j) \end{array} \right] = 1.$$

Opening soundness w.r.t. digest. For any poly-size adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} I \in S_N \\ \wedge \text{Verify}(\text{hk}, \text{rt}, j, 0, \rho_0) = 1 \\ \wedge \text{Verify}(\text{hk}, \text{rt}, j, 1, \rho_1) = 1 \end{array} : \begin{array}{l} (1^N, I) \leftarrow \mathcal{A}_1(1^\lambda), \\ \text{hk} \leftarrow \mathcal{G}(1^\lambda, N, I), \\ (\text{rt}, j, \rho_0, \rho_1) = \mathcal{A}_2(\text{hk}) \end{array} \right] \leq \text{negl}(\lambda).$$

Opening soundness w.r.t. hash value. For any poly-size adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} I \in S_N \wedge j \in I \\ \wedge \text{Verify}(\text{hk}, \text{rt}, j, 1 - x_j, \rho^*) = 1 \end{array} : \begin{array}{l} (1^N, I) \leftarrow \mathcal{A}_1(1^\lambda), \\ (\text{hk}, \text{td}) \leftarrow \mathcal{G}(1^\lambda, N, I), \\ (\text{h}, j, \rho^*) = \mathcal{A}_2(\text{hk}), \\ \text{rt} = \text{Digest}(\text{hk}, \text{h}), \\ (x_i)_{i \in I} = \text{Extract}(\text{td}, \text{h}) \end{array} \right] \leq \text{negl}(\lambda).$$

Index hiding. For any poly-size adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} I_0, I_1 \in S_N \wedge |I_0| = |I_1| \\ \wedge \mathcal{A}_2(\text{hk}) = b \end{array} : \begin{array}{l} (1^N, I_0, I_1) \leftarrow \mathcal{A}_1(1^\lambda), \\ b \leftarrow \{0, 1\}, \\ \text{hk} \leftarrow \mathcal{G}(1^\lambda, N, I_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda). \quad \diamond$$

Then we define flexible RAM SNARGs.

Syntax. Let $\mathcal{R}$ be a RAM machine. A flexible RAM SNARG for $\mathcal{R}$ associated with a hash family with local opening:

$$\text{HT} = (\text{HT}.\mathcal{G}, \text{HT}.\text{Hash}, \text{HT}.\text{Open}, \text{HT}.\text{Verify})$$

consists of the following algorithms (relative to HT):

$\mathcal{G}(1^\lambda, T) \rightarrow \text{crs}$. This is a randomized (not necessarily poly-time) setup algorithm that takes as input a security parameter 1^λ and a time bound T , and outputs a common reference string crs .

$\text{Digest}(\text{hk}, x) \rightarrow \text{rt}$. This is a deterministic polynomial-time algorithm that takes as input a key hk generated by $\text{HT}.\mathcal{G}(1^\lambda)$ and a string x , and outputs the digest $\text{rt} = \text{HT}.\text{Hash}(\text{hk}, x)$. This algorithm is fixed by the choice of HT.

$\mathcal{P}(\text{crs}, \text{hk}, (x_{\text{exp}}, x_{\text{imp}})) \rightarrow \pi$. This is a deterministic polynomial-time prover that takes as input a crs, a hash key hk , and a pair $(x_{\text{exp}}, x_{\text{imp}})$ that $\mathcal{R}(x_{\text{exp}}, x_{\text{imp}}) = 1$ which consists of a (short) explicit input x_{exp} and a (long) implicit input x_{imp} , and outputs a proof π .

$\mathcal{V}(\text{crs}, \text{hk}, \text{rt}, x_{\text{exp}}, \pi) \rightarrow \{0, 1\}$. This is a deterministic polynomial-time verifier that takes as input a crs, a HT key hk , a digest rt of the (long) implicit input, a (short) explicit input x_{exp} , and a proof π , and outputs 1 (accept) or 0 (reject).

Definition 3.12. A flexible RAM SNARG associated with a hash family with local opening HT satisfies all properties from Definition 3.9. In addition, it satisfies the following partial-input soundness property:

Partial-input soundness. If the underlying hash family with local opening is a flSEH family, denoted by

$$\text{flSEH} = (\text{flSEH}.\mathcal{G}, \text{flSEH}.\text{Hash}, \text{flSEH}.\text{Digest}, \text{flSEH}.\text{Open}, \text{flSEH}.\text{Verify}, \text{flSEH}.\text{Extract}),$$

and flSEH is somewhere extractable on the indices sets $S = \{S_N \subseteq 2^{[N]}\}_N$, then for any poly-size adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ and any polynomial $T = T(\lambda)$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} I \in S_N \\ \wedge \mathcal{V}(\text{crs}, \text{vk}, \text{rt}, x_{\text{exp}}, \pi) = 1 \\ \wedge \text{rt} = \text{flSEH}.\text{Digest}(\text{vk}, v) \\ \wedge \mathcal{R}(x_{\text{exp}}, x_{\text{imp}}, T) = 0 \\ \text{and does not read any location in } [N] \setminus I \end{array} : \begin{array}{l} \text{crs} \leftarrow \mathcal{G}(1^\lambda, T) \\ (1^N, I) = \mathcal{A}_1(\text{crs}) \\ (\text{hk}, \text{vk}, \text{td}) \leftarrow \text{flSEH}.\mathcal{G}(1^\lambda, N, I) \\ (v, \text{rt}, x_{\text{exp}}, \pi) = \mathcal{A}_2(\text{crs}, \text{hk}, \text{vk}) \\ \text{define } x_{\text{imp}} \in \{0, 1\}^N : \\ (b_i)_{i \in I} = \text{flSEH}.\text{Extract}(\text{td}, v) \\ \forall i \in I, (x_{\text{imp}})_i = b_i \\ \forall i \in [N] \setminus I, (x_{\text{imp}})_i = 0 \end{array} \right] \leq \text{negl}(\lambda).$$

◇

Remark 7. Our definition of partial-input soundness is slightly more general than [KLVW23]: we consider the underlying hash function as flSEH, instead of SEH. ◇

Theorem 3.13. Assume the existence of seBARG. For any flSEH family, there exists a flexible RAM SNARG associated with this hash family. ◇

Although not been explicitly pointed out, this theorem directly follows from the seBARG to SNARG transform in previous literature [KPY19; CJJ22; KLVW23].

3.7 Indistinguishability Obfuscation for Circuits

We define the notion of indistinguishability obfuscation (iO) for circuits [BGI+01; GGH+13] below.

Definition 3.14 (Indistinguishability Obfuscator (iO) for Circuits). A uniform PPT algorithm iO is called an ε -secure indistinguishability obfuscator for a circuit family $\{C_\lambda\}_{\lambda \in \mathbb{N}}$, where C_λ consists of circuits C of the form $C : \{0, 1\}^\ell \rightarrow \{0, 1\}$, if the following holds:

- **Completeness:** For every $\lambda \in \mathbb{N}$, every $C \in C_\lambda$, every input $x \in \{0, 1\}^\ell$, where $\ell = \ell(\lambda)$ is the input length of C , we have that

$$\Pr [C'(x) = C(x) : C' \leftarrow \text{iO}(1^\lambda, C)] = 1$$

- **ε -Indistinguishability:** For any PPT distinguisher D , for all sufficiently large $\lambda \in \mathbb{N}$, for all pairs of circuits $C_0, C_1 \in C_\lambda$ such that $C_0(x) = C_1(x)$ for all inputs $x \in \{0, 1\}^\ell$, where $\ell = \ell(\lambda)$ is the input length of C_0, C_1 , we have:

$$\left| \Pr [D(1^\lambda, \text{iO}(1^\lambda, C_0)) = 1] - \Pr [D(1^\lambda, \text{iO}(1^\lambda, C_1)) = 1] \right| \leq \varepsilon$$

If ε is negligible in λ then we refer to iO as a secure indistinguishability obfuscator. $\diamond$

3.8 SNARG for NP

Consider the computation of a non-deterministic polynomial-time Turing machine M . Our definition allows for a long common reference string, maintaining verifier efficiency by requiring that $\text{crs} = (\text{crs}_\mathcal{P}, \text{crs}_\mathcal{V})$ be separated into prover and verifier components (where the verifier component is required to be short).

Syntax. A (publicly verifiable) SNARG for M consists of the following polynomial-time algorithms:

$\mathcal{G}(1^\lambda, 1^n, 1^m) \rightarrow (\text{crs}_\mathcal{P}, \text{crs}_\mathcal{V})$. This is a probabilistic polynomial-time algorithm that takes as input a security parameter 1^λ , input length 1^n , witness length 1^m . It outputs a common reference string $\text{crs} = (\text{crs}_\mathcal{P}, \text{crs}_\mathcal{V})$.

$\mathcal{P}(\text{crs}_\mathcal{P}, x, w) \rightarrow \pi$. This polynomial-time algorithm takes as input $\text{crs}_\mathcal{P}$, input $x \in \{0, 1\}^n$ and its associated witness $w \in \{0, 1\}^m$. It outputs a proof π .

$\mathcal{V}(\text{crs}_\mathcal{V}, x, \pi) \rightarrow 0/1$. This deterministic polynomial-time algorithm takes as input a $\text{crs}_\mathcal{V}$, the instance $x \in \{0, 1\}^n$ and a proof π . It outputs a bit (1 to accept, 0 to reject).

Definition 3.15 (SNARG). A succinct non-interactive argument system $\text{SNARG} = (\mathcal{G}, \mathcal{P}, \mathcal{V})$ for non-deterministic polynomial-time Turing machine M is required to satisfy the following properties (and one of adaptive/non-adaptive soundness):

Completeness. For any $\lambda \in \mathbb{N}$, any $n(\lambda), m(\lambda) \leq 2^\lambda$, any $x \in \{0, 1\}^n, w \in \{0, 1\}^m$ such that $M(x, w) = 1$,

$$\Pr \left[\mathcal{V}(\text{crs}_\mathcal{V}, x, \pi) = 1 : \begin{array}{l} (\text{crs}_\mathcal{P}, \text{crs}_\mathcal{V}) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m), \\ \pi \leftarrow \mathcal{P}(\text{crs}_\mathcal{P}, x, w) \end{array} \right] = 1.$$

Non-adaptive Soundness. For any poly-size adversary $\mathcal{A}$ and any polynomials $n(\lambda), m(\lambda)$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$ and every $x \in \{0, 1\}^n$ where $x \notin \mathcal{L}_M$,

$$\Pr \left[\mathcal{V}(\text{crs}_\mathcal{V}, x, \pi^*) = 1 : \begin{array}{l} (\text{crs}_\mathcal{P}, \text{crs}_\mathcal{V}) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m) \\ \pi^* = \mathcal{A}(\text{crs}_\mathcal{P}, \text{crs}_\mathcal{V}, x) \end{array} \right] \leq \text{negl}(\lambda).$$

Adaptive Soundness. For any poly-size adversary $\mathcal{A}$ and any polynomials $n(\lambda), m(\lambda)$ there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \mathcal{V}(\text{crs}_{\mathcal{V}}, x^*, \pi^*) = 1 \\ \wedge x^* \notin \mathcal{L}_M \end{array} : \begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m) \\ (x^*, \pi^*) = \mathcal{A}(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \end{array} \right] \leq \text{negl}(\lambda). \quad \diamond$$

Theorem 3.16 ([SW21]). Assume the existence of iO and one way function. Then there exists a non-adaptively sound SNARG for any non-deterministic polynomial-time Turing machine M . Furthermore, the proof size and verifier's running time is $\text{poly}(\lambda)$, independent of the size or running time of machine M . $\diamond$

Theorem 3.17 ([WW24a]). Assume the existence of sub-exponentially secure iO and sub-exponentially secure one way function. Then there exists an adaptively sound SNARG for any non-deterministic polynomial-time Turing machine M . The proof size is $\text{poly}(\lambda)$, and the length of $\text{crs}_{\mathcal{V}}$ is $\text{poly}(n, \lambda)$. $\diamond$

We note that the length of $\text{crs}_{\mathcal{V}}$ in adaptive SNARG of [WW24a] can be reduced to $\text{poly}(\lambda)$, by delegating the original verification to the prover by a RAM SNARG.

Lemma 3.18. Assume the existence of RAM SNARG, sub-exponentially secure iO and sub-exponentially secure one way function. Then there exists an adaptively sound SNARG for any non-deterministic polynomial-time Turing machine M . The proof size is $\text{poly}(\lambda)$, the length of $\text{crs}_{\mathcal{V}}$ is $\text{poly}(\lambda)$, and verifier's running time is $n\text{poly}(\lambda)$. $\diamond$

Proof sketch. We construct a new SNARG system: to generate the common reference string, we additionally sample the crs_{RAM} of the RAM SNARG, and set the new $\text{crs}'_{\mathcal{P}} = (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}, \text{crs}_{\text{RAM}})$, and $\text{crs}'_{\mathcal{V}}$, being composed of crs_{RAM} plus a digest of $\text{crs}_{\mathcal{V}}$. The new prover $\mathcal{P}'$ additionally generates the RAM SNARG proof π_{RAM} for $\mathcal{V}(\text{crs}_{\mathcal{V}}, x, \pi) = 1$. The new verifier $\mathcal{V}'$ only checks π_{RAM} , by running the RAM SNARG verification algorithm with x as the explicit input, and $\text{crs}_{\mathcal{V}}$ as the implicit input. It is easy to see the new SNARG is still adaptively sound, and satisfies all efficiency requirements. $\square$

4 Commitment of Proximity

In this section we introduce commitment of proximity, a succinct commitment that allows sublinear verification of commitment given oracle access to the underlying message as the opening. In contrast to standard commitments, we don't require hiding property but ask for a special version of binding called binding of proximity: no efficient adversary can generate two messages x, x' with a large distance (i.e. $\Delta(x, x')$ is greater than some threshold), such that x' is a accepting opening to the commitment of x .

This section is organized as follows: we introduce the main tool of unbalanced bipartite expanders in subsection 4.1, then give the definition and our construction in subsection 4.2 and 4.3 respectively.

4.1 Preliminary: Unbalance Bipartite Expander

Definition 4.1 (Bipartite expander). A bipartite graph $(X \cup Y, E)$ is a (K, A) -expander, if for all sets $S \subseteq X$ of at most K left vertices, the neighborhood $N(S) := \{u \in Y : \exists v \in S, (v, u) \in E\}$ is of size at least $A \cdot |S|$. $\diamond$

Theorem 4.2 (See Theorem 4.5 of [KT22]). For every positive integers $N, K_{\max} \leq N$, all $\varepsilon > 0$, and $\frac{16 \log\left(\frac{\log N}{\varepsilon}\right)}{\sqrt{\log K_{\max}}} \leq \alpha \leq 1$, there is a $M \leq D \cdot K_{\max}^{1+\alpha}$, $D = O((\log N \cdot \log K_{\max}/\varepsilon)^{1+1/\alpha})$, and an explicit bipartite graph $(X \cup Y, E)$ with the following properties:

1. $|X| = N, |Y| = M$, every node in X has degree D . There is a function $\Gamma : [N] \times [D] \rightarrow [M]$ that computes the neighbors of each left node in time $\text{polylog}(N)$.
2. Each node $u \in Y$ has degree $\deg(u) \leq O(ND/M)$. Furthermore, we can compute $\deg(u)$ in time $\text{polylog}(N)$, and compute the list of its neighbors $N(u)$ in time $\deg(u) \cdot \text{polylog}(N)$.
3. The graph is a $(K_{\max}, (1 - \varepsilon)D)$ expander. $\diamond$

Remark 8. Property 2 is not explicitly stated in [KT22], but can be derived from their construction: their construction is first pick some finite field $\mathbb{F}_q$, and treat each node as a vector in $\mathbb{F}_q^n$. So $X \subset \mathbb{F}_q^n$ where $n = \log N$, and $Y \subset \mathbb{F}_q^{s+2}$ for another parameter $s \in \mathbb{N}$. Every $x \in X$ has its y -th ($0 \leq y < q$) neighbor as $(y, A_y \cdot x) \in \mathbb{F}_q^{s+2}$, where $A_y \in \mathbb{F}_q^{(s+1) \times n}$ is a matrix with rank $s+1$. Therefore, given any right node (y, v) , the neighbors of it is just $\{u \in X : A_y u = v\}$.

Furthermore, their construction allows to take X as any subset of $\mathbb{F}_q^n$, so we set X to be the N lexicographically smallest vectors. Therefore by Gaussian elimination, we can compute the size of $\{u \in X : A_y u = v\}$ or enumerate the number of vectors in it efficiently, and $|\{u \in X : A_y u = v\}| \leq \frac{N}{q^{s+1}} + 1 = O(ND/M)$. $\diamond$

Corollary 4.3. For every positive integers $N, K_{\max} \leq N$, there is $M = K_{\max} \cdot N^{o(1)}$, $D = N^{o(1)}$, and a bipartite graph $(X \cup Y, E)$ with the following properties:

1. $|X| = N, |Y| = M$, every node in X has degree D .
2. Each node $u \in Y$ has degree $\deg(u) \leq O(ND/M)$. Furthermore, we can compute $\deg(u)$ in time $\text{polylog}(N)$, and compute the list of its neighbors $N(u)$ in time $\deg(u) \cdot \text{polylog}(N)$.
3. The graph is a $(K_{\max}, 1/2)$ expander.

Furthermore, the bipartite graph is explicit in the sense that we have the following algorithms:

$\text{RightSize}(N, K_{\max}) \rightarrow M$. This deterministic algorithm returns $M = |Y|$ of the corresponding bipartite graph, runs in $\text{polylog}(N)$ times.

$\text{Neighbor}(N, K_{\max}, y) \rightarrow N(y) \subseteq [N]$. This deterministic algorithm additionally takes the index of right side node $y \in [M]$ and returns the set of its neighbors $N(y) \subseteq [N]$. The algorithm runs in time $\frac{N}{K_{\max}} \cdot N^{o(1)}$. $\diamond$

Proof. Take $\varepsilon = 1/2$, and $\alpha = \frac{32 \log \log N}{\sqrt{\log K_{\max}}}$. Then by Theorem 4.2, the corresponding expander graph has degree

$$D = O((\log N \cdot \log K_{\max})^{1+1/\alpha}) \leq (\log N)^{O(\sqrt{\log N})} = O(N^{\log \log N / \sqrt{\log N}}) = N^{o(1)},$$

and the number of right nodes is

$$M \leq DK_{\max}^{1+\alpha} \leq N^{o(1)} \cdot K_{\max} \cdot 2^{O(\log \log N \cdot \sqrt{\log K_{\max}})} \leq N^{o(1)} \cdot K_{\max} \cdot 2^{O(\log \log N \cdot \sqrt{\log N})} = K_{\max} \cdot N^{o(1)}. \quad \square$$

4.2 Definitions

Syntax. A commitment of proximity family consists of the following polynomial time algorithms:

$\mathcal{G}(1^\lambda, n, k) \rightarrow \text{ck}$. This probabilistic algorithm takes as input security parameter 1^λ , message length n and distance parameter k . It outputs commitment key $\text{ck} \in \{0, 1\}^{\text{poly}(\lambda)}$.

$\text{Commit}(\text{ck}, x) \rightarrow c$. This deterministic algorithm takes as input a hash key ck generated by $\mathcal{G}(1^\lambda, n, k)$ and string $x \in \{0, 1\}^n$. It outputs a commitment value $c \in \{0, 1\}^*$.

$\text{Verify}^x(\text{ck}, c) \rightarrow 0/1$. This randomized polynomial-time algorithm takes as input commitment key ck generated by $\mathcal{G}(1^\lambda, n, k)$, commitment value c , and has oracle access to the message $x \in \{0, 1\}^n$. It outputs a verdict bit indicating whether x is close to the committed message.

Definition 4.4 (Commitment of Proximity). A commitment of proximity CoP = $(\mathcal{G}, \text{Commit}, \text{Verify}, \text{DVerify})$ is required to satisfy the following properties:

Completeness. For any $\lambda, n, k \in \mathbb{N}$, and any $x \in \{0, 1\}^n$,

$$\Pr \left[\text{Verify}^x(\text{ck}, c) = 1 : \begin{array}{l} \text{ck} \leftarrow \mathcal{G}(1^\lambda, n, k) \\ c \leftarrow \text{Commit}(\text{ck}, x) \end{array} \right] = 1,$$

where probability is taken over randomness of $\mathcal{G}$ and Verify .

Computational binding of proximity. For any $n(\lambda), k(\lambda)$ that are polynomial in λ , and any poly-size adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{Verify}^{x^*}(\text{ck}, c) = 1 \\ \wedge \Delta(x, x^*) > k \end{array} : \begin{array}{l} \text{ck} \leftarrow \mathcal{G}(1^\lambda, n, k) \\ (x, x^*) \leftarrow \mathcal{A}(\text{ck}) \\ c \leftarrow \text{Commit}(\text{ck}, x) \end{array} \right] \leq \text{negl}(\lambda),$$

where probability is taken over randomness of $\mathcal{G}, \mathcal{A}$ and Verify . $\diamond$

4.3 Construction

A commitment of proximity can be constructed from any collision resistant hash function, plus the bipartite expander in Corollary 4.3.

Lemma 4.5. Assuming the existence of collision resistant hash functions, there exists a commitment of proximity scheme with commitment size $k \cdot n^{o(1)} \text{poly}(\lambda)$, verification query complexity $\frac{n}{k} \cdot n^{o(1)} \text{poly}(\lambda)$, and verification time $(k + \frac{n}{k}) \cdot n^{o(1)} \text{poly}(\lambda)$. $\diamond$

Remark 9 (Optimality of the Parameters). We note that our construction achieves almost-optimal parameters, following from two observations:

- Verification query complexity must be $\Omega(n/k)$. In particular, consider the attacker that randomly flips k bits of x to generate another instance x' . To query at least one of the flipped bits (as otherwise binding of proximity is broken) requires $\Omega(n/k)$ query complexity.

- The attack to adaptively secure SNAP shown in Section 6 also pertains to commitment of proximity, which implies a lower bound that commitment size times query complexity must be $\Omega(n)$ for any $k < (1 - O(1))n$.

Now fix verification query complexity as the best possible, i.e., $\Theta(n/k)$. By the second observation, commitment size is $\Omega(k)$. Therefore, verification time is $\Omega(n/k + k)$, since it upper bounds both query complexity and commitment size. This is exactly the parameter we achieve up to $n^{o(1)}\text{poly}(\lambda)$ factors.

In the above parameter setting, we focus on optimizing query complexity. In the case where optimal query complexity is not required, the above construction can be invoked with an altered distance parameter to achieve better (and near-optimal) parameters. For example, for any desired distance k^* , we can choose the distance parameter as $k \leq k^*$ to set the query complexity to be $\frac{n}{k} \geq \frac{n}{k^*}$. Our construction gives the optimal commitment size and verification time given this query complexity (up to $n^{o(1)}\text{poly}(\lambda)$ factors). $\diamond$

In our construction, we use $\text{CRHF}.\mathcal{G}$ and $\text{CRHF}.\text{Hash}$ to represent the key generation and hash evaluation algorithms of the CRHF; we will also use RightSize and Neighbor as stated in Corollary 4.3. Also, we assume $k \leq n$, as otherwise the commitment is trivial (e.g. take $\text{Commit}(\text{ck}, x) = x$).

- $\mathcal{G}(1^\lambda, n, k)$ samples CRHF key $\text{hk} \leftarrow \text{CRHF}.\mathcal{G}(1^\lambda)$, and outputs $\text{ck} = (\text{hk}, n, k, l = \text{RightSize}(n, k))$.
- $\text{Commit}(\text{ck}, x)$ does the following:
 1. Parse ck into hk, n, k, l .
 2. For each $i \in [l]$:
 - (a) Compute $N_i \leftarrow \text{Neighbor}(n, k, i)$, the set of neighbors of i in the bipartite expander.
 - (b) Compute $h_i \leftarrow \text{CRHF}.\text{Hash}(\text{hk}, x_{N_i})$.
 3. Output $c = (h_1, \dots, h_l)$.
- $\text{Verify}^x(\text{ck}, c)$ does the following:
 1. Parse ck into hk, n, k, l , and c into $h_1, \dots, h_l$.
 2. Repeat for $\frac{2l}{k}\lambda$ times:
 - (a) Sample $i \leftarrow [l]$, and compute $N_i \leftarrow \text{Neighbor}(n, k, i)$.
 - (b) Compute x_{N_i} by making oracle queries to each index in N_i .
 - (c) Compute $h_i^* = \text{CRHF}.\text{Hash}(\text{hk}, x_{N_i})$.
 - (d) If $h_i \neq h_i^*$, then output 0.
 3. Output 1.

Analysis The completeness guarantee follows immediately. The efficiency can also be easily observed from the construction:

- Step 1 of Verify takes time $kn^{o(1)}\text{poly}(\lambda)$.
- As $l/k = n^{o(1)}$ by Corollary 4.3, Verify runs its step 3 for $n^{o(1)}\text{poly}(\lambda)$ times.

- In each repetition of step 3, Verify makes $\frac{n}{k}n^{o(1)}$ oracle queries, and computing N_i, h_i^* takes time $\frac{n}{k}n^{o(1)} \cdot \text{poly}(\lambda)$. Here we assume wlog that computing $h_i^* = \text{CRHF.Hash}(\text{hk}, x_{N_i})$ takes time $|x_{N_i}| \text{poly}(\lambda)$.

Now we analyze the computational binding of proximity guarantee.

Lemma 4.6. For any $\lambda, n, k \in \mathbb{N}, l = \text{RightSize}(n, k), \text{ck} \leftarrow \mathcal{G}(1^\lambda, n, k)$, and any $x, x^* \in \{0, 1\}^n$ with $\Delta(x, x^*) > k$, and let $N_i \leftarrow \text{Neighbor}(n, k, i)$ for all $i \in [l]$, then

$$\left| \left\{ i \in [l] : x_{N_i} \neq x_{N_i}^* \right\} \right| \geq \frac{k}{2}. \quad \diamond$$

Proof. Take $S \subset [n]$ to be any subset of $\{i \in [n] : x_i \neq x_i^*\}$ of size k . By Corollary 4.3, since the underlying expander is a $(k, 1/2)$ expander, $|N(S)| \geq k/2$. For any $i \in N(S) \subset [l]$, we have $N_i \cap S \neq \emptyset \implies x_{N_i} \neq x_{N_i}^*$. $\square$

Proof (Lemma 4.5). We prove a stronger version of computational binding of proximity here:

$$\Pr \left[\begin{array}{l} \Pr[\text{Verify}^{x^*}(\text{ck}, c) = 1] > 2^{-\lambda} \\ \wedge \Delta(x, x^*) > k \end{array} : \begin{array}{l} \text{ck} \leftarrow \mathcal{G}(1^\lambda, n, k) \\ (x, x^*) \leftarrow \mathcal{A}(\text{ck}) \\ c \leftarrow \text{Commit}(\text{ck}, x) \end{array} \right] \leq \text{negl}(\lambda).$$

For any $(x, x^*) \leftarrow \mathcal{A}(\text{ck})$, take $c = (h_1, \dots, h_l) \leftarrow \text{Commit}(\text{ck}, x), c^* = (h_1^*, \dots, h_l^*) \leftarrow \text{Commit}(\text{ck}, x^*)$.

If both $\Pr[\text{Verify}^{x^*}(\text{ck}, c) = 1] > 2^{-\lambda}$ and $\Delta(x, x^*) > k$, then:

- $\left| \left\{ i \in [l] : h_i \neq h_i^* \right\} \right| < \frac{k}{2}$, as otherwise by the algorithm of Verify, Verify accepts with probability at most $\left(1 - \frac{k}{2l}\right)^{2l/k \cdot \lambda} \leq 2^{-\lambda}$.
- $\left| \left\{ i \in [l] : x_{N_i} \neq x_{N_i}^* \right\} \right| \geq \frac{k}{2}$, by Lemma 4.6.

Therefore, there exists $i \in [l]$ such that $x_{N_i} \neq x_{N_i}^*$ but $\text{CRHF.Hash}(\text{hk}, x_{N_i}) = h_i = h_i^* = \text{CRHF.Hash}(\text{hk}, x_{N_i}^*)$. Therefore, we can construct an adversary to CRHF with the same success probability. $\square$

Note that collision resistant hash function is implied by LWE, and the bipartite expander specified by Lemma 4.5 exists unconditionally by Corollary 4.3.

Corollary 4.7. Assuming LWE, there exists a commitment of proximity scheme with commitment size $k \cdot n^{o(1)} \text{poly}(\lambda)$, verification query complexity $\frac{n}{k} \cdot n^{o(1)} \text{poly}(\lambda)$, and verification time $(k + \frac{n}{k}) \cdot n^{o(1)} \text{poly}(\lambda)$. $\diamond$

5 Succinct Non-interactive Arguments of Proximity (SNAPs)

In this section, we define and construct succinct non-interactive arguments of proximity (SNAPs). We give a construction of SNAP for P with tight verification efficiency from LWE (see Theorem 5.4, 5.5), and we show our construction of SNAP for P can be generically upgraded to SNAP for NP, using any adaptive SNARG for NP (see Theorem 5.6).

5.1 Definitions

In this section, we give a formal definition of succinct non-interactive argument of proximity (SNAP).

Consider the computation of a non-deterministic polynomial-time Turing machine M (when considering SNAP for P , we take M as a deterministic Turing machine), and let $\mathcal{L}_M$ be the language of instances accepted by M . Our definition allows for a long common reference string, maintaining verifier efficiency by requiring that $\text{crs} = (\text{crs}_P, \text{crs}_V)$ be separated into prover and verifier components (where the verifier component is required to be short). The soundness of SNAP is guaranteed for instances ρ -far from the language, for some distance $\rho = \rho(n)$.

Syntax. A (publicly verifiable) SNAP for M with distance $\rho(\cdot)$ consists of the following polynomial-time algorithms:

$\mathcal{G}(1^\lambda, 1^n, 1^m) \rightarrow (\text{crs}_P, \text{crs}_V)$. This is a probabilistic polynomial-time algorithm that takes as input a security parameter 1^λ , input length 1^n , witness length 1^m . It outputs a common reference string $\text{crs} = (\text{crs}_P, \text{crs}_V)$.

$\mathcal{P}(\text{crs}_P, x, w) \rightarrow \pi$. This polynomial-time algorithm takes as input crs_P , input $x \in \{0, 1\}^n$ and its associated witness $w \in \{0, 1\}^m$. It outputs a proof π .

$\mathcal{V}^x(\text{crs}_V, \pi) \rightarrow 0/1$. This *randomized* polynomial-time algorithm takes as input a crs_V , a proof π , and have oracle access to the instance $x \in \{0, 1\}^n$. It outputs a bit (1 to accept, 0 to reject).

Remark 10. When M is a deterministic Turing machine, we will regard $m = 0$ and omit terms relating to m, w . $\diamond$

Remark 11. The verifier $\mathcal{V}$ must be randomized in the adaptive setting, as otherwise the adaptive adversary can run the verifier by itself and learn which bits the verifier will query. The attacker can then flip the rest unqueried bits. $\diamond$

Definition 5.1 (SNAP). A succinct non-interactive argument of proximity system $\text{SNAP} = (\mathcal{G}, \mathcal{P}, \mathcal{V})$ for non-deterministic polynomial-time Turing machine M with distance $\rho(\cdot)$ is required to satisfy the following properties (and one of adaptive/non-adaptive soundness):

Completeness. For any $\lambda \in \mathbb{N}$, any $n(\lambda), m(\lambda) \leq 2^\lambda$, any $x \in \{0, 1\}^n, w \in \{0, 1\}^m$ such that $M(x, w) = 1$,

$$\Pr \left[\mathcal{V}(\text{crs}_V, x, \pi) = 1 \ : \ \begin{array}{l} (\text{crs}_P, \text{crs}_V) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m), \\ \pi \leftarrow \mathcal{P}(\text{crs}_P, x, w) \end{array} \right] = 1.$$

Non-adaptive Soundness. For any poly-size adversary $\mathcal{A}$ and any polynomials $n(\lambda), m(\lambda)$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$ and every $x \in \{0, 1\}^n$ where $\Delta(x, \mathcal{L}_M) > \rho(n)$,

$$\Pr \left[\mathcal{V}^x(\text{crs}_V, \pi^*) = 1 \ : \ \begin{array}{l} (\text{crs}_P, \text{crs}_V) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m) \\ (\pi^*) = \mathcal{A}(\text{crs}_P, \text{crs}_V, x) \end{array} \right] \leq \text{negl}(\lambda).$$

Adaptive Soundness. For any poly-size adversary $\mathcal{A}$ and any polynomials $n(\lambda), m(\lambda)$ there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \mathcal{V}^{x^*}(\text{crs}_{\mathcal{V}}, \pi^*) = 1 \\ \wedge \Delta(x^*, \mathcal{L}_M) > \rho(n) \end{array} : \begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m) \\ (x^*, \pi^*) = \mathcal{A}(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \end{array} \right] \leq \text{negl}(\lambda). \quad \diamond$$

We also consider a stronger extractability soundness guarantee: there is an extractor that given an instance x and its SNAP proof, the extractor computes a yes instance $x' \in \mathcal{L}$ such that $\Delta(x, x') \leq \rho$. In this paper, we only consider extractability in the context of adaptive soundness.

Definition 5.2 (Adaptive Instance Extractable SNAP). We call a SNAP system with distance $\rho(\cdot)$ an adaptive instance extractable SNAP, if it additionally has an extraction algorithm that can extract $x' \in \mathcal{L}_M$ such that $\Delta(x, x') \leq \rho$, from any accepting (x, π) .

$\text{Extract}(\text{crs}_{\mathcal{V}}, x, \pi) \rightarrow x'$. This deterministic polynomial-time algorithm takes as input a $\text{crs}_{\mathcal{V}}$, a proof π , and the instance $x \in \{0, 1\}^n$. It outputs an instance $x' \in \{0, 1\}^n$.

Adaptive Instance Extractability. For any poly-size adversary $\mathcal{A}$ and polynomials $n(\lambda), m(\lambda)$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \mathcal{V}^{x^*}(\text{crs}_{\mathcal{V}}, \pi^*) = 1 \\ \wedge (x' \notin \mathcal{L}_M \vee \Delta(x^*, x') > \rho(n)) \end{array} : \begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m) \\ (x^*, \pi^*) = \mathcal{A}(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \\ x' \leftarrow \text{Extract}(\text{crs}_{\mathcal{V}}, x^*, \pi^*) \end{array} \right] \leq \text{negl}(\lambda). \quad \diamond$$

Remark 12. We note that adaptive soundness is directly implied by adaptive instance extractability. Our construction of adaptive SNAP for P satisfies adaptive instance extractability, whereas all negative results in this paper apply to standard soundness notions of SNAP *without instance extractability*. $\diamond$

We also consider SNAPS for RAM computation, in which one part of the instance (the explicit part x_{exp}) is directly taken as input by the verifier, while the rest of the input (the implicit part x_{imp}) is accessed via an oracle during verification. Typically, the verification time has a polynomial dependency on $|x_{\text{exp}}|$, but is sublinear in $|x_{\text{imp}}|$.

Let machine M be a deterministic RAM machine with input $x = (x_{\text{exp}}, x_{\text{imp}})$.

Syntax. A (publicly verifiable) SNAP for RAM machine M with distance $\rho(\cdot)$ consists of the following polynomial-time algorithms:

$\mathcal{G}(1^\lambda, 1^s, 1^n) \rightarrow (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}})$. This is a probabilistic polynomial-time algorithm that takes as input a security parameter 1^λ , (short) input length 1^s , (long) input length 1^n . It outputs a common reference string $\text{crs} = (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}})$.

$\mathcal{P}(\text{crs}_{\mathcal{P}}, (x_{\text{exp}}, x_{\text{imp}})) \rightarrow \pi$. This polynomial-time algorithm takes as input $\text{crs}_{\mathcal{P}}$, a pair $x = (x_{\text{exp}}, x_{\text{imp}})$ which consists of a (short) input $x_{\text{exp}} \in \{0, 1\}^s$ and a (long) input $x_{\text{imp}} \in \{0, 1\}^n$. It outputs a proof π .

$\mathcal{V}^{x_{\text{imp}}}(\text{crs}_{\mathcal{V}}, x_{\text{exp}}, \pi) \rightarrow 0/1$. This randomized polynomial-time algorithm takes as input a $\text{crs}_{\mathcal{V}}$, the short input x_{exp} , a proof π , and have oracle access to the instance x_{imp} . It outputs a bit (1 to accept, 0 to reject).

$\text{Extract}(\text{crs}_{\mathcal{V}}, (x_{\text{exp}}, x_{\text{imp}}), \pi) \rightarrow x_{\text{imp}}^*$. This deterministic polynomial-time algorithm takes as input a $\text{crs}_{\mathcal{V}}$, the input pair $(x_{\text{exp}}, x_{\text{imp}})$, and a proof π . It outputs a (true) long input x_{imp}^* .

In this paper we only concern SNAPs for RAM computation with adaptive instance extractability.

Definition 5.3 (SNAP for RAM Computation). A succinct non-interactive argument of proximity system $\text{SNAP} = (\mathcal{G}, \mathcal{P}, \mathcal{V}, \text{Extract})$ for deterministic polynomial-time RAM machine M with distance $\rho(\cdot)$ is required to satisfy the following properties:

Completeness. For any $\lambda \in \mathbb{N}$, any $s(\lambda), n(\lambda) \leq 2^\lambda$, any $x_{\text{exp}} \in \{0, 1\}^s, x_{\text{imp}} \in \{0, 1\}^n$ such that $M(x_{\text{exp}}, x_{\text{imp}}) = 1$,

$$\Pr \left[\mathcal{V}^{x_{\text{imp}}}(\text{crs}_{\mathcal{V}}, x_{\text{exp}}, \pi) = 1 \quad : \quad \begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^s, 1^n), \\ \pi \leftarrow \mathcal{P}(\text{crs}_{\mathcal{P}}, (x_{\text{exp}}, x_{\text{imp}})) \end{array} \right] = 1.$$

Adaptive Instance Extractability. For any poly-size adversary $\mathcal{A}$ and polynomials $s(\lambda), n(\lambda)$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \mathcal{V}^{x_{\text{imp}}^*}(\text{crs}_{\mathcal{V}}, x_{\text{exp}}^*, \pi^*) = 1 \\ \wedge (M(x_{\text{exp}}^*, x_{\text{imp}}^*) = 0 \vee \Delta(x_{\text{imp}}^*, x_{\text{imp}}') > \rho(n)) \end{array} : \begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^s, 1^n) \\ ((x_{\text{exp}}^*, x_{\text{imp}}^*), \pi^*) \leftarrow \mathcal{A}(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \\ x_{\text{imp}}' \leftarrow \text{Extract}(\text{crs}_{\mathcal{V}}, (x_{\text{exp}}^*, x_{\text{imp}}^*), \pi^*) \end{array} \right] \leq \text{negl}(\lambda). \quad \diamond$$

5.2 Main Theorem Statement

We construct a SNAP for RAM computation with almost-tight verification efficiency, based on a wide range of standard cryptographic assumptions, including polynomial hardness of LWE. The construction is exhibited in section 5.4.

Theorem 5.4. Assume the existence of a rate-1 SEH scheme (see section 3.4) and a seBARG scheme (Definition 3.2), any polynomial-time RAM machine $\mathcal{R}$ has an adaptive instance extractable SNAP with distance $\rho(n)$ that satisfies:

- The length of $\text{crs} = (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}})$ is $\text{poly}(\lambda)$.
- The proof length is $\rho(n) \cdot n^{o(1)} \text{poly}(\lambda)$.
- The verifier runs in $\left(\rho n^{o(1)} + \frac{n^{1+o(1)}}{\rho} + s \right) \text{poly}(\lambda)$ time, with query complexity $\frac{n^{1+o(1)}}{\rho} \text{poly}(\lambda)$. Here s is the length of explicit input, and n is the length of implicit input. $\diamond$

Taking $\rho(n) = \sqrt{n}$, we achieve SNAP for P with almost $\sqrt{n}$ verification efficiency.

Theorem 5.5. With the same assumptions as in Theorem 5.4, any polynomial-time RAM machine $\mathcal{R}$ has an adaptive instance extractable SNAP with distance $\sqrt{n}$, in which $|\text{crs}| = \text{poly}(\lambda)$, and verification time is $(n^{1/2+o(1)} + s) \text{poly}(\lambda)$ (s is the length of explicit input, and n is the length of implicit input). $\diamond$

Our next result is a generic upgrade from SNAP for RAM computation with adaptive instance extractability (and in particular, our SNAP construction) to SNAP for NP, via any adaptive SNARG for NP. The upgrade also preserves verification efficiency. This upgrade is shown in Section 5.6.

Theorem 5.6. Assume the existence of an adaptive instance extractable SNAP with distance $\rho(n)$ and verification time $(n^{c+o(1)} + s)\text{poly}(\lambda)$ for any polynomial-time RAM machine, plus adaptively sound SNARG for NP where $|\text{crs}_V|, |\pi| \leq L(\lambda, n, m)$, there exists an adaptive instance extractable SNAP for NP with distance $\rho(n)$ and verification time $(n^{c+o(1)} + L)\text{poly}(\lambda)$. Furthermore, $|\text{crs}_V|$ in the resulting SNAP for NP is bounded by L plus $|\text{crs}_V|$ in the original SNAP for RAM. $\diamond$

Recently adaptive SNARG for NP with proof length $\text{poly}(\lambda)$ is constructed from sub-exponentially secure iO and one way functions [WW24a]. By additionally applying a RAM SNARG to delegate verification to the prover, we can further reduce the length of $|\text{crs}_V|$ to $\text{poly}(\lambda)$ (see Lemma 3.18). Combining our SNAP for P (Theorem 5.5) with this SNARG for NP, by Theorem 5.6, we obtain the following SNAP for NP.

Theorem 5.7. Assume polynomial hardness of LWE, sub-exponentially secure one way functions and sub-exponentially secure iO, there exists an adaptive instance extractable SNAP for NP with distance $\sqrt{n}$, whose $|\text{crs}_V| = \text{poly}(\lambda)$ and verification time is $n^{1/2+o(1)}\text{poly}(\lambda)$. $\diamond$

5.3 Additional Preliminary: Secure Sketch

We use secure sketch [DORS08] as one building block of our construction. A secure sketch scheme allows one to compute a sketch for an instance x , such that given any x' with small distance with respect to x , x can be efficiently recovered with the help of the sketch. While typically the sketch is only required to hide the original message x , in certain construction the sketch can be even succinct, having a sublinear length in $|x|$. We utilize such “succinct” secure sketch in this construction, which is given in, e.g. Construction 3 of [DORS08] via syndrome decoding.

Syntax. A secure sketch system consists of two polynomial time algorithms:

$\text{Sketch}(x, d) \rightarrow \text{ske}$. This deterministic polynomial-time algorithm takes as input the message $x \in \{0, 1\}^*$ and designed distance d . It outputs the sketch $\text{ske} \in \{0, 1\}^*$.

$\text{Recover}(x', \text{ske}) \rightarrow x$. This deterministic polynomial-time algorithm takes as input the tampered message $x' \in \{0, 1\}^*$ and sketch ske . It outputs the original message $x \in \{0, 1\}^*$.

Definition 5.8 (Secure Sketch). A secure sketch system $(\text{Sketch}, \text{Recover})$ is required to satisfy the following properties:

Succinctness. For any $x \in \{0, 1\}^*, d \in \mathbb{N}$, the length of $\text{ske} \leftarrow \text{Sketch}(x, d)$ is at most $d \cdot \text{polylog}(|x|)$.

Completeness. For any $n, d \in \mathbb{N}$ and $x, x' \in \{0, 1\}^n$ such that $\Delta(x, x') \leq d$, we have

$$\text{Recover}(x', \text{Sketch}(x, d)) = x. \quad \diamond$$

5.4 Construction

Let $\mathcal{R}$ be the polynomial-time RAM machine in the theorem statement.

We define a candidate SNAP for $\mathcal{R}$ using the following building blocks:

- A rate-1 SEH scheme $\text{SEH} = (\text{SEH}.\mathcal{G}, \text{SEH}.\text{Hash}, \text{SEH}.\text{Open}, \text{SEH}.\text{Verify}, \text{SEH}.\text{Extract})$, defined in section 3.4.
- A somewhere extractable batch argument system seBARG , as defined in section 3.2.
- A secure sketch system $(\text{Sketch}, \text{Recover})$ as defined in section 5.3.
- A commitment of proximity system $\text{CoP} = (\text{CoP}.\mathcal{G}, \text{CoP}.\text{Commit}, \text{CoP}.\text{Verify}, \text{CoP}.\text{DVerify})$ as constructed by Lemma 4.5. We note that collision resistant hash function is implied by SEH.
- A hash family with succinct local opening $\text{HT} = (\text{HT}.\mathcal{G}, \text{HT}.\text{Hash}, \text{HT}.\text{Open}, \text{HT}.\text{Verify})$, as defined in section 3.3. We assume computation of hash value on strings of length n takes time $n\text{poly}(\lambda)$.
- A RAM SNARG system $\text{SNARG}_{\mathcal{R}}$ as defined in section 3.5, for the machine $\mathcal{R}$ in the theorem statement, which takes input $(x_{\text{exp}}, x_{\text{imp}})$ and runs for polynomial steps.

$\text{SNARG}_{\mathcal{R}}$ is associated with a hash family with succinct local opening that takes some $\text{hk}^{\text{HT}} \leftarrow \text{HT}.\mathcal{G}$ as hash key. The exact construction is explained later.

- A flexible RAM SNARG system SNARG_P as defined in section 3.6 for the following machine: $\mathcal{R}_P$ takes input $x_{\text{exp}} = (n, i, t, \text{ck}, \text{hk}^{\text{HT}}, \{c_t, \text{ske}_t, \text{rt}_t\}_{t \in \{0,1\}})$ and $x_{\text{imp}} = \text{layer} \in \{0,1\}^{2^{k-i}N_i}$ (see below for definition of k, N_i). It outputs 1 if, take $\text{layer}_t = \text{layer}|_{S_{i,t}}$, we have $c_t = \text{CoP}.\text{Commit}(\text{ck}, \text{layer}_t)$, $\text{ske}_t = \text{Sketch}(\text{layer}_t, \rho)$ and $\text{rt}_t = \text{HT}.\text{Hash}(\text{hk}^{\text{HT}}, \text{layer}_t)$.

SNARG_P is associated with the fSEH that can be regarded as a concatenation of SEH and HT: informally, $\text{fSEH}.\text{Hash}(\text{hk}, x)$ treats $x = \{x_j \in \{0,1\}^N\}_{j \in [m]}$ (here N, m are some parameters) and computes $v = \{\text{rt}_j = \text{SEH}.\text{Hash}(\text{hk}^{\text{SEH}}, x_j)\}_{j \in [m]}$; $\text{fSEH}.\text{Digest}(\text{vk}, v)$ computes $\text{HT}.\text{Hash}(\text{hk}^{\text{HT}}, \cdot)$ on $v = \{\text{rt}_j\}_{j \in [m]}$. We will give a more detailed exposition of fSEH later in the prover algorithm. To this fSEH , $\text{hk} = \text{vk} = (\text{hk}^{\text{SEH}}, \text{hk}^{\text{HT}})$ are generated by $\text{SEH}.\mathcal{G}$ and $\text{HT}.\mathcal{G}$ respectively.

Our construction makes use of the following conventions:

- Assume wlog that $n = 2^k$ for some $k \in \mathbb{N}$, and we also take $d \in \mathbb{N}$ as the smallest number that $2^{k-d} \leq \rho(n)$. We will build the Merkle tree for the bottom d layers.
- We choose a sequence of polynomials $N_0, N_1, \dots, N_d$ as the Merkle tree node size by layers. We take $N_0 = 1$, and for each $0 \leq i < d$, $N_{i+1} = N_i + \text{poly}(\lambda)$ as the SEH hash size while being extractable on N_i bits. Note that $N_d = d\text{poly}(\lambda) + 1$ is still a polynomial in λ .
- We partition a layer i of the Merkle tree into two parts: “left children” and “right children”, by whether a node is the left child or right child of its father. Formally, treating the layer as a string of length $2^{k-i}N_i$, for $t \in \{0,1\}$ define $S_{i,t} = ([tN_i, (t+1)N_i) + 2N_i\mathbb{Z}) \cap \{0, \dots, 2^{k-i}N_i - 1\}$.

Our SNAP construction works as follows:

The Setup Algorithm. $\text{SNAP}.\mathcal{G}(1^\lambda, 1^s, 1^n)$ does the following:

1. For all $i \in \{0, \dots, d-1\}$, do the following:
 - Sample a hash key $\text{hk}_i^{\text{SEH}} \leftarrow \text{SEH}.\mathcal{G}(1^\lambda, 2N_i, I_i^*)$, where $I_i^* \subset [2N_i]$ is of size N_i . We arbitrarily take I_i^* as $[N_i]$.
 - Sample commitment key $\text{ck}_i \leftarrow \text{CoP}.\mathcal{G}(1^\lambda, 1^{2^{k-i}N_i/2}, 1^{\rho/2})$.
2. For all $i \in \{0, \dots, d\}$, sample hash key $\text{hk}_i^{\text{HT}} \leftarrow \text{HT}.\mathcal{G}(1^\lambda, 2^{2d-i}N_i/2)$.
3. Sample common reference string of the flexible RAM SNARG for $\mathcal{R}_P$: $\text{crs}_P \leftarrow \text{SNARG}_P.\mathcal{G}(1^\lambda, T)$ where $T = \text{poly}(n, \lambda)$ is roughly the maximum time over $0 \leq i \leq d-1$ of computing $\text{CoP}.\text{Commit}(\text{ck}_i, \cdot)$, $\text{Sketch}(\cdot, \rho)$ for any message in $\{0, 1\}^{2^{k-i}N_i/2}$ and computing $\text{HT}.\text{Hash}(\text{hk}_i^{\text{HT}}, \cdot)$.
4. Sample common reference of RAM SNARG for $\mathcal{R}$: $\text{crs}_\mathcal{R} \leftarrow \text{SNARG}_\mathcal{R}.\mathcal{G}(1^\lambda, T')$ where $T' = \text{poly}(s, n)$ is the running time of $\mathcal{R}$ for any input in $\{0, 1\}^s \times \{0, 1\}^n$.

The common reference string is set to be

$$\text{crs}_\mathcal{P} = \text{crs}_\mathcal{V} = (\{\text{hk}_i^{\text{SEH}}, \text{ck}_i\}_{i \in \{0, \dots, d-1\}}, \{\text{hk}_i^{\text{HT}}\}_{i \in \{0, \dots, d\}}, \text{crs}_P, \text{crs}_\mathcal{R}).$$

The Prover Algorithm. $\text{SNAP}.\mathcal{P}(\text{crs}_\mathcal{P}, (x_{\text{exp}}, x_{\text{imp}}))$ operates as follows.

1. Parse $\text{crs}_\mathcal{P} = (\{\text{hk}_i^{\text{SEH}}, \text{ck}_i\}_{i \in \{0, \dots, d-1\}}, \{\text{hk}_i^{\text{HT}}\}_{i \in \{0, \dots, d\}}, \text{crs}_P, \text{crs}_\mathcal{R})$.
2. Starting from x_{imp} , compute a Merkle tree $\{\text{node}_{i,j} \in \{0, 1\}^N\}_{i \in \{0, \dots, d\}, j \in \{0, \dots, 2^{k-i}-1\}}$ using hash function $\text{SEH}.\text{Hash}(\text{hk}_i^{\text{SEH}}, \cdot)$:
 - Partition x_{imp} into 2^k blocks $\text{node}_{0,0}, \dots, \text{node}_{0,2^k-1}$, where each node has size N_0 .
 - For each $1 \leq i \leq d, j \in \{0, \dots, 2^{k-i}-1\}$, let $\text{node}_{i,j} = \text{SEH}.\text{Hash}(\text{hk}_{i-1}^{\text{SEH}}, \text{node}_{i-1,2j} || \text{node}_{i-1,2j+1})$.
 - For each $0 \leq i \leq d$, take $\text{layer}_i \in \{0, 1\}^{2^{k-i}N_i}$ to be the concatenation of $\{\text{node}_{i,j} : 0 \leq j < 2^{k-i}\}$.
3. For each $i \in \{0, \dots, d-1\}$, do the following:
 - (a) For $t \in \{0, 1\}$, set $\text{layer}_{i,t} \in \{0, 1\}^{2^{k-i}N_i/2}$ to be $\text{layer}_i|_{S_{i,t}}$, and then compute $\text{c}_{i,t} \leftarrow \text{CoP}.\text{Commit}(\text{ck}_i, \text{layer}_{i,t})$, $\text{ske}_{i,t} \leftarrow \text{Sketch}(\text{layer}_{i,t}, \rho)$, $\text{rt}_{i,t} \leftarrow \text{HT}.\text{Hash}(\text{hk}_i^{\text{HT}}, \text{layer}_{i,t})$.
 - (b) Prepare the RAM SNARG instance, where $x_{\text{exp}}^{i,t} = (n, i, t, \text{ck}_i, \text{hk}_i^{\text{HT}}, \{\text{c}_{i,t'}, \text{ske}_{i,t'}, \text{rt}_{i,t'}\}_{t' \in \{0,1\}})$, and $x_{\text{imp}}^i = \text{layer}_i$.
 - (c) For $t \in \{0, 1\}$, compute a RAM SNARG proof for the correctness of $\text{c}_{i,t}$, $\text{ske}_{i,t}$ and $\text{rt}_{i,t}$:

$$\pi_{i,t} \leftarrow \text{SNARG}_P.\mathcal{P}(\text{crs}_P, (\text{hk}_i^{\text{SEH}}, \text{hk}_{i+1}^{\text{HT}}), (x_{\text{exp}}^{i,t}, x_{\text{imp}}^i)).$$

Here SNARG_P is associated with the fSEH family that takes $(\text{hk}_i^{\text{SEH}}, \text{hk}_{i+1}^{\text{HT}})$ as hash key. The hash algorithm $\text{Hash}((\text{hk}_i^{\text{SEH}}, \text{hk}_{i+1}^{\text{HT}}), x \in \{0, 1\}^{2^{k-i}N_i})$ is expected to

take layer_i as input and compute $\{\text{rt}_{i+1,t}\}_{t \in \{0,1\}}$ as output: it splits input x into $\{\text{node}_{i,j}\}_{0 \leq j < 2^{k-i}}$, computes $\text{node}_{i+1,j} = \text{SEH.Hash}(\text{hk}_i^{\text{SEH}}, \text{node}_{i,2j} || \text{node}_{i,2j+1})$ for $0 \leq j < 2^{k-(i+1)}$, then assembles $\{\text{node}_{i+1,j}\}$ into $\{\text{layer}_{i+1,t}\}_{t \in \{0,1\}}$, and finally outputs $\{\text{rt}_{i+1,t} = \text{HT.Hash}(\text{hk}_{i+1}^{\text{HT}}, \text{layer}_{i+1,t})\}_{t \in \{0,1\}}$. Note that this flSEH family is extractable on $\{I^{\otimes 2^{k-i}} : |I| \leq N_i \wedge I \subset [2N_i]\}$, including $S_{i,t}$ for $t \in \{0,1\}$.

4. Compute a RAM SNARG proof for $\mathcal{R}(x_{\text{exp}}, x_{\text{imp}}) = 1$: $\pi_{\mathcal{R}} \leftarrow \text{SNARG}_{\mathcal{R}}.\mathcal{P}(\text{crs}_{\mathcal{R}}, \text{hk}_0^{\text{HT}}, (x_{\text{exp}}, x_{\text{imp}}))$. Here $\text{SNARG}_{\mathcal{R}}$ is associated with the hash family that takes hk_0^{HT} as hash key. The hash algorithm $\text{Hash}(\text{hk}_0^{\text{HT}}, x \in \{0,1\}^n)$ is expected to take x_{imp} as input and compute $(\text{rt}_{0,0}, \text{rt}_{0,1})$ as output: it splits input x into $\{x_t = x|_{S_{0,t}}\}_{t \in \{0,1\}}$, and outputs $\{\text{rt}_t = \text{HT.Hash}(\text{hk}_0^{\text{HT}}, x_t)\}_{t \in \{0,1\}}$.
5. Send the proof

$$\pi = (\{\text{node}_{d,j}\}_{j \in \{0, \dots, 2^{k-d}-1\}}, \{c_{i,t}, \text{ske}_{i,t}, \text{rt}_{i,t}, \pi_{i,t}\}_{i \in \{0, \dots, d-1\}, t \in \{0,1\}}, \pi_{\mathcal{R}}).$$

The Verifier Algorithm. $\text{SNAP}.\mathcal{V}^{x_{\text{imp}}}(\text{crs}_{\mathcal{V}}, x_{\text{exp}}, \pi)$ does the following:

1. Parse $\text{crs}_{\mathcal{V}} = (\{\text{hk}_i^{\text{SEH}}, \text{ck}_i\}_{i \in \{0, \dots, d-1\}}, \{\text{hk}_i^{\text{HT}}\}_{i \in \{0, \dots, d\}}, \text{crs}_P, \text{crs}_{\mathcal{R}})$, and parse π into components as described in the prover algorithm.
2. Check that $\text{SNARG}_{\mathcal{R}}.\mathcal{V}(\text{crs}_{\mathcal{R}}, \text{hk}_0^{\text{HT}}, \{\text{rt}_{0,t}\}_{t \in \{0,1\}}, x_{\text{exp}}, \pi_{\mathcal{R}}) = 1$.
3. For $t \in \{0,1\}$, compute $\text{rt}_{d,t} = \text{HT.Hash}(\text{hk}_d^{\text{HT}}, \text{layer}_{d,t})$, where $\text{layer}_{d,t} = \text{layer}_d|_{S_{d,t}}$ and layer_d is the concatenation of $\{\text{node}_{d,j}\}_{j \in \{0, \dots, 2^{k-d}-1\}}$.
4. For each $i \in \{0, \dots, d-1\}$, $t \in \{0,1\}$, do the following:
 - (a) Check that

$$\text{SNARG}_P.\mathcal{V}(\text{crs}_P, (\text{hk}_i^{\text{SEH}}, \text{hk}_{i+1}^{\text{HT}}), \{\text{rt}_{i+1,t'}\}_{t' \in \{0,1\}}, x_{\text{exp}}^{i,t}, \pi_{i,t}) = 1,$$

where $x_{\text{exp}}^{i,t} = (n, i, t, \text{ck}_i, \text{hk}_i^{\text{HT}}, \{c_{i,t'}, \text{ske}_{i,t'}, \text{rt}_{i,t'}\}_{t' \in \{0,1\}})$.

- (b) Run $b \leftarrow \text{CoP.Verify}^{\text{layer}_{i,t}}(\text{ck}_i, c_{i,t})$, where every bit of $\text{layer}_{i,t}$ is computed via oracle access to x_{imp} in the following way:
 - Locate $\text{node}_{i,j}$ as the Merkle tree node that contains the queried bit.
 - Compute the leaves in the subtree of $\text{node}_{i,j}$ by making queries to x_{imp} , i.e. making queries to x_{imp} for all indices in $\{2^i j \cdot N_0, \dots, 2^i(j+1) \cdot N_0 - 1\}$.
 - Using the leaves computed in the last step to compute $\text{node}_{i,j}$. Then return the queried bit.

The verifier rejects if $b = 0$.

The Extractor Algorithm. $\text{SNAP.Extract}(\text{crs}_{\mathcal{V}}, (x_{\text{exp}}, x_{\text{imp}}), \pi)$ does the following:

1. Parse $\text{crs}_{\mathcal{V}} = (\{\text{hk}_i^{\text{SEH}}, \text{ck}_i\}_{i \in \{0, \dots, d-1\}}, \{\text{hk}_i^{\text{HT}}\}_{i \in \{0, \dots, d\}}, \text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{R}})$, and parse π into components as described in the prover algorithm.
2. Let $\text{layer}_0 = x_{\text{imp}}$, and split it into $\text{layer}_{0,t} = \text{layer}_0|_{s_{0,t}}$ as the Prover algorithm.
3. Compute $\text{layer}'_{0,t} = \text{Recover}(\text{layer}_{0,t}, \text{ske}_{i,t})$. Recompose $\text{layer}'_{0,t}$ into layer'_0 .
4. Output $x'_{\text{imp}} = \text{layer}'_0$.

5.5 Analysis

We prove that the SNAP construct presented in Section 5.4 satisfies Theorem 5.4.

5.5.1 Completeness and Efficiency

We examine completeness by iterating over all checks performed by the verifier.

- $\text{SNARG}_{\mathcal{R}}.\mathcal{V}(\text{crs}_{\mathcal{R}}, \text{hk}_0^{\text{HT}}, \{\text{rt}_{0,t}\}_{t \in \{0,1\}}, x_{\text{exp}}, \pi_{\mathcal{R}}) = 1$. Note that $\{\text{rt}_{0,t}\}_{t \in \{0,1\}}$ is the hash value of x_{imp} , while $\mathcal{R}(x_{\text{exp}}, x_{\text{imp}}) = 1$. This step follows from the completeness of RAM SNARG.
- For every $i \in \{0, \dots, d-1\}, t \in \{0, 1\}$:
 - The SNARG verifier checks the proof for the statement that all of $\{c_{i,t'}, \text{ske}_{i,t'}, \text{rt}_{i,t'}\}_{t' \in \{0,1\}}$ are correctly computed from layer_i , using $\{\text{rt}_{i+1,t'}\}_{t' \in \{0,1\}}$ as the digest for layer_i . It accepts by completeness of RAM SNARG.
 - $\text{CoP.Verify}^{\text{layer}_{i,t}}(\text{ck}_i, c_{i,t})$ accepts, as our oracle simulation correctly recomputes $\text{layer}_{i,t}$ via queries to x_{imp} , and by completeness of CoP verification.

The efficiency guarantees can be observed from the construction:

- $\text{crs}_{\mathcal{P}} = \text{crs}_{\mathcal{V}}$ only consists of several (at most $\text{poly}(\lambda)$) hash keys and RAM SNARG common reference strings, where each term is bounded by $\text{poly}(\lambda)$.
- We examine the length of each component of π :
 - The length of $\{\text{node}_{d,j}\}_{j \in \{0, \dots, 2^{k-d}-1\}}$ is $2^{k-d}N_d = \rho \text{poly}(\lambda)$.
 - For each i, t , $|c_{i,t}| \leq \rho \cdot n^{o(1)} \text{poly}(\lambda)$, $|\text{ske}_{i,t}| \leq \rho \text{poly}(\lambda)$, and $|\text{rt}_{i,t}|, |\pi_{i,t}| \leq \text{poly}(\lambda)$.
 - $|\pi_{\mathcal{R}}| \leq \text{poly}(\lambda)$.

Therefore, the length of π is bounded by $\rho(n) \cdot n^{o(1)} \text{poly}(\lambda)$.

- We examine the runtime of the verifier by steps:
 - The runtime of computing $\text{rt}_{d,t}$ is $|\text{layer}_d| \text{poly}(\lambda) = \rho \text{poly}(\lambda)$.
 - The runtime to verify each $\pi_{i,t}$ is $\text{poly}(\lambda) \cdot |x_{\text{exp}}^{i,t}| = \rho n^{o(1)} \text{poly}(\lambda)$; the runtime to verify $\pi_{\mathcal{R}}$ is $|x_{\text{exp}}| \text{poly}(\lambda) = s \text{poly}(\lambda)$.

- To analyze the runtime of $\text{CoP.Verify}^{\text{layer}_{i,t}}(\text{ck}_i, \text{c}_{i,t})$, we observe that CoP.V itself runs in time $(\rho + \frac{2^{-i}n}{\rho})n^{o(1)}\text{poly}(\lambda)$, while making $\frac{2^{-i}n^{1+o(1)}}{\rho}\text{poly}(\lambda)$ queries to $\text{layer}_{i,t}$; in addition, the runtime to compute each bit of $\text{layer}_{i,t}$ is $2^i N_0 + 2^i \text{poly}(\lambda) = 2^i \text{poly}(\lambda)$. Then in total, the runtime of $\text{CoP.Verify}^{\text{layer}_{i,t}}(\text{ck}_i, \text{c}_{i,t})$ is $(\rho + \frac{n}{\rho})n^{o(1)}\text{poly}(\lambda)$, and $\frac{n^{1+o(1)}}{\rho}\text{poly}(\lambda)$ oracle queries is made.

Therefore, we conclude the runtime of the verifier is $(\rho n^{o(1)} + n^{1+o(1)}/\rho + s)\text{poly}(\lambda)$, and the query complexity is $\frac{n^{1+o(1)}}{\rho}\text{poly}(\lambda)$.

5.5.2 Adaptive Instance Extractability: Overview and Basic Conventions

We now focus on proving the adaptive instance extractability. In the construction, the honest prover computes a Merkle tree from x_{imp} , but only include the top $\sqrt{n}$ -size level in the proof. The security proof relies on recovering the Merkle tree from the instance chosen by the prover, and the secure sketches of each layer. The crucial part of this security proof is to argue the “correctness” of the recovered Merkle tree, which is defined as the poly-time computable predicate that the digests for each layer is correct with respect to that recovered layer. We show that

- The top $\sqrt{n}$ -size layer is always correct, as it is sent in clear in the proof.
- The correctness of each layer implies correctness of the next lower layer, via a careful application of SNARG soundness and somewhere extractability.
- The correctness of the bottom layers implies the recovered x_{imp} is in the language, and x_{imp} chosen by the malicious prover is close to the it. This implies adaptive instance extractability.

The above induction-like argument is a summary of our security proof. The following proof will first show the third bullet, and then the second bullet (while the first bullet is immediate by definition).

In the following we introduce a few notations:

- We add superscript $*$ to every component of the malicious prover output, to stress that they are maliciously generated, e.g. we write

$$\pi^* = (\{\text{node}_{d,j}^*\}_{j \in \{0, \dots, 2^{k-d}-1\}}, \{\text{c}_{i,t}^*, \text{ske}_{i,t}^*, \text{rt}_{i,t}^*, \pi_{i,t}^*\}_{i \in \{0, \dots, d-1\}, t \in \{0,1\}}, \pi_{\mathcal{R}}^*).$$

We also write $\text{rt}_{d,t}^*$ being the one computed from $\{\text{node}_{d,j}^*\}_{j \in \{0, \dots, 2^{k-d}-1\}}$ in the same way as the verification algorithm.

- Let $(x_{\text{exp}}^*, x_{\text{imp}}^*) \in \{0,1\}^s \times \{0,1\}^n$ be the input chosen by the malicious prover, and let $\text{layer}_{i,t}^*$ be the t -th partition of i -th layer of the Merkle tree computed from x_{imp}^* , in the same way as what the honest prover does to compute $\text{layer}_{i,t}$ from x_{imp} .
- For $0 \leq i \leq d-1$, let $\text{layer}'_{i,t} = \text{Sketch.Recover}(\text{layer}_{i,t}^*, \text{ske}_{i,t}^*)$, and let layer'_i or $\{\text{node}'_{i,j}\}_{j \in \{0, \dots, 2^{k-i}-1\}}$ be the Merkle tree layer recomposed from $\{\text{layer}'_{i,t}\}_{t \in \{0,1\}}$. *Specially*, take

$\text{layer}'_d = \left\{ \text{node}_{d,j}^* \right\}_{j \in \{0, \dots, 2^{k-d}-1\}}$. Take $x'_{\text{imp}} = \text{layer}'_0$ as the output of $\text{Extract}(\text{crs}_{\mathcal{V}}, x^*, \pi^*)$. Looking ahead, we will show $\text{rt}_{i,t}^* = \text{HT.Hash}(\text{hk}_i^{\text{HT}}, \text{layer}'_{i,t})$. Specially, for $i = 0$, this would imply $\pi_{\mathcal{R}}$ is actually a RAM SNARG proof of $\mathcal{R}(x_{\text{exp}}^*, x'_{\text{imp}}) = 1$.

- To simplify notations, for each $0 \leq i \leq d$, we define the event $\text{CORRECT}(\text{rt}_{i,t}^*)$ as $\text{rt}_{i,t}^* = \text{HT.Hash}(\text{hk}_i^{\text{HT}}, \text{layer}'_{i,t})$, and CORRECT-LAYER_i as $\bigwedge_t \text{CORRECT}(\text{rt}_{i,t}^*)$. Note that CORRECT-LAYER_d is always true.

We state our goal of adaptive instance extractability as follows.

Lemma 5.9. For any polynomial-time adversary $\mathcal{A}$ and polynomial $s(\lambda), n(\lambda)$, there exists a negligible function $\text{negl}(\cdot)$, such that for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \mathcal{V}_{\text{imp}}^{x^*}(\text{crs}_{\mathcal{V}}, x_{\text{exp}}^*, \pi^*) = 1 \\ \wedge (\mathcal{R}(x_{\text{exp}}^*, x'_{\text{imp}}) = 0 \vee \Delta(x_{\text{imp}}^*, x'_{\text{imp}}) > \rho) \end{array} : \begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^s, 1^n) \\ (x^*, \pi^*) = \mathcal{A}(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \end{array} \right] \leq \text{negl}(\lambda). \quad \diamond$$

The main idea of this security proof is to show correctness of the recovered Merkle tree with respect to digest values. In particular, we will prove the following:

Lemma 5.10. For any polynomial-time adversary $\mathcal{A}$ and polynomials $s(\lambda), n(\lambda)$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \mathcal{V}_{\text{imp}}^{x^*}(\text{crs}_{\mathcal{V}}, x_{\text{exp}}^*, \pi^*) = 1 \\ \wedge \exists 0 \leq i \leq d, \neg \text{CORRECT-LAYER}_i \end{array} : \begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^s, 1^n) \\ (x^*, \pi^*) = \mathcal{A}(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \end{array} \right] \leq \text{negl}(\lambda). \quad \diamond$$

5.5.3 Adaptive Instance Extractability From Correctness of Bottom Layers

In this section, we show how Lemma 5.10 would imply Lemma 5.9. In fact, correctness of the bottom layers, i.e. $\text{CORRECT-LAYER}_0 \wedge \text{CORRECT-LAYER}_1$ is sufficient to imply the extracted x_{imp} satisfies the requirement of instance extractability.

In this section, we omit the soundness game $(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^s, 1^n), (x^*, \pi^*) = \mathcal{A}(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}})$ in $\Pr[\cdot]$ by default.

Lemma 5.11. For any polynomial-time adversary $\mathcal{A}$ and polynomials $s(\lambda), n(\lambda)$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \mathcal{V}_{\text{imp}}^{x^*}(\text{crs}_{\mathcal{V}}, x_{\text{exp}}^*, \pi^*) = 1 \\ \wedge \text{CORRECT-LAYER}_1 \\ \wedge \Delta(x_{\text{imp}}^*, x'_{\text{imp}}) > \rho \end{array} \right] \leq \text{negl}(\lambda). \quad \diamond$$

Lemma 5.12. For any polynomial-time adversary $\mathcal{A}$ and polynomials $s(\lambda), n(\lambda)$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \mathcal{V}_{\text{imp}}^{x^*}(\text{crs}_{\mathcal{V}}, x_{\text{exp}}^*, \pi^*) = 1 \\ \wedge \text{CORRECT-LAYER}_0 \\ \wedge \mathcal{R}(x_{\text{exp}}^*, x'_{\text{imp}}) = 0 \end{array} \right] \leq \text{negl}(\lambda). \quad \diamond$$

Proof (Lemma 5.9 from Lemma 5.10, 5.11, 5.12). By Lemma 5.11 and Lemma 5.10,

$$\begin{aligned} & \Pr \left[\begin{array}{l} \mathcal{V}^{x^*_{\text{imp}}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \Delta(x^*_{\text{imp}}, x'_{\text{imp}}) > \rho \end{array} \right] \\ & \leq \Pr \left[\begin{array}{l} \mathcal{V}^{x^*_{\text{imp}}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \neg \text{CORRECT-LAYER}_1 \end{array} \right] + \Pr \left[\begin{array}{l} \mathcal{V}^{x^*_{\text{imp}}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \text{CORRECT-LAYER}_1 \\ \wedge \Delta(x^*_{\text{imp}}, x'_{\text{imp}}) > \rho \end{array} \right] \leq \text{negl}(\lambda). \end{aligned}$$

Similarly, By Lemma 5.12 and Lemma 5.10,

$$\begin{aligned} & \Pr \left[\begin{array}{l} \mathcal{V}^{x^*_{\text{imp}}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \mathcal{R}(x^*_{\text{exp}}, x'_{\text{imp}}) = 0 \end{array} \right] \\ & \leq \Pr \left[\begin{array}{l} \mathcal{V}^{x^*_{\text{imp}}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \neg \text{CORRECT-LAYER}_0 \end{array} \right] + \Pr \left[\begin{array}{l} \mathcal{V}^{x^*_{\text{imp}}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \text{CORRECT-LAYER}_0 \\ \wedge \mathcal{R}(x^*_{\text{exp}}, x'_{\text{imp}}) = 0 \end{array} \right] \leq \text{negl}(\lambda). \end{aligned}$$

The rest follows by a union bound. $\square$

The proof of Lemma 5.11 is delayed and would be implied by Lemma 5.15. We state Corollary 5.16 as a more general version of Lemma 5.11.

To prove Lemma 5.12, i.e. to show $\mathcal{R}(x^*_{\text{exp}}, x'_{\text{imp}}) = 1$, we observe that as $\text{rt}_{0,t}^* = \text{HT.Hash}(\text{hk}_0^{\text{HT}}, \text{layer}'_{0,t})$, $\pi_{\mathcal{R}}$ serves as a RAM SNARG proof that $\mathcal{R}(x_{\text{exp}}, x'_{\text{imp}} = \text{layer}'_0) = 1$.

Proof (Lemma 5.12).

$$\begin{aligned} & \Pr \left[\begin{array}{l} \mathcal{V}^{x^*_{\text{imp}}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \text{CORRECT-LAYER}_0 \\ \wedge \mathcal{R}(x^*_{\text{imp}}, x'_{\text{imp}}) = 0 \end{array} \right] \\ & \leq \Pr \left[\begin{array}{l} \forall t \in \{0, 1\}, \text{rt}_{0,t}^* = \text{HT.Hash}(\text{hk}_0^{\text{HT}}, \text{layer}'_{0,t}) \\ \wedge \text{SNARG}_{\mathcal{R}}(\mathcal{V}(\text{crs}_{\mathcal{R}}, \text{hk}_0^{\text{HT}}, \{\text{rt}_{0,t}^*\}_{t \in \{0,1\}}, x^*_{\text{exp}}, \pi^*_{\mathcal{R}}) = 1 \\ \wedge \mathcal{R}(x^*_{\text{exp}}, x'_{\text{imp}}) = 0 \end{array} \right] \leq \text{negl}(\lambda), \end{aligned}$$

where the second inequality follows from soundness of RAM SNARG. $\square$

5.5.4 Correctness of Digests by Induction

In this section we prove Lemma 5.10. The high level idea is an induction from $i = d - 1$ to $i = 0$, that $\text{CORRECT-LAYER}_{i+1}$ implies CORRECT-LAYER_i if the SNAP proof is accepted. We formalize this process in the following lemma.

Lemma 5.13. For any polynomial-time adversary $\mathcal{A}$ and polynomials $s(\lambda), n(\lambda)$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$ and every $i \in \{0, \dots, d - 1\}, t \in \{0, 1\}$,

$$\Pr \left[\begin{array}{l} \mathcal{V}^{x^*_{\text{imp}}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \text{CORRECT-LAYER}_{i+1} \\ \wedge \neg \text{CORRECT}(\text{rt}_{i,t}^*) \end{array} : \begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^s, 1^n) \\ (x^*, \pi^*) = \mathcal{A}(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \end{array} \right] \leq \text{negl}(\lambda). \quad \diamond$$

Corollary 5.14. For any polynomial-time adversary $\mathcal{A}$ and polynomials $s(\lambda), n(\lambda)$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$ and every $i \in \{0, \dots, d-1\}$,

$$\Pr \left[\begin{array}{l} \mathcal{V}^{x^*}_{\text{imp}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \text{CORRECT-LAYER}_{i+1} \\ \wedge \neg \text{CORRECT-LAYER}_i \end{array} : \begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^s, 1^n) \\ (x^*, \pi^*) = \mathcal{A}(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \end{array} \right] \leq \text{negl}(\lambda). \quad \diamond$$

Proof (Lemma 5.10 via Corollary 5.14). Let the negligible function specified in Corollary 5.14 be δ . Then

$$\begin{aligned} & \Pr \left[\begin{array}{l} \mathcal{V}^{x^*}_{\text{imp}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \exists 0 \leq i \leq d, \neg \text{CORRECT-LAYER}_i \end{array} : \begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^s, 1^n) \\ (x^*, \pi^*) = \mathcal{A}(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \end{array} \right] \\ & \leq \Pr \left[\begin{array}{l} \mathcal{V}^{x^*}_{\text{imp}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \neg \text{CORRECT-LAYER}_d \end{array} : \begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^s, 1^n) \\ (x^*, \pi^*) = \mathcal{A}(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \end{array} \right] + \\ & \quad \sum_{i=0}^{d-1} \Pr \left[\begin{array}{l} \mathcal{V}^{x^*}_{\text{imp}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \text{CORRECT-LAYER}_{i+1} \\ \wedge \neg \text{CORRECT-LAYER}_i \end{array} : \begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^s, 1^n) \\ (x^*, \pi^*) = \mathcal{A}(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \end{array} \right] \\ & \leq d\delta. \quad \square \end{aligned}$$

The idea behind Lemma 5.13 is, given $\text{CORRECT-LAYER}_{i+1}$ i.e. $\text{rt}_{i+1,t}^* = \text{HT.Hash}(\text{hk}_{i+1}^{\text{HT}}, \text{layer}'_{i+1,t})$ for $t \in \{0, 1\}$, and $\text{SNARG}_{\mathcal{P}}.\mathcal{V}(\text{crs}_{\mathcal{P}}, (\text{hk}_i^{\text{SEH}}, \text{hk}_{i+1}^{\text{SEH}}), \{\text{rt}_{i+1,t}^*\}_{t \in \{0,1\}}, x_{\text{exp}}^{i,t}, \pi_{i,t}^*) = 1$, we are able to use partial input soundness of $\text{SNARG}_{\mathcal{P}}$ to argue that, if we extract $\text{layer}_{i,t}$ from layer'_{i+1} using SEH trapdoor associated with hk_i^{SEH} , then all of $c_{i,t}^*$, $\text{ske}_{i,t}^*$ and $\text{rt}_{i,t}^*$ should be correctly computed from the extracted $\text{layer}_{i,t}$.

To formalize this idea, consider an alternative crs generation algorithm $\mathcal{G}^{i,t}(1^\lambda, 1^s, 1^n)$ for every $i \in \{0, \dots, d-1\}, t \in \{0, 1\}$: the only difference to $\mathcal{G}(1^\lambda, 1^s, 1^n)$ is, instead of taking $\text{hk}_i^{\text{SEH}} \leftarrow \text{SEH}.\mathcal{G}(1^\lambda, 2N_i, I^*)$, we take $(\text{hk}_i^{\text{SEH}}, \text{td}_{\text{SEH}}^{i,t}) \leftarrow \text{SEH}.\mathcal{G}(1^\lambda, 2N_i, [0, N_i] + tN_i)$. In the following, by $\Pr_{\mathcal{G}^{i,t}}[\cdot]$ we mean the probability under the experiment $(\text{crs}, \text{td}_{\text{SEH}}^{i,t}) \leftarrow \mathcal{G}^{i,t}(1^\lambda, 1^s, 1^n), (x^*, \pi^*) \leftarrow \mathcal{A}(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}})$. Also by $\Pr_{\mathcal{G}}[\cdot]$ we mean the standard experiment in soundness definition, i.e. $\text{crs} \leftarrow \mathcal{G}(1^\lambda, 1^s, 1^n), (x^*, \pi^*) \leftarrow \mathcal{A}(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}})$.

When crs is generated from $\mathcal{G}^{i,t}$, define $\text{layer}_{i,t}^{\mathcal{E}}$ by the following way: for $\text{layer}'_{i+1} = \{\text{node}'_{i+1,j}\}_{j \in \{0, \dots, 2^{k-(i+1)}-1\}}$ (recall that we define layer'_{i+1} as the composition of $\text{layer}'_{i+1,t} = \text{Recover}(\text{layer}'_{i+1,t}, \text{ske}_{i+1,t}^*)$), we take $\text{layer}_{i,t}^{\mathcal{E}}$ to be the concatenation of $\{\text{SEH.Extract}(\text{td}_{\text{SEH}}^{i,t}, \text{node}'_{i+1,j})\}_{j \in \{0, \dots, 2^{k-(i+1)}-1\}}$.

The following lemma shows that, by invoking partial input soundness of $\text{SNARG}_{\mathcal{P}}$, we can show correctness of $c_{i,t}^*, \text{ske}_{i,t}^*, \text{rt}_{i,t}^*$ with respect to $\text{layer}_{i,t}^{\mathcal{E}}$ (if crs is generated by $\mathcal{G}^{i,t}$).

Lemma 5.15. For any polynomial-time adversary $\mathcal{A}$ and polynomials $s(\lambda), n(\lambda)$, there exists a neg-

ligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$ and $0 \leq i \leq d-1, 0 \leq t \leq 1$,

$$\Pr_{\mathcal{G}^{i,t}} \left[\begin{array}{l} \mathcal{V}^{x^*}_{\text{imp}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \wedge \\ \text{CORRECT-LAYER}_{i+1} \wedge \\ \left(\begin{array}{l} c^*_{i,t} \neq \text{CoP.Commit}(\text{ck}_i, \text{layer}^{\mathcal{E}}_{i,t}) \\ \vee \text{ske}^*_{i,t} \neq \text{Sketch}(\text{layer}^{\mathcal{E}}_{i,t}, \rho) \\ \vee \text{rt}^*_{i,t} \neq \text{HT.Hash}(\text{hk}^{\text{HT}}_i, \text{layer}^{\mathcal{E}}_{i,t}) \end{array} \right) \end{array} \right] \leq \text{negl}(\lambda). \quad \diamond$$

Proof (Lemma 5.15). We apply partial-input soundness of SNARG_P : the term we want to bound is less than

$$\Pr_{\mathcal{G}^{i,t}} \left[\begin{array}{l} \text{SNARG}_P.\mathcal{V}(\text{crs}_P, (\text{hk}^{\text{SEH}}_i, \text{hk}^{\text{HT}}_{i+1}), \{\text{rt}^*_{i+1,t}\}_{t \in \{0,1\}}, x^*_{\text{exp}}, \pi_{i,t}) = 1 \wedge \\ \forall t \in \{0,1\}, \text{rt}^*_{i+1,t} = \text{HT.Hash}(\text{hk}^{\text{HT}}_{i+1}, \text{layer}'_{i+1,t}) \wedge \\ \text{layer}^{\mathcal{E}}_{i,t} = \text{SEH.Extract}(\text{td}^{\text{SEH}}_{i,t}, \text{layer}'_{i+1,t}) \wedge \\ \left(\begin{array}{l} c^*_{i,t} \neq \text{CoP.Commit}(\text{ck}_i, \text{layer}^{\mathcal{E}}_{i,t}) \\ \vee \text{ske}^*_{i,t} \neq \text{Sketch}(\text{layer}^{\mathcal{E}}_{i,t}, \rho) \\ \vee \text{rt}^*_{i,t} \neq \text{HT.Hash}(\text{hk}^{\text{HT}}_i, \text{layer}^{\mathcal{E}}_{i,t}) \end{array} \right) \end{array} \right] \leq \text{negl}(\lambda). \quad \square$$

Lemma 5.15 implies Lemma 5.13: By RAM SNARG soundness $c^*_{i,t}, \text{ske}^*_{i,t}$ is the corresponding commitment/sketch computed from $\text{layer}^{\mathcal{E}}_{i,t}$, the former implies $\Delta(\text{layer}^{\mathcal{E}}_{i,t}, \text{layer}^*_{i,t}) \leq \rho/2$; then by completeness of sketches, $\text{layer}^{\mathcal{E}}_{i,t} = \text{Recover}(\text{layer}^*_{i,t}, \text{ske}^*_{i,t}) = \text{layer}'_{i,t}$. So we conclude with $\text{CORRECT}(\text{rt}^*_{i,t})$, which would hold with high probability even if we switch crs generation from $\mathcal{G}^{i,t}$ to $\mathcal{G}$ as SEH is index hiding.

Proof (Lemma 5.13 via Lemma 5.15). Take $\delta(\cdot)$ as the negligible function specified by Lemma 5.15. We have

$$\Pr_{\mathcal{G}^{i,t}} \left[\begin{array}{l} \mathcal{V}^{x^*}_{\text{imp}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \text{CORRECT-LAYER}_{i+1} \\ \wedge \neg(c^*_{i,t} = \text{CoP.Commit}(\text{ck}_i, \text{layer}^{\mathcal{E}}_{i,t}) \wedge \text{ske}^*_{i,t} = \text{Sketch}(\text{layer}^{\mathcal{E}}_{i,t}, \rho)) \end{array} \right] \leq \delta.$$

By computational binding of proximity of CoP, there exists negligible function ν that for all $0 \leq i \leq d-1, 0 \leq t \leq 1$,

$$\Pr_{\mathcal{G}^{i,t}} \left[\begin{array}{l} \text{CoP.Verify}(\text{ck}_i, c^*_{i,t}, \text{layer}^*_{i,t}) = 1 \\ \wedge c^*_{i,t} = \text{CoP.Commit}(\text{ck}_i, \text{layer}^{\mathcal{E}}_{i,t}) \\ \wedge \Delta(\text{layer}^*_{i,t}, \text{layer}^{\mathcal{E}}_{i,t}) > \rho/2 \end{array} \right] \leq \nu.$$

As $\text{CoP.Verify}(\text{ck}_i, c^*_{i,t}, \text{layer}^*_{i,t}) = 1$ is implied by $\mathcal{V}(\text{crs}_{\mathcal{V}}, \pi^*, x^*) = 1$, we have

$$\Pr_{\mathcal{G}^{i,t}} \left[\begin{array}{l} \mathcal{V}^{x^*}_{\text{imp}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \text{CORRECT-LAYER}_{i+1} \\ \wedge \Delta(\text{layer}^*_{i,t}, \text{layer}^{\mathcal{E}}_{i,t}) > \rho/2 \end{array} \right]$$

$$\begin{aligned}
&\leq \Pr_{\mathcal{G}^{i,t}} \left[\begin{array}{c} \mathcal{V}^{x^*}_{\text{imp}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \text{CORRECT-LAYER}_{i+1} \\ \wedge c^*_{i,t} \neq \text{CoP.Commit}(\text{ck}_i, \text{layer}^{\mathcal{E}}_{i,t}) \end{array} \right] + \Pr_{\mathcal{G}^{i,t}} \left[\begin{array}{c} \mathcal{V}^{x^*}_{\text{imp}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \text{CORRECT-LAYER}_{i+1} \\ \wedge c^*_{i,t} = \text{CoP.Commit}(\text{ck}_i, \text{layer}^{\mathcal{E}}_{i,t}) \\ \wedge \Delta(\text{layer}^*_{i,t}, \text{layer}^{\mathcal{E}}_{i,t}) > \rho/2 \end{array} \right] \\
&\leq \delta + \delta' + \nu.
\end{aligned}$$

On the other hand, by completeness of sketches, two events $\text{ske}^*_{i,t} = \text{Sketch}(\text{layer}^{\mathcal{E}}_{i,t}, \rho)$ and $\Delta(\text{layer}^*_{i,t}, \text{layer}^{\mathcal{E}}_{i,t}) \leq \rho$ implies that

$$\text{layer}'_{i,t} = \text{Recover}(\text{layer}^*_{i,t}, \text{ske}^*_{i,t}) = \text{layer}^{\mathcal{E}}_{i,t}.$$

Therefore

$$\begin{aligned}
&\Pr_{\mathcal{G}^{i,t}} \left[\begin{array}{c} \mathcal{V}^{x^*}_{\text{imp}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \text{CORRECT-LAYER}_{i+1} \\ \wedge \neg \text{CORRECT}(\text{rt}^*_{i,t}) \end{array} \right] \\
&\leq \Pr_{\mathcal{G}^{i,t}} \left[\begin{array}{c} \mathcal{V}^{x^*}_{\text{imp}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \text{CORRECT-LAYER}_{i+1} \\ \wedge \text{rt}^*_{i,t} \neq \text{HT.Hash}(\text{hk}_i^{\text{HT}}, \text{layer}^{\mathcal{E}}_{i,t}) \end{array} \right] + \Pr_{\mathcal{G}^{i,t}} \left[\begin{array}{c} \mathcal{V}^{x^*}_{\text{imp}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \text{CORRECT-LAYER}_{i+1} \\ \wedge \text{layer}'_{i,t} \neq \text{layer}^{\mathcal{E}}_{i,t} \end{array} \right] \\
&\leq \delta + \Pr_{\mathcal{G}^{i,t}} \left[\begin{array}{c} \mathcal{V}^{x^*}_{\text{imp}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \text{CORRECT-LAYER}_{i+1} \\ \wedge \text{ske}^*_{i,t} \neq \text{Sketch}(\text{layer}^{\mathcal{E}}_{i,t}, \rho) \end{array} \right] + \Pr_{\mathcal{G}^{i,t}} \left[\begin{array}{c} \mathcal{V}^{x^*}_{\text{imp}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \text{CORRECT-LAYER}_{i+1} \\ \wedge \Delta(\text{layer}^*_{i,t}, \text{layer}^{\mathcal{E}}_{i,t}) > \rho \end{array} \right] \leq \text{negl}(\lambda).
\end{aligned}$$

Finally, by index hiding of SEH, for every $i \in \{0, \dots, d-1\}, t \in \{0, 1\}$, $\Pr_{\mathcal{G}} \left[\begin{array}{c} \mathcal{V}^{x^*}_{\text{imp}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \\ \wedge \text{CORRECT-LAYER}_{i+1} \\ \wedge \neg \text{CORRECT}(\text{rt}^*_{i,t}) \end{array} \right]$ is also negligible. We note that the order of quantifiers of $\text{negl}(\cdot)$ and i, t can always be switched via complexity leveraging. $\square$

We note that the above proof also implies the following corollary (by considering both $\Delta(\text{layer}^*_{i,t}, \text{layer}^{\mathcal{E}}_{i,t}) \leq \rho/2$ and $\text{layer}^{\mathcal{E}}_{i,t} = \text{layer}'_{i,t}$):

Corollary 5.16. For any polynomial-time adversary $\mathcal{A}$ and polynomials $s(\lambda), n(\lambda)$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$ and $0 \leq i \leq d-1, 0 \leq t \leq 1$,

$$\Pr_{\mathcal{G}} \left[\begin{array}{c} \mathcal{V}^{x^*}_{\text{imp}}(\text{crs}_{\mathcal{V}}, x^*_{\text{exp}}, \pi^*) = 1 \wedge \\ \text{CORRECT-LAYER}_{i+1} \wedge \\ \Delta(\text{layer}^*_{i,t}, \text{layer}'_{i,t}) > \rho/2 \end{array} \right] \leq \text{negl}(\lambda). \quad \diamond$$

5.6 Generic Upgrade to SNAPs for NP

In this section we prove Theorem 5.6: we can upgrade any adaptive instance extractable SNAP for RAM computation (in particular, one constructed in Theorem 5.4) via any adaptively sound SNARG for NP, to get an adaptive instance extractable SNAP for NP. In a nutshell, this is done by delegating SNARG verification to the prover by the SNAP system.

Formally, we fix any language $\mathcal{L} \in \text{NP}$, and use the following building blocks:

- A SNARG system for $\mathcal{L}$ with algorithms $(\text{SNARG}.\mathcal{G}, \text{SNARG}.\mathcal{P}, \text{SNARG}.\mathcal{V})$. Furthermore, there is a function $L(\lambda, n, m)$ such that $|\text{crs}_{\mathcal{V}}|, |\pi| \leq L(\lambda, n, m)$.
- An adaptive instance extractable SNAP system $(\text{SNAP}.\mathcal{G}, \text{SNAP}.\mathcal{P}, \text{SNAP}.\mathcal{V}, \text{SNAP}.\text{Extract})$, for the RAM machine M that takes input $x_{\text{exp}} = (\text{crs}_{\mathcal{V}}, \pi)$, $x_{\text{imp}} = x$, and outputs $\text{SNARG}.\mathcal{V}(\text{crs}_{\mathcal{V}}, x, \pi)$; this SNAP has verification time $(n^{c+o(1)} + s)\text{poly}(\lambda)$.

Our SNAP for $\mathcal{L}$ construction works as follows:

The Setup Algorithm. $\mathcal{G}(1^\lambda, 1^n, 1^m)$ does the following:

1. Sample $(\text{crs}_{\mathcal{P}}^{\text{SNARG}}, \text{crs}_{\mathcal{V}}^{\text{SNARG}}) \leftarrow \text{SNARG}.\mathcal{G}(1^\lambda, 1^n, 1^m)$.
2. Sample $(\text{crs}_{\mathcal{P}}^{\text{SNAP}}, \text{crs}_{\mathcal{V}}^{\text{SNAP}}) \leftarrow \text{SNAP}.\mathcal{G}(1^\lambda, 1^{2L}, 1^n)$.

The common reference string is set to be

$$\text{crs}_{\mathcal{P}} = (\text{crs}_{\mathcal{P}}^{\text{SNARG}}, \text{crs}_{\mathcal{V}}^{\text{SNARG}}, \text{crs}_{\mathcal{P}}^{\text{SNAP}}), \text{crs}_{\mathcal{V}} = (\text{crs}_{\mathcal{V}}^{\text{SNARG}}, \text{crs}_{\mathcal{V}}^{\text{SNAP}}).$$

The Prover Algorithm. $\mathcal{P}(\text{crs}_{\mathcal{P}}, x, w)$ does the following:

1. Parse $\text{crs}_{\mathcal{P}} = (\text{crs}_{\mathcal{P}}^{\text{SNARG}}, \text{crs}_{\mathcal{V}}^{\text{SNARG}}, \text{crs}_{\mathcal{P}}^{\text{SNAP}})$.
2. Compute $\pi_{\text{SNARG}} \leftarrow \text{SNARG}.\mathcal{P}(\text{crs}_{\mathcal{P}}^{\text{SNARG}}, x, w)$.
3. Prepare $x_{\text{exp}} = (\text{crs}_{\mathcal{V}}^{\text{SNARG}}, \pi_{\text{SNARG}})$, and $x_{\text{imp}} = x$.
4. Compute $\pi_{\text{SNAP}} = \text{SNAP}.\mathcal{P}(\text{crs}_{\mathcal{P}}^{\text{SNAP}}, (x_{\text{exp}}, x_{\text{imp}}))$.
5. Send the proof $\pi = (\pi_{\text{SNARG}}, \pi_{\text{SNAP}})$.

The Verifier Algorithm. $\mathcal{V}^x(\text{crs}_{\mathcal{V}}, \pi)$ does the following:

1. Parse $\text{crs}_{\mathcal{V}} = (\text{crs}_{\mathcal{V}}^{\text{SNARG}}, \text{crs}_{\mathcal{V}}^{\text{SNAP}})$.
2. Parse $\pi = (\pi_{\text{SNARG}}, \pi_{\text{SNAP}})$.
3. Output $b \leftarrow \text{SNAP}.\mathcal{V}^x(\text{crs}_{\mathcal{V}}^{\text{SNAP}}, (\text{crs}_{\mathcal{V}}^{\text{SNARG}}, \pi_{\text{SNARG}}), \pi_{\text{SNAP}})$.

The Extractor Algorithm $\text{Extract}(\text{crs}_{\mathcal{V}}, x, \pi)$ does the following:

1. Parse $\text{crs}_{\mathcal{V}} = (\text{crs}_{\mathcal{V}}^{\text{SNARG}}, \text{crs}_{\mathcal{V}}^{\text{SNAP}})$.
2. Parse $\pi = (\pi_{\text{SNARG}}, \pi_{\text{SNAP}})$.
3. Prepare $x_{\text{exp}} = (\text{crs}_{\mathcal{V}}^{\text{SNARG}}, \pi_{\text{SNARG}})$, and $x_{\text{imp}} = x$.
4. Output $x' \leftarrow \text{SNAP}.\text{Extract}(\text{crs}_{\mathcal{V}}^{\text{SNAP}}, (x_{\text{exp}}, x_{\text{imp}}), \pi_{\text{SNAP}})$.

Proof (Theorem 5.6). We show the adaptive instance extractable SNAP system for $\mathcal{L}$ constructed above satisfies all properties required by Theorem 5.6.

The completeness guarantee follows immediately from the completeness guarantee of the underlying primitives. To analyze the efficiency guarantee, we observe that:

- $|x_{\text{exp}}| = |\text{crs}_{\mathcal{V}}^{\text{SNARG}}| + |\pi_{\text{SNARG}}| = O(L)$.
- $|\text{crs}_{\mathcal{V}}| = |\text{crs}_{\mathcal{V}}^{\text{SNARG}}| + |\text{crs}_{\mathcal{V}}^{\text{SNAP}}| \leq L + |\text{crs}_{\mathcal{V}}^{\text{SNAP}}| \leq (n^{c+o(1)} + L)\text{poly}(\lambda)$.
- $|\pi| = |\pi_{\text{SNARG}}| + |\pi_{\text{SNAP}}| = (n^{c+o(1)} + L)\text{poly}(\lambda)$.
- The runtime of verifier is $(n^{c+o(1)} + |x_{\text{exp}}|)\text{poly}(\lambda) = (n^{c+o(1)} + L)\text{poly}(\lambda)$.

Finally, we prove the adaptive instance extractability guarantee. By the adaptive instance extractability of the SNAP system for RAM computation, there exists a negligible function $\delta(\cdot)$ that, for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{SNAP}.\mathcal{V}^{x^*}(\text{crs}_{\mathcal{V}}^{\text{SNAP}}, (\text{crs}_{\mathcal{V}}^{\text{SNARG}}, \pi_{\text{SNARG}}^*), \pi_{\text{SNAP}}^*) = 1 \wedge \\ (\text{SNARG}.\mathcal{V}(\text{crs}_{\mathcal{V}}^{\text{SNARG}}, x', \pi_{\text{SNARG}}^*) = 0 \vee \Delta(x^*, x') > \rho(n)) \end{array} : \begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m) \\ (x^*, \pi^*) \leftarrow \mathcal{A}(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \\ (\pi_{\text{SNARG}}^*, \pi_{\text{SNAP}}^*) = \pi^* \\ x' \leftarrow \text{Extract}(\text{crs}_{\mathcal{V}}, x^*, \pi^*) \end{array} \right] \leq \delta.$$

By adaptive soundness of the SNARG system, there exists a negligible function $\delta'(\cdot)$ that, for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{SNARG}.\mathcal{V}(\text{crs}_{\mathcal{V}}^{\text{SNARG}}, x', \pi_{\text{SNARG}}^*) = 1 \\ \wedge x' \notin \mathcal{L} \end{array} : \begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m) \\ (x^*, \pi^*) \leftarrow \mathcal{A}(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \\ (\pi_{\text{SNARG}}^*, \pi_{\text{SNAP}}^*) = \pi^* \\ x' \leftarrow \text{Extract}(\text{crs}_{\mathcal{V}}, x^*, \pi^*) \end{array} \right] \leq \delta'.$$

Therefore, we have

$$\begin{aligned} & \Pr \left[\begin{array}{l} \mathcal{V}^{x^*}(\text{crs}_{\mathcal{V}}, \pi^*) = 1 \\ \wedge (x' \notin \mathcal{L} \vee \Delta(x^*, x') > \rho(n)) \end{array} : \begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m) \\ (x^*, \pi^*) \leftarrow \mathcal{A}(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \\ (\pi_{\text{SNARG}}^*, \pi_{\text{SNAP}}^*) = \pi^* \\ x' \leftarrow \text{Extract}(\text{crs}_{\mathcal{V}}, x^*, \pi^*) \end{array} \right] \\ & \leq \Pr \left[\begin{array}{l} \text{SNAP}.\mathcal{V}^{x^*}(\text{crs}_{\mathcal{V}}^{\text{SNAP}}, (\text{crs}_{\mathcal{V}}^{\text{SNARG}}, \pi_{\text{SNARG}}^*), \pi_{\text{SNAP}}^*) = 1 \\ \wedge x' \notin \mathcal{L} \end{array} : \begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m) \\ (x^*, \pi^*) \leftarrow \mathcal{A}(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \\ (\pi_{\text{SNARG}}^*, \pi_{\text{SNAP}}^*) = \pi^* \\ x' \leftarrow \text{Extract}(\text{crs}_{\mathcal{V}}, x^*, \pi^*) \end{array} \right] \\ & \quad + \Pr \left[\begin{array}{l} \text{SNAP}.\mathcal{V}^{x^*}(\text{crs}_{\mathcal{V}}^{\text{SNAP}}, (\text{crs}_{\mathcal{V}}^{\text{SNARG}}, \pi_{\text{SNARG}}^*), \pi_{\text{SNAP}}^*) = 1 \\ \wedge \Delta(x^*, x') > \rho(n) \end{array} : \begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m) \\ (x^*, \pi^*) \leftarrow \mathcal{A}(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \\ (\pi_{\text{SNARG}}^*, \pi_{\text{SNAP}}^*) = \pi^* \\ x' \leftarrow \text{Extract}(\text{crs}_{\mathcal{V}}, x^*, \pi^*) \end{array} \right] \\ & \leq (\delta + \delta') + \delta. \quad \square \end{aligned}$$

6 Unconditional Impossibility Result

In this section, we show unconditional impossibility results about adaptively secure SNAPs.

Theorem 6.1. For *some* constant $\varepsilon > 0$, there exists a language $L \in \mathcal{P}$ such that, it is impossible to have an *adaptively* secure SNAP for L with distance εn with verifier query complexity q and proof size p such that $pq = o(n)$. $\diamond$

Theorem 6.2. For *every* constant $\varepsilon > 0$, there exists a language $L \in \mathcal{P}$ such that, it is impossible to have an *adaptively* secure SNAP for L with distance $(1/2 - \varepsilon)n$ with verifier query complexity q and proof size p such that $pq = o\left(\frac{n}{\log n}\right)$. $\diamond$

Remark 13. These two theorems relativize; e.g. they also hold in the random oracle model. $\diamond$

The two theorems follow from the following lemma and linear secret sharing of Theorem 6.10 and Lemma 6.11 respectively.

Lemma 6.3. Assuming the existence of an explicit linear secret sharing family for infinitely many n with t -privacy and ε -distance (see Definition 6.9), there exists a language L in $\mathcal{P}$ such that, every SNAP for L with distance εn is not adaptively sound even against uniform adversaries, if the query complexity of verifier q and proof size p has $p(n)q(n) \leq t(n)$ (both p, q are a function of instance size n). $\diamond$

We note that Theorem 6.2 implies our construction of SNAP in Theorem 5.5 is optimal up to a $n^{o(1)}$ factor.

In the following, we put additional preliminaries in Section 6.1. After that, we will introduce a lemma that enables the adversary to resample a false instance after generating a proof for true instance in Section 6.2, and prove Lemma 6.3 in Section 6.3.

6.1 Additional Preliminaries

6.1.1 Information Theoretical Tools

Definition 6.4 (KL Divergence). For random variables X, Y, Z where X, Y are distributed in space $\mathcal{X}$, Z is distributed in space $\mathcal{Z}$, the conditional Kullback-Leibler distance is defined as

$$D(X||Y|Z) = \sum_{z \in \mathcal{Z}} \Pr[Z = z] \sum_{x \in \mathcal{X}} \Pr[X = x|Z = z] \log \left(\frac{\Pr[X = x|Z = z]}{\Pr[Y = x|Z = z]} \right). \quad \diamond$$

Fact 6.5 (Chain rule).

$$\begin{aligned} & D(X_1, \dots, X_k || Y_1, \dots, Y_k) \\ &= D(X_1 || X_2) + D(X_2 || Y_2 | X_1) + \dots + D(X_k || Y_k | X_1, \dots, X_{k-1}). \end{aligned} \quad \diamond$$

Lemma 6.6. For any function f , $D(f(X)||f(Y)|Z) \leq D(X||Y|Z)$. $\diamond$

Lemma 6.7. Let X, Y, Z, C be four random variables such that: X, Y are identically distributed; Y is independent of (Z, C) ; and X is independent of C . Then $D(X||Y|C, Z) \leq H(Z)$. $\diamond$

Proof.

$$\begin{aligned}
D(X||Y|C, Z) &= \sum_{c,z} \Pr[C = c, Z = z] \sum_x \Pr[X = x|C = c, Z = z] \log \left(\frac{\Pr[X = x|C = c, Z = z]}{\Pr[Y = x|C = c, Z = z]} \right) \\
&= \sum_{c,z} \Pr[C = c, Z = z] \sum_x \Pr[X = x|C = c, Z = z] \log \left(\frac{\Pr[X = x, Z = z|C = c]}{\Pr[Y = x, Z = z|C = c]} \right) \\
&\leq \sum_{c,z} \Pr[C = c, Z = z] \sum_x \Pr[X = x|C = c, Z = z] \log \left(\frac{\Pr[X = x|C = c]}{\Pr[Y = x] \Pr[Z = z|C = c]} \right) \\
&= \sum_{c,z} \Pr[C = c, Z = z] \sum_x \Pr[X = x|C = c, Z = z] \log \left(\frac{\Pr[X = x]}{\Pr[Y = x] \Pr[Z = z|C = c]} \right) \\
&= \sum_{c,z} \Pr[C = c, Z = z] \sum_x \Pr[X = x|C = c, Z = z] \log \left(\frac{1}{\Pr[Z = z|C = c]} \right) \\
&= \sum_{c,z} \Pr[Z = z, C = c] \log \left(\frac{1}{\Pr[Z = z|C = c]} \right) \\
&= H(Z|C) \leq H(Z). \quad \square
\end{aligned}$$

Fact 6.8. $\Delta(X, Z, C; Y, Z, C) \leq \sqrt{\frac{1}{2}D(X||Y|Z, C)}$. $\diamond$

6.1.2 Secret Sharing

Definition 6.9 (Linear Secret Sharing). A linear secret sharing of size n consists of two affine subspaces: $C_0, C_1 \subset \{0, 1\}^n$, satisfying the following property:

- t -privacy: for any $S \subseteq [n]$ with $|S| \leq t(n)$, $\{x_S | x \leftarrow C_0\}$ has the same distribution as $\{x'_S | x' \leftarrow C_1\}$.
- ε -distance: for any $x \in C_0, x' \in C_1$, $\Delta(x, x') \geq \varepsilon(n)$.

Furthermore, we say a linear secret sharing family $\{C_0^n, C_1^n\}_{n \in \mathbb{N}}$ is explicit if we can compute the generator matrix of C_0, C_1 efficiently given 1^n . $\diamond$

Such secret sharing scheme can be constructed from algebraic geometry codes (AG codes) (see ref. [CDN15]).

Theorem 6.10. There exists an explicit linear secret sharing family for infinitely many n , which satisfies αn -privacy and βn -distance for some constant $\alpha, \beta > 0$. $\diamond$

Another way to construct linear secret sharing is to concatenate Shamir secret sharing with an optimal linear code of $\log n$ -dimension generated by exhaustively searching all Toeplitz matrices.

Lemma 6.11. For every $\varepsilon \in (0, 1/2)$, there exists a constant $\alpha > 0$ and an explicit linear secret sharing family for infinitely many n , which satisfies $\frac{\alpha n}{\log n}$ -privacy and εn distance. $\diamond$

Proof sketch. Take $\varepsilon' \in (\varepsilon, 1/2)$ as any constant. By probabilistic argument, there exists a constant $c > 0$ such that for large enough l , there exists a Toeplitz matrix $C \in \mathbb{Z}_2^{cl \times l}$ such that, for every $x \in \mathbb{Z}_2^l$, $\|Cx\| > \varepsilon' \cdot cl$.

Take $n = cl \cdot 2^l$. We construct the linear secret sharing as a Shamir secret sharing over $\mathbb{F}_n$ between 2^l parties with threshold $(1 - \varepsilon/\varepsilon')2^l$, concatenated with the linear code defined by C . One can show that the resulting code has ε -distance, and $\frac{1-\varepsilon/\varepsilon'}{c \log n}$ -privacy. $\square$

6.2 Efficient Oracle Pre-sampling Lemma

We introduce a lemma in the context of auxiliary input random oracle model (AI-ROM) [CDGS18], where we consider a random oracle (not necessarily uniformly random) and an auxiliary input that depends on the oracle. A standard result in AI-ROM is called the pre-sampling technique: it can be shown that after fixing a few bits in the oracle, for polynomial-time programs the auxiliary input seems to independent of the rest of the oracle.

We make an analogue between AI-ROM and SNAPS, by taking the random oracle as some randomly distributed instance, and the auxiliary input as the SNAP proof. By this analogue, the pre-sampling technique translates to the observation that the SNAP proof conveys no information about the instance, except for a few pre-sampled locations. Looking ahead, we will construct an adversary that first generates an honest proof π for some instance $x' \in \mathcal{L}$, and then use π as the cheating proof for another instance x . The bit-fixing technique ensures that the verifier cannot tell if the instance is switched or not.

The following lemma shows that when applying the pre-sampling technique, the set of fixed bits can be computed efficiently from the auxiliary input. The lemma also considers a random variable C independent of the oracle, which corresponds to the crs.

Lemma 6.12 (Efficient Oracle Pre-sampling, uniform version). Let O be any distribution of oracles (treated as a random variable later), and C be any random variable that is independent of O . Let Z be a random variable which may depend on O, C . We set $p := \log \#|Z|$ being the bit length to represent Z .

Let $V^o(c, z)$ be a randomized oracle algorithm that makes at most q queries, and let $(i * V)^o(c, z)$ be the oracle algorithm that runs V for i times with oracle o , input (c, z) and independent randomness during each execution. For any $p, T \geq 1$, there exists $0 \leq i < T$ such that, with S being a random set of oracle inputs consists of all oracle queries made during an execution of $(i * V)^O(C, Z)$ (with fresh randomness), V cannot distinguish O and a “resampled” oracle O'/S :

$$\Delta(V^O(C, Z); V^{O'/S}(C, Z)) \leq \sqrt{\frac{p}{2T}},$$

where O'/S is sampled according to O conditioned only on being consistent with O on S , but is otherwise independent of C, Z . $\diamond$

Remark 14. A non-uniform version of this lemma is stated and proved in Appendix A, which states that there is a non-uniform polynomial-time pre-sampling procedure that can fool all size-bounded adversaries. We hope that it might be of independent interest. $\diamond$

We introduce notations used by the proof. For oracle algorithm F , we use the variable $F^o(c, z)$ as the entire computation history of running F with oracle o and input c, z , including oracle query-

response pairs, randomness and the input c, z . Let $F^O(C, Z)$ be the induced random variable computed from O, C, Z . We will omit the input if it is set to be (C, Z) by default.

Let O' be an oracle distribution that is distributed identically to O but is independent of Z and C . Let O'/S_i be the distribution O' conditioned on agreeing with O on all the queries in $(i * V)^O(C, Z)$. The goal is to bound the statistical distance $\Delta\left((i * V)^O, V^{O'/S_i}; ((i + 1) * V)^O\right)$

We use the variable J_k to quantify the correlation between O and Z that is discovered by running $(k * V)$. Formally, let

$$J_k = D\left((k * V)^O \parallel (k * V)^{O'} \mid C, Z\right) = D\left((k * V)^O \parallel (k * V)^{O'}\right).$$

Note that $D\left((k * V)^O \parallel (k * V)^{O'} \mid C, Z\right) = D\left((k * V)^O \parallel (k * V)^{O'}\right)$ for $k \geq 1$ since (C, Z) is contained in $(k * V)^O$ and has the same marginal distribution in $(k * V)^O$ and $(k * V)^{O'}$.

Lemma 6.13. For any $k \geq 0$, $J_k \leq H(Z) \leq p$. $\diamond$

Proof. Let $R_1, R_2, \dots, R_k$ be the random variable representing the random tape used by V in its $(1, 2, \dots, k)$ -th execution, respectively.

$$\begin{aligned} J_k &= D\left((k * V)^O \parallel (k * V)^{O'} \mid C, Z\right) \\ &\leq D(O, R_1, \dots, R_k, C, Z \parallel O', R_1, \dots, R_k, C, Z \mid C, Z) \\ &= D(O, R_1, \dots, R_k \parallel O', R_1, \dots, R_k \mid C, Z) \\ &\leq H(Z) \leq p. \quad (\text{Lemma 6.7}) \end{aligned} \quad \square$$

We observe that $J_{k+1} - J_k$ quantifies the correlation between O and O'/S_k discovered by the $(k + 1)$ -th execution of V . Formally,

$$\begin{aligned} J_{k+1} - J_k &= D\left(((k + 1) * V)^O \parallel ((k + 1) * V)^{O'}\right) - D\left((k * V)^O \parallel (k * V)^{O'}\right) \\ &= D\left((k * V)^O, V^O \parallel (k * V)^{O'}, V^{O'}\right) - D\left((k * V)^O \parallel (k * V)^{O'}\right) \\ &= D(V^O \parallel V^{O'/S_k} \mid (k * V)^O). \end{aligned} \quad (1)$$

The last equality is by chain rule.

Lemma 6.14. There exists $0 \leq i < T$ such that $J_{i+1} - J_i \leq \frac{p}{T}$. $\diamond$

Proof.

$$\sum_{k=0}^{T-1} (J_{k+1} - J_k) = J_T - J_0 \leq J_T \leq p,$$

where the first inequality is by non-negativity of KL divergence, and second inequality is by Lemma A.2. Therefore

$$\min_{0 \leq k < T} (J_{k+1} - J_k) \leq \frac{1}{T} \sum_{k=0}^{T-1} (J_{k+1} - J_k) \leq \frac{p}{T}. \quad \square$$

Proof (Lemma 6.12). Let $0 \leq i < T$ be the one specified by Lemma 6.14 such that $J_{i+1} - J_i \leq \frac{p}{T}$. Without loss of generality, assume $V^O(C, Z)$ and $V^{O'/S}(C, Z)$ consists of the entire computation history of the oracle algorithm. Then

$$\begin{aligned}
& \Delta(V^O(C, Z); V^{O'/S}(C, Z)) \\
& \leq \Delta(V^O, (i * V)^O; V^{O'/S}, (i * V)^O) \\
& \leq \sqrt{\frac{1}{2} D(V^O \| V^{O'/S_i} | (i * V)^O)} \\
& \stackrel{(1)}{=} \sqrt{\frac{1}{2} (J_{i+1} - J_i)} \leq \sqrt{\frac{p}{2T}}. \quad \square
\end{aligned}$$

6.3 Attack By Pre-sampling

We prove Lemma 6.3 by constructing a language L , then constructing an attack to any (purported) SNAP for L with $pq \leq t(|x|)$.

Language construction Take a linear secret sharing family $\{C_0^n, C_1^n\}_{n \in \mathbb{N}}$ with parameters as in Lemma 6.3. We define the language L as consisting of valid secret sharings of 0: for each $x \in \{0, 1\}^n$, $x \in L$ if $x \in C_0^n$. We omit n and directly write C_0, C_1 or p, q, t in the following.

The attack We construct an adversary $\mathcal{A}$ that convinces the SNAP verifier about some false $x \in C_1$ with non-negligible probability. The attack $\mathcal{A}(\text{crs}_P, \text{crs}_V)$ operates as follows.

1. Sample $x \leftarrow C_0$.
2. Run $\mathcal{P}(\text{crs}_P, x)$ honestly to generate a proof π .
3. Set T in the statement of Lemma 6.12 as proof size p . Sample $i \leftarrow \{0, 1, \dots, T-1\}$.
4. Run $\mathcal{V}^x(\text{crs}_V, \pi)$ for i times, recording all locations that has been queried in the set S .
5. Sample $x^* \leftarrow C_1$ conditioned on being consistent with x on S . It can be done efficiently as C_1 only contains linear constraints. We will show that such x^* always exists.
6. Output $x^*, \pi^* = \pi$.

Lemma 6.15. If $pq \leq t(n)$, then

$$\Pr[\mathcal{A} \text{ wins}] = \Pr \left[\begin{array}{c} \mathcal{V}^{x^*}(\text{crs}_V, \pi^*) = 1 \\ \wedge x^* \in C_1 \end{array} : \begin{array}{c} (\text{crs}_P, \text{crs}_V) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m) \\ (x^*, \pi^*) = \mathcal{A}(\text{crs}_P, \text{crs}_V) \end{array} \right] \geq \Omega\left(\frac{1}{p}\right). \quad \diamond$$

Proof. We apply Lemma 6.12, taking O as x being sampled from C_0 , C as $(\text{crs}_P, \text{crs}_V) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m)$, Z as the proof $\pi \leftarrow \mathcal{P}(\text{crs}_P, x)$. Lemma 6.12 implies that, let x' being sampled from C_0 conditioned on being consistent with x on S , then there exists $0 \leq i^* < T = p$ such that

$$\Pr \left[\begin{array}{c} (\text{crs}_P, \text{crs}_V) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m) \\ x \leftarrow C_0 \\ \pi \leftarrow \mathcal{P}(\text{crs}_P, x) \end{array} : \mathcal{V}^x(\text{crs}_V, \pi) = 1 \right]$$

$$- \Pr \left[\begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m) \\ x \leftarrow C_0 \\ \pi \leftarrow \mathcal{P}(\text{crs}_{\mathcal{P}}, x) \\ S \leftarrow (i^* * \mathcal{V})^x(\text{crs}_{\mathcal{V}}, \pi) \\ x' \leftarrow C_0|x_S \end{array} : \mathcal{V}^{x'}(\text{crs}_{\mathcal{V}}, \pi) = 1 \right] \leq \sqrt{\frac{|\pi|}{2T}} = \sqrt{\frac{1}{2}}.$$

By completeness, we know

$$\Pr \left[\begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m) \\ x \leftarrow C_0 \\ \pi \leftarrow \mathcal{P}(\text{crs}_{\mathcal{P}}, x) \end{array} : \mathcal{V}^x(\text{crs}_{\mathcal{V}}, \pi) = 1 \right] = 1.$$

Therefore,

$$\Pr \left[\begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m) \\ x \leftarrow C_0 \\ \pi \leftarrow \mathcal{P}(\text{crs}_{\mathcal{P}}, x) \\ S \leftarrow (i^* * \mathcal{V})^x(\text{crs}_{\mathcal{V}}, \pi) \\ x' \leftarrow C_0|x_S \end{array} : \mathcal{V}^{x'}(\text{crs}_{\mathcal{V}}, \pi) = 1 \right] \geq 1 - \sqrt{\frac{1}{2}} > \frac{1}{4}.$$

The next observation is, by the privacy of secret sharing, the output of V would be the same if it has oracle access to $x^* \leftarrow C_1|x_S$ instead of $x' \leftarrow C_1|x_S$.

Claim 6.16. Fix any choice of crs, π, S, x_S generated by running the attack. As long as $pq \leq t$, then $C_1|x_S$ is non-empty, and the following two distributions are equivalent:

$$\{\mathcal{V}^{x'}(\text{crs}_{\mathcal{V}}, \pi) : x' \leftarrow C_0|x_S\} \equiv \{\mathcal{V}^{x^*}(\text{crs}_{\mathcal{V}}, \pi) : x^* \leftarrow C_1|x_S\},$$

where $\mathcal{V}(\cdot)$ denotes the randomness and oracle query-answer pairs of in the execution of $\mathcal{V}$. $\diamond$

Proof. As $|S| \leq (p-1)q \leq t$, $C_1|x_S$ is non-empty as $\Pr_{x^* \leftarrow C_1}[x_S^* = x_S] = \Pr_{x' \leftarrow C_0}[x'_S = x_S] > 0$.

Fix the random tape of $\mathcal{V}$. Then for any set of oracle query-answer pairs (U, y_U) during one execution of $\mathcal{V}$, we have

$$\begin{aligned} & \Pr_{x' \leftarrow C_0|x_S} [\mathcal{V}^{x'}(\text{crs}_{\mathcal{V}}, \pi) = (U, y_U)] \\ &= \Pr_{x' \leftarrow C_0|x_S} [x'_U = y_U] \\ &= \Pr_{x' \leftarrow C_0} [x'_S = x_S \wedge x'_U = y_U] / \Pr_{x' \leftarrow C_0} [x'_S = x_S] \\ &= \Pr_{x^* \leftarrow C_1} [x_S^* = x_S \wedge x_U^* = y_U] / \Pr_{x^* \leftarrow C_1} [x_S^* = x_S] \quad (\text{by } t\text{-privacy, as } |S| + |U| \leq pq \leq t) \\ &= \Pr_{x^* \leftarrow C_1|x_S} [\mathcal{V}^{x^*}(\text{crs}_{\mathcal{V}}, \pi) = (U, y_U)]. \end{aligned} \quad \square$$

Since $pq \leq t$, we conclude that

$$\Pr \left[\begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m) \\ x \leftarrow C_0 \\ \pi \leftarrow \mathcal{P}(\text{crs}_{\mathcal{P}}, x) \\ S \leftarrow (i^* * \mathcal{V})^x(\text{crs}_{\mathcal{V}}, \pi) \\ x^* \leftarrow C_1|x_S \end{array} : \mathcal{V}^{x^*}(\text{crs}_{\mathcal{V}}, \pi) = 1 \right]$$

$$= \Pr \left[\begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m) \\ x \leftarrow C_0 \\ \pi \leftarrow \mathcal{P}(\text{crs}_{\mathcal{P}}, x) \\ S \leftarrow (i^* * \mathcal{V})^x(\text{crs}_{\mathcal{V}}, \pi) \\ x' \leftarrow C_0|_S \end{array} : \mathcal{V}^{x'}(\text{crs}_{\mathcal{V}}, \pi) = 1 \right] > \frac{1}{4}.$$

As $\mathcal{A}$ makes the correct guess $i = i^*$ with probability $1/p$, $\mathcal{A}$ breaks soundness with probability at least $1/4p$. $\square$

7 Lower Bound for Non-adaptively Secure SNAP

In this section, we prove a lower bound to non-adaptively secure SNAPs for P, that if there exists non-adaptively secure SNAP for P with query complexity q and communication complexity c such that $q \cdot c = o(n)$ (n is the instance size), then there exists a non-adaptively secure SNARG for NP with non-trivial communication complexity (i.e. the proof size is $o(m)$, where m is the witness size and we assume $m \geq n$).

Theorem 7.1. There exists a constant $\varepsilon > 0$ such that for every language $\mathcal{L} \in \text{NP}$ with witness size $m \geq n$ and polynomial $q(n) = o(n)$, there exists a language $\mathcal{L}' \in \text{P}$ such that every SNAP for $\mathcal{L}'$ with query complexity q , communication complexity $c = c(n)$ and distance εn such that $q(n) \cdot c(n) = o(n)$ implies a SNARG system for $\mathcal{L}$ with communication complexity $o(m)$. $\diamond$

We note that if the SNAP has succinct CRS i.e. $|\text{crs}| = \text{poly}(\lambda)$, then so do our non-adaptive SNARG.

7.1 Construction of SNARG for NP from SNAP for P

We fix the language $\mathcal{L} \in \text{NP}$ and polynomial $q(n) = o(n)$, and make use of the following conventions:

- Let n, m be the instance length and witness length for $\mathcal{L}$ respectively. Take a parameter t such that $t = \Theta(q(2tm))$. Note that t is a polynomial in m as q is sublinear.
- Let $\varepsilon > 0$ be a constant for which there exists an error correcting code $\text{ECC} : \{0, 1\}^n \rightarrow \{0, 1\}^{tm}$ with efficient encoding that is resilient to (3ε) -fraction of adversarially chosen errors.
- The SNAP system for language $\mathcal{L}'$ (defined below) consists of algorithms $(\text{SNAP}.\mathcal{G}, \text{SNAP}.\mathcal{P}, \text{SNAP}.\mathcal{V})$, where when instance size is ℓ , $\text{SNAP}.\mathcal{V}$ makes $q(\ell)$ queries and the proof size is $c(\ell)$, and non-adaptive soundness holds for distance $\varepsilon\ell$.
- The construction also uses a PRG $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{mt}$.

We define the language $\mathcal{L}'$ as

$$\mathcal{L}' = \{ \text{ECC}(x), s_1, \dots, s_t : (x, \oplus_{i=1}^t s_i) \in \mathcal{R}_{\mathcal{L}} \}.$$

As ECC is efficiently decodable, $\mathcal{L}' \in \text{P}$. Note that each instance in $\mathcal{L}$ with instance size n and witness size m can be converted to an instance in $\mathcal{L}'$ with length $\ell = 2tm$.

Our SNARG for $\mathcal{L}$ with instance size n runs SNAP for $\mathcal{L}'$ with instance size ℓ .

The Setup Algorithm. $\mathcal{G}(1^\lambda, 1^n, 1^m)$ does the following:

1. Sample $(\text{crs}_{\mathcal{P}}^{\text{SNAP}}, \text{crs}_{\mathcal{V}}^{\text{SNAP}}) \leftarrow \text{SNAP}.\mathcal{G}(1^\lambda, 1^\ell)$.
2. Sample $\bar{s} \leftarrow \{0, 1\}^\lambda$.

The common reference string is set to be

$$\text{crs}_{\mathcal{P}} = (\text{crs}_{\mathcal{P}}^{\text{SNAP}}, \bar{s}), \text{crs}_{\mathcal{V}} = (\text{crs}_{\mathcal{V}}^{\text{SNAP}}, \bar{s}).$$

The Prover Algorithm. $\mathcal{P}(\text{crs}_{\mathcal{P}}, x, w)$ does the following:

1. Parse $\text{crs}_{\mathcal{P}} = (\text{crs}_{\mathcal{P}}^{\text{SNAP}}, \bar{s})$, compute $s_1, \dots, s_t = G(\bar{s}) \in (\{0, 1\}^m)^t$.
2. Sample $k \leftarrow [t]$, compute $s'_k = (\oplus_{i \neq k} s_i) \oplus w$.
3. Prepare $\mathcal{L}'$ instance $x' = (\text{ECC}(x), s_1, \dots, s_{k-1}, s'_k, s_{k+1}, \dots, s_t)$.
4. Compute $\pi^{\text{SNAP}} = \text{SNAP}.\mathcal{P}(\text{crs}_{\mathcal{P}}^{\text{SNAP}}, x')$.
5. Send the proof $\pi = \pi^{\text{SNAP}}$.

The Verifier Algorithm. $\mathcal{V}(\text{crs}_{\mathcal{V}}, x, \pi)$ does the following:

1. Parse $\text{crs}_{\mathcal{V}} = (\text{crs}_{\mathcal{V}}^{\text{SNAP}}, \bar{s})$, and $\pi = \pi^{\text{SNAP}}$. Compute $s_1, \dots, s_t = G(\bar{s}) \in (\{0, 1\}^m)^t$.
2. Prepare $\mathcal{L}'$ instance $x'' = (\text{ECC}(x), s_1, \dots, s_t)$.
3. Run $b \leftarrow \text{SNAP}.\mathcal{V}^{x''}(\text{crs}_{\mathcal{V}}^{\text{SNAP}}, \pi^{\text{SNAP}})$.
4. Accept if $b = 1$.

7.2 Analysis

We prove that the SNARG system has proof size sublinear in m , as well as constant completeness and non-adaptive soundness. Note that the completeness can be boosted into $1 - \text{negl}(n)$ via parallel repetition.

Nontrivial proof size. The size of proof π is $|\pi^{\text{SNAP}}| = c(\ell)$. Furthermore, as $q(\ell)c(\ell) = o(\ell)$, $t = \Theta(q(\ell))$, we have

$$c(\ell) = o\left(\frac{\ell}{q(\ell)}\right) = o\left(\frac{tm}{t}\right) = o(m).$$

Completeness. We prove that for any $(x, w) \in \mathcal{R}_{\mathcal{L}} \cap (\{0, 1\}^n \times \{0, 1\}^m)$,

$$\Pr \left[\mathcal{V}(\text{crs}_{\mathcal{V}}, x, \pi) = 1 : \begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m) \\ \pi = \mathcal{P}(\text{crs}_{\mathcal{P}}, x, w) \end{array} \right] \geq 0.49.$$

By our construction, equivalently we show that

$$\Pr \left[\begin{array}{l} (\text{crs}_{\mathcal{P}}^{\text{SNAP}}, \text{crs}_{\mathcal{V}}^{\text{SNAP}}) \leftarrow \text{SNAP}.\mathcal{G}(1^\lambda, 1^\ell) \\ s_1, \dots, s_t \leftarrow G(\mathcal{U}_\lambda) \\ k \leftarrow [t] \\ \pi^{\text{SNAP}} = \text{SNAP}.\mathcal{P}(\text{crs}_{\mathcal{P}}^{\text{SNAP}}, x') \end{array} : \text{SNAP}.\mathcal{V}^{x''}(\text{crs}_{\mathcal{V}}^{\text{SNAP}}, \pi^{\text{SNAP}}) = 1 \right] \geq 0.49,$$

where $x' = (\text{ECC}(x), s_1, \dots, s'_k, \dots, s_t)$, $x'' = (\text{ECC}(x), s_1, \dots, s_k, \dots, s_t)$ as defined in the protocol description. By PRG security, it suffices to show

$$\Pr \left[\begin{array}{l} (\text{crs}_{\mathcal{P}}^{\text{SNAP}}, \text{crs}_{\mathcal{V}}^{\text{SNAP}}) \leftarrow \text{SNAP}.\mathcal{G}(1^\lambda, 1^\ell) \\ s_1, \dots, s_t \leftarrow (\{0, 1\}^m)^t \\ k \leftarrow [t] \\ \pi^{\text{SNAP}} = \text{SNAP}.\mathcal{P}(\text{crs}_{\mathcal{P}}^{\text{SNAP}}, x') \end{array} : \text{SNAP}.\mathcal{V}^{x''}(\text{crs}_{\mathcal{V}}^{\text{SNAP}}, \pi^{\text{SNAP}}) = 1 \right] \geq \frac{1}{2}.$$

Let GOOD denote the event that $\text{SNAP}.\mathcal{V}^{x''}(\text{crs}_{\mathcal{V}}^{\text{SNAP}}, \pi^{\text{SNAP}})$ does not query any of the coordinates that belong to s_k . Assume the SNAP system has completeness $1 - \delta$ ($\delta < 0.1$), then

$$\begin{aligned} & \Pr \left[\begin{array}{l} (\text{crs}_{\mathcal{P}}^{\text{SNAP}}, \text{crs}_{\mathcal{V}}^{\text{SNAP}}) \leftarrow \text{SNAP}.\mathcal{G}(1^\lambda, 1^\ell) \\ s_1, \dots, s_t \leftarrow (\{0, 1\}^m)^t \\ k \leftarrow [t] \\ \pi^{\text{SNAP}} = \text{SNAP}.\mathcal{P}(\text{crs}_{\mathcal{P}}^{\text{SNAP}}, x') \end{array} : \text{SNAP}.\mathcal{V}^{x''}(\text{crs}_{\mathcal{V}}^{\text{SNAP}}, \pi^{\text{SNAP}}) = 1 \mid \text{GOOD} \right] \\ &= \Pr \left[\begin{array}{l} (\text{crs}_{\mathcal{P}}^{\text{SNAP}}, \text{crs}_{\mathcal{V}}^{\text{SNAP}}) \leftarrow \text{SNAP}.\mathcal{G}(1^\lambda, 1^\ell) \\ s_1, \dots, s_t \leftarrow (\{0, 1\}^m)^t \\ k \leftarrow [t] \\ \pi^{\text{SNAP}} = \text{SNAP}.\mathcal{P}(\text{crs}_{\mathcal{P}}^{\text{SNAP}}, x') \end{array} : \text{SNAP}.\mathcal{V}^{x'}(\text{crs}_{\mathcal{V}}^{\text{SNAP}}, \pi^{\text{SNAP}}) = 1 \mid \text{GOOD} \right] \\ &\geq 1 - \delta / \Pr[\text{GOOD}]. \end{aligned}$$

As

$$\Pr[\text{SNAP}.\mathcal{V}^{x''}(\text{crs}_{\mathcal{V}}^{\text{SNAP}}, \pi^{\text{SNAP}}) = 1] \geq \Pr[\text{SNAP}.\mathcal{V}^{x''}(\text{crs}_{\mathcal{V}}^{\text{SNAP}}, \pi^{\text{SNAP}}) = 1 \mid \text{GOOD}] \cdot \Pr[\text{GOOD}],$$

it suffices to prove that $\Pr[\text{GOOD}] \geq \frac{3}{4}$. Fix $\text{crs}_{\mathcal{P}}^{\text{SNAP}}, \text{crs}_{\mathcal{V}}^{\text{SNAP}}$. Since for any π^{SNAP} and verifier randomness, $\text{SNAP}.\mathcal{V}^{x''}(\text{crs}_{\mathcal{V}}, \pi_{\text{SNAP}})$ queries any bit of s_k if and only if $\text{SNAP}.\mathcal{V}^{x'}(\text{crs}_{\mathcal{V}}, \pi_{\text{SNAP}})$ queries any bit of s'_k , we focus on proving

$$\Pr \left[\text{SNAP}.\mathcal{V}^{x'}(\text{crs}_{\mathcal{V}}, \text{SNAP}.\mathcal{P}(\text{crs}_{\mathcal{P}}, x')) \text{ queries } s'_k : \begin{array}{l} s_1, \dots, s_t \leftarrow (\{0, 1\}^m)^t \\ x' = (\text{ECC}(x), s_1, \dots, s'_k, \dots, s_t) \end{array} \right] \leq \frac{1}{4}.$$

Note that x' is sampled identically to the concatenation $\text{ECC}(x)$ and an additive secret sharing of w , so any program does not have any advantage in guessing k . On the other hand, we treat $\text{SNAP}.\mathcal{V}^\square(\text{crs}_{\mathcal{V}}, \text{SNAP}.\mathcal{P}(\text{crs}_{\mathcal{P}}, \square))$ as a program that takes x' as input and outputs q guesses about k . It follows from the union bound that any of these guesses is correct with probability at most $\frac{q}{t} = \frac{1}{4}$.

Non-adaptive soundness. We prove that for any poly-size adversary $\mathcal{A}$ and polynomial $m = m(n)$, there exists a negligible function $\text{negl}(\cdot)$ such that for any $n \in \mathbb{N}$, $x \notin \mathcal{L}$ with $|x| = n$,

$$\Pr \left[\mathcal{V}(\text{crs}_{\mathcal{V}}, x, \pi^*) : \begin{array}{l} (\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m) \\ \pi^* = \mathcal{A}(\text{crs}_{\mathcal{P}}, \text{crs}_{\mathcal{V}}, x) \end{array} \right] \leq \text{negl}(n).$$

By our construction, that is equivalent to

$$\Pr \left[\begin{array}{l} (\text{crs}_{\mathcal{P}}^{\text{SNAP}}, \text{crs}_{\mathcal{V}}^{\text{SNAP}}) \leftarrow \text{SNAP}.\mathcal{G}(1^\lambda, 1^\ell) \\ \bar{s} \leftarrow \{0, 1\}^\lambda \\ \text{SNAP}.\mathcal{V}^{x''}(\text{crs}_{\mathcal{V}}^{\text{SNAP}}, \pi^*) = 1 : \begin{array}{l} \pi^* = \mathcal{A}(\text{crs}_{\mathcal{P}}^{\text{SNAP}}, \text{crs}_{\mathcal{V}}^{\text{SNAP}}, \bar{s}, x) \\ (s_1, \dots, s_t) = G(\bar{s}) \\ x'' = (\text{ECC}(x), s_1, \dots, s_t) \end{array} \end{array} \right] \leq \text{negl}(n).$$

Assume that $\mathcal{A}$ breaks the above game on instance x with non-negligible probability $1/p(n)$. We fix $\bar{s}$ that maximizes success probability of $\mathcal{A}$. Therefore, for $x'' = (\text{ECC}(x), s_1, \dots, s_t)$ where $(s_1, \dots, s_t) = G(\bar{s})$,

$$\Pr \left[\text{SNAP}.\mathcal{V}^{x''}(\text{crs}_{\mathcal{V}}^{\text{SNAP}}, \pi^*) = 1 : \begin{array}{l} (\text{crs}_{\mathcal{P}}^{\text{SNAP}}, \text{crs}_{\mathcal{V}}^{\text{SNAP}}) \leftarrow \text{SNAP}.\mathcal{G}(1^\lambda, 1^\ell) \\ \pi^* = \mathcal{A}(\text{crs}_{\mathcal{P}}^{\text{SNAP}}, \text{crs}_{\mathcal{V}}^{\text{SNAP}}, \bar{s}, x) \end{array} \right] \geq 1/p(n).$$

As $x \notin \mathcal{L}$, $\Delta(x'', \mathcal{L}') \geq 3\varepsilon|\text{ECC}(x)| = \varepsilon \cdot 3tm > \varepsilon\ell$. Hence $\mathcal{A}$ breaks the non-adaptive soundness of the SNAP system on instance x'' with probability $1/p(n)$.

8 Fully Succinct SNAP with Non-adaptive Soundness

In this section, we construct non-adaptively secure SNAP for NP, whose verification time becomes $\text{poly}(\lambda)$ when we take distance $d = \Theta(n)$.

Theorem 8.1. Assume the polynomial hardness of LWE, sub-exponentially secure iO and sub-exponentially secure one-way function. For every language $\mathcal{L} \in \text{NP}$ and any distance $d = d(n)$, there exists a non-adaptively secure SNAP with proof length $n/d \cdot \text{poly}(\lambda)$, verification query complexity $n/d \cdot \text{poly}(\lambda)$ and verification efficiency $n/d \cdot \text{poly}(\lambda)$, but with long prover's CRS length $\text{poly}(\lambda, n, m)$. $\diamond$

8.1 Preliminary: Somewhere Statistical Correlation Intractable Hash Function

Before proving the theorem, we first introduce somewhere statistical correlation intractable hash function as one ingredient of the construction. The following is taken from [HLR21; HW24].

Throughout this section, let $R \subseteq X \times Y^t$ be a binary relation.

Definition 8.2 (Product Relation). We say that R is a product relation if for every x , the set $R(x) = \{y : (x, y) \in R\} \subseteq Y^t$ has a decomposition

$$R(x) = S_1 \times S_2 \times \dots \times S_t,$$

(where $S_1, \dots, S_t$ may depend on x). $\diamond$

Definition 8.3 (Efficient Product Verifiability). We say that a relation R is efficiently product verifiable if there is a polynomial-size circuit C such that, on every input x with some corresponding sets $(S_1, \dots, S_t)$ corresponding to x , it holds that $C(x, y, i) = 1$ if and only if $y \in S_i$. $\diamond$

Definition 8.4 (Product Sparsity). We say that a product relation R has product sparsity ρ if for every input x , the sets $S_1, \dots, S_t$ have size at most $\rho|Y|$. $\diamond$

Definition 8.5. We say that a function $h : X \rightarrow Y$ perfectly avoids a binary relation R if for all $x \in X$, $h(x) \notin R(x)$. We say that a hash family ensemble $\mathcal{H} = \{\mathcal{H}_\lambda\}_{\lambda \in \mathbb{N}}$ statistically avoids a relation ensemble $R = \{R_\lambda\}_{\lambda \in \mathbb{N}}$ if when sampling $H \leftarrow \mathcal{H}_\lambda$, it holds with all but $\lambda^{-\omega(1)}$ probability that H perfectly avoids R . $\diamond$

Definition 8.6 (Somewhere Statistical Correlation Intractability [CCH+19]). Let $R = \{R_\lambda\}_{\lambda \in \mathbb{N}}$ be a binary relation ensemble. A hash family ensemble $\mathcal{H} = \{\mathcal{H}_\lambda\}_{\lambda \in \mathbb{N}}$ is said to be somewhere statistically correlation intractable (SS-CI) for R if there exists a computationally indistinguishable hash family ensemble $\mathcal{F} = \{\mathcal{F}_\lambda\}_{\lambda \in \mathbb{N}}$ that statistically avoids R . We say that such a family $\mathcal{F}$ is an R -avoiding mode of $\mathcal{H}$. $\diamond$

Theorem 8.7 (Implicit in [HW24], see also Theorem 5.1 of [HLR21]). Let $R = R_\lambda \subseteq X_\lambda \times Y_\lambda^{t_\lambda}$ be an ensemble of product relations that are time- $T(\lambda)$ product-verifiable as in Definition 8.3 with product sparsity at most ρ , where $|Y_\lambda|, \log |X_\lambda|, T, t_\lambda$ are all polynomial in λ and $t_\lambda \geq \lambda/\log(1/\rho)$.

Then there exists a hash family $\mathcal{H} = \{\mathcal{H}_\lambda : X_\lambda \rightarrow Y_\lambda^{t_\lambda}\}_{\lambda \in \mathbb{N}}$ that is somewhere statistically correlation intractable for R under existence of indistinguishability obfuscation and the hardness of learning with errors. Moreover, $\mathcal{H}$ depends only on $(X_\lambda, Y_\lambda, T_\lambda, t_\lambda, \varepsilon)$ (and is otherwise independent of R) and can be evaluated in time $\text{poly}(\log |X|, |Y|, t, T)$. $\diamond$

8.2 Construction

In this section we present the construction of the SNAP system described in Theorem 8.1. We fix the language $\mathcal{L} \in \text{NP}$. The construction uses the following ingredients:

- A SS-CI hash family $\mathcal{H}$ for relation ensemble $R = R_\lambda \subseteq \{0, 1\}^n \times [n]^{t_\lambda}$ with product sparsity $1-d/n$ that can be product-verified in $\text{poly}(\lambda, n)$ time. Note that $\lambda/\log(1/\rho) \leq \lambda/\log(1+d/n) = \lambda \cdot O(n/d)$, therefore by Theorem 8.7, it suffices to take $t_\lambda = n/d \cdot \text{poly}(\lambda)$ to ensure such SS-CI hash family exist.

We call its hash key generation algorithm as $\text{hk} \leftarrow \mathcal{H}.\mathcal{G}(1^\lambda, 1^n)$, and its evaluation algorithm as $\text{Hash}(\text{hk}, \cdot)$.

- An adaptively secure SNARG system for the language $\mathcal{L}'$ that is parameterized by $\text{hk} \leftarrow \mathcal{H}.\mathcal{G}(1^\lambda, 1^n)$, defined as

$$\mathcal{L}' = \{(i_k || b_k)_{k \in [t_\lambda]} : \exists (x, w) \in R_\lambda \text{ s.t. } \text{Hash}(\text{hk}, x) = (i_k)_{k \in [t_\lambda]} \wedge \forall k \in [t_\lambda], x_{i_k} = b_k\}.$$

Write the instance size of $\mathcal{L}'$ as $n' = t_\lambda \log n = n/d \cdot \text{poly}(\lambda)$, and its witness length as $n + m$.

We require the SNARG system to have $\text{poly}(\lambda)$ proof length and $n' \text{poly}(\lambda)$ verification time, despite having a large NP verification circuit that needs to hard-code hk . This SNARG system is constructed from using Waters-Wu's adaptive SNARG for NP [WW24a] with $\text{poly}(\lambda)$ proof

length, and further apply a RAM delegation to reduce $|\text{crs}_V|$ to $\text{poly}(\lambda)$, and verifier's running time to $n' \text{poly}(\lambda)$ (see Lemma 3.18). This construction relies on polynomial hardness of LWE, sub-exponentially secure OWF and sub-exponentially secure iO.

As the verification circuit of $\mathcal{L}'$ has hk hard-coded in, we require the SNARG crs generation algorithm also take hk as input.

Our non-adaptively secure SNAP for $\mathcal{L}$ works as follows:

The Setup Algorithm. $\mathcal{G}(1^\lambda, 1^n, 1^m)$ does the following:

1. Sample $\text{hk} \leftarrow \mathcal{H}.\mathcal{G}(1^\lambda, 1^n)$.
2. Sample $(\text{crs}_P^{\text{SNARG}}, \text{crs}_V^{\text{SNARG}}) \leftarrow \text{SNARG}.\mathcal{G}(1^\lambda, 1^{n'}, 1^{n+m}, \text{hk})$.

The common reference string is set to be

$$\text{crs}_P = (\text{hk}, \text{crs}_P^{\text{SNARG}}), \text{crs}_V = \text{crs}_V^{\text{SNARG}}.$$

The Prover Algorithm $\mathcal{P}(\text{crs}_P, x, w)$ does the following:

1. Parse crs_P into $(\text{hk}, \text{crs}_P^{\text{SNARG}})$.
2. Compute $(i_k)_{k \in t_\lambda} = \text{Hash}(\text{hk}, x)$.
3. For each $k \in [t_\lambda]$, compute $b_k = x_{i_k}$.
4. Compute $\pi^{\text{SNARG}} = \text{SNARG}.\mathcal{P}(\text{crs}_P^{\text{SNARG}}, (i_k || b_k)_{k \in t_\lambda}, (x, w))$.
5. Output the proof $\pi = ((i_k || b_k)_{k \in t_\lambda}, \pi^{\text{SNARG}})$.

The Verifier Algorithm $\mathcal{V}^x(\text{crs}_V, \pi)$ does the following:

1. Parse crs_V into $\text{crs}_V^{\text{SNARG}}$.
2. Parse π into $((i_k || b_k)_{k \in t_\lambda}, \pi^{\text{SNARG}})$.
3. For each $k \in [t_\lambda]$, check whether $x_{i_k} = b_k$.
4. Check if $\text{SNARG}.\mathcal{V}(\text{crs}_V^{\text{SNARG}}, (i_k || b_k)_{k \in t_\lambda}, \pi^{\text{SNARG}}) = 1$.

8.3 Analysis

The completeness guarantee follows from the construction. On efficiency, we check that:

- The proof length is $n' + |\pi^{\text{SNARG}}| = n/d \cdot \text{poly}(\lambda)$.
- The verifier's query complexity is $t_\lambda = n/d \cdot \text{poly}(\lambda)$.

- The verifier's running time is again $n/d \cdot \text{poly}(\lambda)$, as to execute $\text{SNARG.V}(\text{crs}_V^{\text{SNARG}}, (i_k || b_k)_{k \in [t_\lambda]}, \pi^{\text{SNARG}})$ takes time $n' \text{poly}(\lambda) = n/d \cdot \text{poly}(\lambda)$.

Now we analyze the non-adaptive soundness guarantee, i.e. we show that for any adversary $\mathcal{A}$ and polynomials $n = n(\lambda), m = m(\lambda)$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$ and $x \in \{0, 1\}^n$ s.t. $\Delta(x, \mathcal{L}) > \varepsilon n$,

$$\Pr \left[\mathcal{V}^x(\text{crs}_V, \pi^*) = 1 : \begin{array}{l} (\text{crs}_P, \text{crs}_V) \leftarrow \mathcal{G}(1^\lambda, 1^n, 1^m) \\ \pi^* \leftarrow \mathcal{A}(\text{crs}_P, \text{crs}_V) \end{array} \right] \leq \text{negl}(\lambda).$$

Fix a sequence of (false) instances $\{x_\lambda \in \{0, 1\}^{n(\lambda)}\}_{\lambda \in \mathbb{N}}$. Consider the product relation $R = R_\lambda \subseteq \{0, 1\}^n \times [n]^{t_\lambda}$: for all $x' \in \{0, 1\}^n$, $R_\lambda(x') = S_\lambda(x')^{t_\lambda}$ where $S_\lambda(x') \subseteq [n]$ is defined as

$$S_\lambda(x') = \begin{cases} \emptyset, & \text{if } \Delta(x_\lambda, x') \leq \varepsilon n, \\ \{i \in [n] : x'_i = x_\lambda|_i\}, & \text{if } \Delta(x_\lambda, x') > \varepsilon n. \end{cases}$$

As R has product sparsity $1 - \varepsilon$ and $t_\lambda \geq \lambda/\varepsilon$, by Theorem 8.7, $\mathcal{H}$ is somewhere statistically correlation intractable for R , i.e. $\mathcal{H}$ has an R -avoiding mode $\mathcal{F}$.

Let $\mathcal{G}^\mathcal{F}(1^\lambda, 1^n, 1^m)$ be an alternative setup algorithm that samples $\text{hk} \leftarrow \mathcal{F}.\mathcal{G}(1^\lambda, 1^n)$.

Claim 8.8. If $\text{Hash}(\text{hk}, \cdot)$ perfectly avoids R , then for any $(i_k || b_k)_{k \in [t_\lambda]} \in \mathcal{L}'$, there exists $k \in [t_\lambda]$ such that $b_k \neq x_\lambda|_{i_k}$. $\diamond$

Proof. Fix λ . Since $(i_k || b_k)_{k \in [t_\lambda]} \in \mathcal{L}'$, we take $(x', w) \in R_\mathcal{L}$ as the corresponding witness where $x' \in \mathcal{L} \cap \{0, 1\}^n$. We have $\Delta(x', x_\lambda) > \varepsilon n$ and therefore,

$$\text{Hash}(\text{hk}, x') \notin R(x') \implies \exists k \in [t_\lambda], x'_{i_k} \neq x_\lambda|_{i_k}.$$

Since (x', w) is a valid witness, $b_k = x'_{i_k} \neq x_\lambda|_{i_k}$. $\square$

Therefore,

$$\Pr \left[\begin{array}{l} \mathcal{V}^{x_\lambda}(\text{crs}_V, \pi^*) = 1 \\ \wedge (i_k^* || b_k^*)_{k \in [t_\lambda]} \in \mathcal{L}' \end{array} : \begin{array}{l} (\text{crs}_P, \text{crs}_V) \leftarrow \mathcal{G}^\mathcal{F}(1^\lambda, 1^n, 1^m) \\ \pi^* \leftarrow \mathcal{A}(\text{crs}_P, \text{crs}_V) \\ \pi^* = ((i_k^* || b_k^*)_{k \in [t_\lambda]}, \pi^{\text{RAMDel}*}) \end{array} \right] \leq \text{negl}(\lambda).$$

On the other hand, by soundness of RAM delegation and adaptive soundness of the SNARG system,

$$\Pr \left[\begin{array}{l} \mathcal{V}^{x_\lambda}(\text{crs}_V, \pi^*) = 1 \\ \wedge (i_k^* || b_k^*)_{k \in [t_\lambda]} \notin \mathcal{L}' \end{array} : \begin{array}{l} (\text{crs}_P, \text{crs}_V) \leftarrow \mathcal{G}^\mathcal{F}(1^\lambda, 1^n, 1^m) \\ \pi^* \leftarrow \mathcal{A}(\text{crs}_P, \text{crs}_V) \\ \pi^* = ((i_k^* || b_k^*)_{k \in [t_\lambda]}, \pi^{\text{RAMDel}*}) \end{array} \right] \leq \text{negl}(\lambda).$$

So we conclude that

$$\Pr \left[\mathcal{V}^{x_\lambda}(\text{crs}_V, \pi^*) = 1 : \begin{array}{l} (\text{crs}_P, \text{crs}_V) \leftarrow \mathcal{G}^\mathcal{F}(1^\lambda, 1^n, 1^m) \\ \pi^* \leftarrow \mathcal{A}(\text{crs}_P, \text{crs}_V) \end{array} \right] \leq \text{negl}(\lambda),$$

which implies non-adaptive soundness as $\{\text{hk} \leftarrow \mathcal{H}.\mathcal{G}(1^\lambda, 1^n)\} \approx_c \{\text{hk} \leftarrow \mathcal{F}.\mathcal{G}(1^\lambda, 1^n)\}$.

Acknowledgements

We thank Ron Rothblum for the discussion. Daniel Wichs is supported by NSF CNS-2349972 and CNS-2055510. Part of this research was conducted while Liyan Chen was at Tsinghua University.

References

- [AHIV17] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkatasubramanian. “Ligero: Lightweight Sublinear Arguments Without a Trusted Setup”. In: *ACM CCS 2017*. Ed. by Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu. ACM Press, 2017, pp. 2087–2104. doi: [10.1145/3133956.3134104](https://doi.org/10.1145/3133956.3134104) (cit. on p. 3).
- [ABDY24] Gal Arnon, Shany Ben-David, and Eylon Yogev. *Hamming Weight Proofs of Proximity with One-Sided Error*. TCC. 2024. URL: <https://eprint.iacr.org/2024/832> (cit. on p. 1).
- [BFLS91] László Babai, Lance Fortnow, Leonid A. Levin, and Mario Szegedy. “Checking Computations in Polylogarithmic Time”. In: *23rd ACM STOC*. ACM Press, May 1991, pp. 21–31. doi: [10.1145/103418.103428](https://doi.org/10.1145/103418.103428) (cit. on p. 6).
- [BGI+01] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil P. Vadhan, and Ke Yang. “On the (Im)possibility of Obfuscating Programs”. In: *CRYPTO 2001*. Ed. by Joe Kilian. Vol. 2139. LNCS. Springer, Berlin, Heidelberg, Aug. 2001, pp. 1–18. doi: [10.1007/3-540-44647-8_1](https://doi.org/10.1007/3-540-44647-8_1) (cit. on p. 24).
- [BCGRS17] Eli Ben-Sasson, Alessandro Chiesa, Ariel Gabizon, Michael Riabzev, and Nicholas Spooner. “Interactive Oracle Proofs with Constant Rate and Query Complexity”. In: *ICALP*. Vol. 80. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2017, 40:1–40:15 (cit. on p. 6).
- [BCG+14] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. “Zerocash: Decentralized Anonymous Payments from Bitcoin”. In: *2014 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2014, pp. 459–474. doi: [10.1109/SP.2014.36](https://doi.org/10.1109/SP.2014.36) (cit. on p. 1).
- [BGHSV06] Eli Ben-Sasson, Oded Goldreich, Prahladh Harsha, Madhu Sudan, and Salil P. Vadhan. “Robust PCPs of Proximity, Shorter PCPs, and Applications to Coding”. In: *SIAM J. Comput.* 36.4 (2006), pp. 889–974 (cit. on p. 6).
- [BRV18] Itay Berman, Ron D. Rothblum, and Vinod Vaikuntanathan. “Zero-Knowledge Proofs of Proximity”. In: *ITCS 2018*. Ed. by Anna R. Karlin. Vol. 94. LIPIcs, Jan. 2018, 19:1–19:20. doi: [10.4230/LIPICS.ITCS.2018.19](https://doi.org/10.4230/LIPICS.ITCS.2018.19) (cit. on p. 6).
- [BCC+17] Nir Bitansky, Ran Canetti, Alessandro Chiesa, Shafi Goldwasser, Huijia Lin, Aviad Rubinstein, and Eran Tromer. “The Hunting of the SNARK”. In: *Journal of Cryptology* 30.4 (Oct. 2017), pp. 989–1066. doi: [10.1007/s00145-016-9241-9](https://doi.org/10.1007/s00145-016-9241-9) (cit. on pp. 2, 9).
- [BCIOP13] Nir Bitansky, Alessandro Chiesa, Yuval Ishai, Rafail Ostrovsky, and Omer Paneth. “Succinct Non-interactive Arguments via Linear Interactive Proofs”. In: *TCC 2013*. Ed. by Amit Sahai. Vol. 7785. LNCS. Springer, Berlin, Heidelberg, Mar. 2013, pp. 315–333. doi: [10.1007/978-3-642-36594-2_18](https://doi.org/10.1007/978-3-642-36594-2_18) (cit. on p. 2).

- [BBKLP23] Zvika Brakerski, Maya Farber Brodsky, Yael Tauman Kalai, Alex Lombardi, and Omer Paneth. “SNARGs for Monotone Policy Batch NP”. In: *CRYPTO 2023, Part II*. Ed. by Helena Handschuh and Anna Lysyanskaya. Vol. 14082. LNCS. Springer, Cham, Aug. 2023, pp. 252–283. doi: [10.1007/978-3-031-38545-2_9](https://doi.org/10.1007/978-3-031-38545-2_9) (cit. on p. 16).
- [BHK17] Zvika Brakerski, Justin Holmgren, and Yael Tauman Kalai. “Non-interactive delegation and batch NP verification from standard computational assumptions”. In: *49th ACM STOC*. Ed. by Hamed Hatami, Pierre McKenzie, and Valerie King. ACM Press, June 2017, pp. 474–482. doi: [10.1145/3055399.3055497](https://doi.org/10.1145/3055399.3055497) (cit. on p. 2).
- [BN13] Ahto Buldas and Margus Niitsoo. “Black-Box Separations and Their Adaptability to the Non-uniform Model”. In: *ACISP 13*. Ed. by Colin Boyd and Leonie Simpson. Vol. 7959. LNCS. Springer, Berlin, Heidelberg, July 2013, pp. 152–167. doi: [10.1007/978-3-642-39059-3_11](https://doi.org/10.1007/978-3-642-39059-3_11) (cit. on pp. 7, 8).
- [CCH+19] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, Ron D. Rothblum, and Daniel Wichs. “Fiat-Shamir: from practice to theory”. In: *51st ACM STOC*. Ed. by Moses Charikar and Edith Cohen. ACM Press, June 2019, pp. 1082–1090. doi: [10.1145/3313276.3316380](https://doi.org/10.1145/3313276.3316380) (cit. on p. 57).
- [CG18] Alessandro Chiesa and Tom Gur. “Proofs of Proximity for Distribution Testing”. In: *ITCS 2018*. Ed. by Anna R. Karlin. Vol. 94. LIPIcs, Jan. 2018, 53:1–53:14. doi: [10.4230/LIPIcs.ITCS.2018.53](https://doi.org/10.4230/LIPIcs.ITCS.2018.53) (cit. on p. 6).
- [CGJJZ23] Arka Rai Choudhuri, Sanjam Garg, Abhishek Jain, Zhengzhong Jin, and Jiaheng Zhang. “Correlation Intractability and SNARGs from Sub-exponential DDH”. In: *CRYPTO 2023, Part IV*. Ed. by Helena Handschuh and Anna Lysyanskaya. Vol. 14084. LNCS. Springer, Cham, Aug. 2023, pp. 635–668. doi: [10.1007/978-3-031-38551-3_20](https://doi.org/10.1007/978-3-031-38551-3_20) (cit. on p. 1).
- [CJJ22] Arka Rai Choudhuri, Abhishek Jain, and Zhengzhong Jin. “SNARGs for $\mathcal{P}$ from LWE”. In: *62nd FOCS*. IEEE Computer Society Press, Feb. 2022, pp. 68–79. doi: [10.1109/FOCS52979.2021.00016](https://doi.org/10.1109/FOCS52979.2021.00016) (cit. on pp. 1, 16, 18, 22, 24).
- [CDGS18] Sandro Coretti, Yevgeniy Dodis, Siyao Guo, and John P. Steinberger. “Random Oracles and Non-uniformity”. In: *EUROCRYPT 2018, Part I*. Ed. by Jesper Buus Nielsen and Vincent Rijmen. Vol. 10820. LNCS. Springer, Cham, 2018, pp. 227–258. doi: [10.1007/978-3-319-78381-9_9](https://doi.org/10.1007/978-3-319-78381-9_9) (cit. on pp. 7, 8, 49).
- [CDN15] Ronald Cramer, Ivan Bjerre Damgård, and Jesper Buus Nielsen. *Secure Multiparty Computation and Secret Sharing*. Cambridge University Press, 2015 (cit. on p. 48).
- [DGKV22] Lalita Devadas, Rishab Goyal, Yael Kalai, and Vinod Vaikuntanathan. “Rate-1 Non-Interactive Arguments for Batch-NP and Applications”. In: *63rd FOCS*. IEEE Computer Society Press, 2022, pp. 1057–1068. doi: [10.1109/FOCS54457.2022.00103](https://doi.org/10.1109/FOCS54457.2022.00103) (cit. on pp. 11, 20).
- [DORS08] Yevgeniy Dodis, Rafail Ostrovsky, Leonid Reyzin, and Adam D. Smith. “Fuzzy Extractors: How to Generate Strong Keys from Biometrics and Other Noisy Data”. In: *SIAM J. Comput.* 38.1 (2008), pp. 97–139 (cit. on p. 34).
- [EKR04] Funda Ergün, Ravi Kumar, and Ronitt Rubinfeld. “Fast approximate probabilistically checkable proofs”. In: *Information and Computation* 189.2 (2004), pp. 135–159 (cit. on p. 6).

- [GGH+13] Sanjam Garg, Craig Gentry, Shai Halevi, Mariana Raykova, Amit Sahai, and Brent Waters. “Candidate Indistinguishability Obfuscation and Functional Encryption for all Circuits”. In: *54th FOCS*. IEEE Computer Society Press, Oct. 2013, pp. 40–49. doi: [10.1109/FOCS.2013.13](https://doi.org/10.1109/FOCS.2013.13) (cit. on p. 24).
- [GW11] Craig Gentry and Daniel Wichs. “Separating succinct non-interactive arguments from all falsifiable assumptions”. In: *43rd ACM STOC*. Ed. by Lance Fortnow and Salil P. Vadhan. ACM Press, June 2011, pp. 99–108. doi: [10.1145/1993636.1993651](https://doi.org/10.1145/1993636.1993651) (cit. on p. 2).
- [GGR98] Oded Goldreich, Shari Goldwasser, and Dana Ron. “Property testing and its connection to learning and approximation”. In: *J. ACM* 45.4 (July 1998), 653–750. issn: 0004-5411. doi: [10.1145/285055.285060](https://doi.org/10.1145/285055.285060). url: <https://doi.org/10.1145/285055.285060> (cit. on p. 1).
- [GGR15] Oded Goldreich, Tom Gur, and Ron D. Rothblum. “Proofs of Proximity for Context-Free Languages and Read-Once Branching Programs - (Extended Abstract)”. In: *ICALP 2015, Part I*. Ed. by Magnús M. Halldórsson, Kazuo Iwama, Naoki Kobayashi, and Bettina Speckmann. Vol. 9134. LNCS. Springer, Berlin, Heidelberg, July 2015, pp. 666–677. doi: [10.1007/978-3-662-47672-7_54](https://doi.org/10.1007/978-3-662-47672-7_54) (cit. on p. 6).
- [GKR15] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. “Delegating Computation: Interactive Proofs for Muggles”. In: *J. ACM* 62.4 (2015), 27:1–27:64 (cit. on p. 1).
- [GLSTW23] Alexander Golovnev, Jonathan Lee, Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. “Brakedown: Linear-Time and Field-Agnostic SNARKs for R1CS”. In: *CRYPTO 2023, Part II*. Ed. by Helena Handschuh and Anna Lysyanskaya. Vol. 14082. LNCS. Springer, Cham, Aug. 2023, pp. 193–226. doi: [10.1007/978-3-031-38545-2_7](https://doi.org/10.1007/978-3-031-38545-2_7) (cit. on p. 3).
- [Gro10] Jens Groth. “Short Pairing-Based Non-interactive Zero-Knowledge Arguments”. In: *ASIACRYPT 2010*. Ed. by Masayuki Abe. Vol. 6477. LNCS. Springer, Berlin, Heidelberg, Dec. 2010, pp. 321–340. doi: [10.1007/978-3-642-17373-8_19](https://doi.org/10.1007/978-3-642-17373-8_19) (cit. on p. 2).
- [GLR18] Tom Gur, Yang P. Liu, and Ron D. Rothblum. “An Exponential Separation Between MA and AM Proofs of Proximity”. In: *ICALP 2018*. Ed. by Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella. Vol. 107. LIPIcs. Schloss Dagstuhl, July 2018, 73:1–73:15. doi: [10.4230/LIPIcs.ICALP.2018.73](https://doi.org/10.4230/LIPIcs.ICALP.2018.73) (cit. on p. 6).
- [GR15] Tom Gur and Ron D. Rothblum. “Non-Interactive Proofs of Proximity”. In: *ITCS 2015*. Ed. by Tim Roughgarden. ACM, Jan. 2015, pp. 133–142. doi: [10.1145/2688073.2688079](https://doi.org/10.1145/2688073.2688079) (cit. on p. 6).
- [GR17] Tom Gur and Ron D. Rothblum. “A Hierarchy Theorem for Interactive Proofs of Proximity”. In: *ITCS 2017*. Ed. by Christos H. Papadimitriou. Vol. 4266. 67: LIPIcs, Jan. 2017, 39:1–39:43. doi: [10.4230/LIPIcs.ITCS.2017.39](https://doi.org/10.4230/LIPIcs.ITCS.2017.39) (cit. on p. 6).
- [HLR21] Justin Holmgren, Alex Lombardi, and Ron D. Rothblum. “Fiat-Shamir via list-recoverable codes (or: parallel repetition of GMW is not zero-knowledge)”. In: *53rd ACM STOC*. Ed. by Samir Khuller and Virginia Vassilevska Williams. ACM Press, June 2021, pp. 750–760. doi: [10.1145/3406325.3451116](https://doi.org/10.1145/3406325.3451116) (cit. on pp. 15, 56, 57).

- [HW24] Justin Holmgren and Brent Waters. *Separating Selective Opening Security From Standard Security, Assuming IO*. Cryptology ePrint Archive, Paper 2024/1080. 2024. URL: <https://eprint.iacr.org/2024/1080> (cit. on pp. 15, 56, 57).
- [HW15] Pavel Hubacek and Daniel Wichs. “On the Communication Complexity of Secure Function Evaluation with Long Output”. In: *ITCS 2015*. Ed. by Tim Roughgarden. ACM, Jan. 2015, pp. 163–172. doi: [10.1145/2688073.2688105](https://doi.org/10.1145/2688073.2688105) (cit. on p. 19).
- [HJKS22] James Hulett, Ruta Jawale, Dakshita Khurana, and Akshayaram Srinivasan. “SNARGs for P from Sub-exponential DDH and QR”. In: *EUROCRYPT 2022, Part II*. Ed. by Orr Dunkelman and Stefan Dziembowski. Vol. 13276. LNCS. Springer, Cham, 2022, pp. 520–549. doi: [10.1007/978-3-031-07085-3_18](https://doi.org/10.1007/978-3-031-07085-3_18) (cit. on pp. 1, 18).
- [JLS22] Aayush Jain, Huijia Lin, and Amit Sahai. “Indistinguishability Obfuscation from LPN over $\mathbb{F}_p$, DLIN, and PRGs in NC^0 ”. In: *EUROCRYPT 2022, Part I*. Ed. by Orr Dunkelman and Stefan Dziembowski. Vol. 13275. LNCS. Springer, Cham, 2022, pp. 670–699. doi: [10.1007/978-3-031-06944-4_23](https://doi.org/10.1007/978-3-031-06944-4_23) (cit. on p. 4).
- [JLS24] Aayush Jain, Huijia Lin, and Amit Sahai. “Indistinguishability Obfuscation from Well-Founded Assumptions”. In: *Commun. ACM* 67.3 (2024), pp. 97–105 (cit. on p. 4).
- [KLVW23] Yael Kalai, Alex Lombardi, Vinod Vaikuntanathan, and Daniel Wichs. “Boosting Batch Arguments and RAM Delegation”. In: *55th ACM STOC*. Ed. by Barna Saha and Rocco A. Servedio. ACM Press, June 2023, pp. 1545–1552. doi: [10.1145/3564246.3585200](https://doi.org/10.1145/3564246.3585200) (cit. on pp. 1, 16, 18, 22, 24).
- [KPY19] Yael Tauman Kalai, Omer Paneth, and Lisa Yang. “How to delegate computations publicly”. In: *51st ACM STOC*. Ed. by Moses Charikar and Edith Cohen. ACM Press, June 2019, pp. 1115–1124. doi: [10.1145/3313276.3316411](https://doi.org/10.1145/3313276.3316411) (cit. on p. 24).
- [KR15] Yael Tauman Kalai and Ron D. Rothblum. “Arguments of Proximity - [Extended Abstract]”. In: *CRYPTO 2015, Part II*. Ed. by Rosario Gennaro and Matthew J. B. Robshaw. Vol. 9216. LNCS. Springer, Berlin, Heidelberg, Aug. 2015, pp. 422–442. doi: [10.1007/978-3-662-48000-7_21](https://doi.org/10.1007/978-3-662-48000-7_21) (cit. on pp. 1, 2, 4, 15).
- [KVZ21] Yael Tauman Kalai, Vinod Vaikuntanathan, and Rachel Yun Zhang. “Somewhere Statistical Soundness, Post-Quantum Security, and SNARGs”. In: *TCC 2021, Part I*. Ed. by Kobbi Nissim and Brent Waters. Vol. 13042. LNCS. Springer, Cham, Nov. 2021, pp. 330–368. doi: [10.1007/978-3-030-90459-3_12](https://doi.org/10.1007/978-3-030-90459-3_12) (cit. on pp. 1, 16).
- [KT22] Itay Kaley and Amnon Ta-Shma. “Unbalanced Expanders from Multiplicity Codes”. In: *APPROX/RANDOM*. Vol. 245. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022, 12:1–12:14 (cit. on pp. 10, 27).
- [KSY20] Liran Katzir, Clara Shikhelman, and Eylon Yogev. “Interactive Proofs for Social Graphs”. In: *CRYPTO 2020, Part III*. Ed. by Daniele Micciancio and Thomas Ristenpart. Vol. 12172. LNCS. Springer, Cham, Aug. 2020, pp. 574–601. doi: [10.1007/978-3-030-56877-1_20](https://doi.org/10.1007/978-3-030-56877-1_20) (cit. on p. 1).
- [Kiy23] Susumu Kiyoshima. “Holographic SNARGs for P and Batch-NP from (Polynomially Hard) Learning with Errors”. In: *TCC 2023, Part III*. Ed. by Guy N. Rothblum and Hoeteck Wee. Vol. 14371. LNCS. Springer, Cham, 2023, pp. 333–362. doi: [10.1007/978-3-031-48621-0_12](https://doi.org/10.1007/978-3-031-48621-0_12) (cit. on p. 6).

- [Mer88] Ralph C. Merkle. “A Digital Signature Based on a Conventional Encryption Function”. In: *CRYPTO’87*. Ed. by Carl Pomerance. Vol. 293. LNCS. Springer, Berlin, Heidelberg, Aug. 1988, pp. 369–378. doi: [10.1007/3-540-48184-2_32](https://doi.org/10.1007/3-540-48184-2_32) (cit. on pp. 18, 19).
- [Mic00] Silvio Micali. “Computationally Sound Proofs”. In: *SIAM J. Comput.* 30.4 (Oct. 2000), 1253–1298. ISSN: 0097-5397. doi: [10.1137/S0097539795284959](https://doi.org/10.1137/S0097539795284959). URL: <https://doi.org/10.1137/S0097539795284959> (cit. on p. 2).
- [OPWW15] Tatsuaki Okamoto, Krzysztof Pietrzak, Brent Waters, and Daniel Wichs. “New Realizations of Somewhere Statistically Binding Hashing and Positional Accumulators”. In: *ASIACRYPT 2015, Part I*. Ed. by Tetsu Iwata and Jung Hee Cheon. Vol. 9452. LNCS. Springer, Berlin, Heidelberg, 2015, pp. 121–145. doi: [10.1007/978-3-662-48797-6_6](https://doi.org/10.1007/978-3-662-48797-6_6) (cit. on p. 19).
- [PP22] Omer Paneth and Rafael Pass. “Incrementally Verifiable Computation via Rate-1 Batch Arguments”. In: *63rd FOCS*. IEEE Computer Society Press, 2022, pp. 1045–1056. doi: [10.1109/FOCS54457.2022.00102](https://doi.org/10.1109/FOCS54457.2022.00102) (cit. on p. 11).
- [RRR16] Omer Reingold, Guy N. Rothblum, and Ron D. Rothblum. “Constant-round interactive proofs for delegating computation”. In: *48th ACM STOC*. Ed. by Daniel Wichs and Yishay Mansour. ACM Press, June 2016, pp. 49–62. doi: [10.1145/2897518.2897652](https://doi.org/10.1145/2897518.2897652) (cit. on p. 6).
- [RVW13] Guy N. Rothblum, Salil P. Vadhan, and Avi Wigderson. “Interactive proofs of proximity: delegating computation in sublinear time”. In: *45th ACM STOC*. Ed. by Dan Boneh, Tim Roughgarden, and Joan Feigenbaum. ACM Press, June 2013, pp. 793–802. doi: [10.1145/2488608.2488709](https://doi.org/10.1145/2488608.2488709) (cit. on p. 6).
- [RS96] Ronitt Rubinfeld and Madhu Sudan. “Robust Characterizations of Polynomials with Applications to Program Testing”. In: *SIAM Journal on Computing* 25.2 (1996), pp. 252–271. doi: [10.1137/S0097539793255151](https://doi.org/10.1137/S0097539793255151). eprint: <https://doi.org/10.1137/S0097539793255151>. URL: <https://doi.org/10.1137/S0097539793255151> (cit. on p. 1).
- [SW14] Amit Sahai and Brent Waters. “How to use indistinguishability obfuscation: deniable encryption, and more”. In: *46th ACM STOC*. Ed. by David B. Shmoys. ACM Press, 2014, pp. 475–484. doi: [10.1145/2591796.2591825](https://doi.org/10.1145/2591796.2591825) (cit. on p. 1).
- [SW21] Amit Sahai and Brent Waters. “How to Use Indistinguishability Obfuscation: Deniable Encryption, and More”. In: *SIAM J. Comput.* 50.3 (2021), pp. 857–908 (cit. on p. 26).
- [Unr07] Dominique Unruh. “Random Oracles and Auxiliary Input”. In: *CRYPTO 2007*. Ed. by Alfred Menezes. Vol. 4622. LNCS. Springer, Berlin, Heidelberg, Aug. 2007, pp. 205–223. doi: [10.1007/978-3-540-74143-5_12](https://doi.org/10.1007/978-3-540-74143-5_12) (cit. on pp. 3, 7, 8).
- [WW22] Brent Waters and David J. Wu. “Batch Arguments for NP and More from Standard Bilinear Group Assumptions”. In: *CRYPTO 2022, Part II*. Ed. by Yevgeniy Dodis and Thomas Shrimpton. Vol. 13508. LNCS. Springer, Cham, Aug. 2022, pp. 433–463. doi: [10.1007/978-3-031-15979-4_15](https://doi.org/10.1007/978-3-031-15979-4_15) (cit. on pp. 1, 18).
- [WW24a] Brent Waters and David J. Wu. *A Pure Indistinguishability Obfuscation Approach to Adaptively-Sound SNARGs for NP*. Cryptology ePrint Archive, Paper 2024/933. 2024. URL: <https://eprint.iacr.org/2024/933> (cit. on pp. 1, 2, 4, 26, 34, 57).

- [WW24b] Brent Waters and David J. Wu. “Adaptively-Sound Succinct Arguments for NP from Indistinguishability Obfuscation”. In: *56th ACM STOC*. Ed. by Bojan Mohar, Igor Shinkar, and Ryan O’Donnell. ACM Press, June 2024, pp. 387–398. doi: [10.1145/3618260.3649671](https://doi.org/10.1145/3618260.3649671) (cit. on pp. 1, 2).
- [WZ24] Brent Waters and Mark Zhandry. “Adaptive Security in SNARGs via iO and Lossy Functions”. In: *CRYPTO 2024, Part X*. Ed. by Leonid Reyzin and Douglas Stebila. Vol. 14929. LNCS. Springer, Cham, Aug. 2024, pp. 72–104. doi: [10.1007/978-3-031-68403-6_3](https://doi.org/10.1007/978-3-031-68403-6_3) (cit. on pp. 1, 2).

A A Non-uniform Efficient Oracle Pre-sampling Lemma

Lemma A.1 (Efficient Oracle Pre-sampling, non-uniform version). Let O be any distribution of oracles (treated as a random variable later), and C be any random variable that is independent of O . Let Z be a random variable which may depend on O, C . We set $p := \log \#|Z|$ being the bit length to represent Z .

For any $T, q, f, p \geq 1$, there exists a deterministic oracle-aided circuit $F^o(c, z)$ of size $\text{poly}(f, T)$ that makes at most f queries and let $S = \{i_1, \dots, i_f\}$ as oracle queries made by $F^O(C, Z)$. Then for any oracle-aided distinguisher $A^o(c, z)$ with circuit complexity T and query complexity q ,

$$\Delta(A^O(C, Z); A^{O'/S}(C, Z)) \leq \sqrt{\frac{pq}{2f}},$$

where O'/S is an oracle sampled according to O conditioned only on being consistent with O on $S = \{i_1, \dots, i_f\} \leftarrow F^O(C, Z)$ (hence it is also a random variable). $\diamond$

We introduce notations used by the proof. For oracle algorithm F , we use the variable $F_k^o(c, z)$ as the response to the k -th oracle query made by F when running with oracle o and input c, z . Let $F_k^O(C, Z)$ be the induced random variable. We will omit the input to F if it is set to be (C, Z) by default. Write $F_{k \rightarrow l} = F_{k+1}, \dots, F_l$ and $F_{*l} = F_{0 \rightarrow l}$. For any two oracle algorithm A, B , we also write $A \circ B$ as the oracle algorithm that first runs B and then A .

Let O' be the distribution of oracles that is distributed identically to O but is independent from Z and C . Let O'/S_k be the distribution O' conditioned on agreeing with O on all the queries in $F_{*k}^O(C, Z)$. The goal is to bound the statistical distance $\Delta(F_{*k}^O, F_{k \rightarrow k+q}^{O'/S_k}, F_{*k+q}^O)$.

We use the variable $J_k(F)$ to quantify the correlation between O and Z that is discovered by first k queries of F . Formally, let

$$J_k(F) = D(F_{*k}^O || F_{*k}^{O'} | C, Z).$$

Lemma A.2. For any oracle algorithm F and $k \geq 0$, $J_k(F) \leq H(Z) \leq p$. $\diamond$

Proof.

$$\begin{aligned} J_k(F) &= D(F_{*k}^O || F_{*k}^{O'} | C, Z) \leq D(O, C, Z || O', C, Z | C, Z) \\ &= D(O || O' | C, Z) \\ &\leq H(Z) \leq p. \quad (\text{Lemma 6.7}) \end{aligned}$$

$\square$

We observe that $J_l(F) - J_k(F)$ quantifies the correlation between O and O'/S_k discovered by $(k+1), \dots, l$ -th queries of F . Formally,

$$\begin{aligned} J_l(F) - J_k(F) &= D(F_{*l}^O \| F_{*l}^{O'} | C, Z) - D(F_{*k}^O \| F_{*k}^{O'} | C, Z) \\ &= D(F_{*k}^O, F_{k \rightarrow l}^O \| F_{*k}^{O'}, F_{k \rightarrow l}^{O'} | C, Z) - D(F_{*k}^O \| F_{*k}^{O'} | C, Z) \\ &= D(F_{k \rightarrow l}^O \| F_{k \rightarrow l}^{O'/S_k} | F_{*k}^O, C, Z). \end{aligned} \quad (2)$$

The last equality is by chain rule.

Let $J_k = \max_{\text{size}(F) \leq kT, \#Q(F)=k} J_k(F)$, ranging from all oracle aided circuits F with size at most kT and makes exactly k queries. By definition $J_k \leq J_{k+1}$. We take F_k as the corresponding F that achieves J_k .

Lemma A.3. There exists $0 \leq k^* \leq f$ such that $J_{k^*+q} - J_{k^*} \leq \frac{pq}{f}$. $\diamond$

Proof. We have

$$\sum_{k=0}^{f-1} (J_{k+q} - J_k) \leq J_{f+q-1} + \dots + J_f \leq pq,$$

where the first inequality is by non-negativity of KL divergence, and second inequality is by Lemma A.2. Therefore

$$\min_{0 \leq k < f} (J_{k+q} - J_k) \leq \frac{1}{f} \sum_{k=0}^{f-1} (J_{k+q} - J_k) \leq \frac{pq}{f}. \quad \square$$

In the following, we take $F := F_{k^*}$, where k^* is specified by the above lemma such that $J_{k^*+q} - J_{k^*} \leq pq/f$.

Lemma A.4. For any oracle algorithm $A^o(c, z)$ with circuit complexity $\leq T$ that makes q queries, $J_{k^*+q}(A \circ F) - J_{k^*}(A \circ F) \leq \frac{pq}{f}$. $\diamond$

Proof. As $A \circ F$ has circuit complexity $T + k^*T \leq (k^* + q)T$, $J_{k^*+q}(A \circ F) \leq J_{k^*+q}$. On the other hand, $J_{k^*}(A \circ F) = J_{k^*}(F) = J_{k^*}$. Therefore

$$J_{k^*+q}(A \circ F) - J_{k^*}(A \circ F) \leq J_{k^*+q} - J_{k^*} \leq \frac{pq}{f}. \quad \square$$

The last step is to apply the standard inequality that bounds statistical distance by KL divergence, i.e. Fact 6.8.

Proof (Lemma A.1). The oracle-aided circuit F is specified above as F_{k^*} and used in Lemma A.4. Without loss of generality, assume $A^o(c, z)$ is deterministic and makes exactly q queries. Denote $F' = A \circ F$. Then

$$\begin{aligned} &\Delta(A^O(C, Z); A^{O'/S}(C, Z)) \\ &\leq \Delta(A_{*q}^O, F_{*k^*}^O, C, Z; A_{*q}^{O'/S}, F_{*k^*}^O, C, Z) \\ &= \Delta(F_{k^* \rightarrow k^*+q}^{O'}, F_{*k^*}^{O'}, C, Z; F_{k^* \rightarrow k^*+q}^{O'/S_{k^*}}, F_{*k^*}^{O'}, C, Z) \end{aligned}$$

$$\begin{aligned}
&\leq \sqrt{\frac{1}{2} D(F_{k^* \rightarrow k^*+q}^{O'} \| F_{k^* \rightarrow k^*+q}^{O'/S_{k^*}} | F_{*k^*}^{O'}, C, Z)} \quad (\text{Fact 6.8}) \\
&\stackrel{(2)}{=} \sqrt{\frac{1}{2} (J_{k^*+q}(A \circ F) - J_{k^*}(A \circ F))} \\
&\leq \sqrt{\frac{pq}{2f}} \quad (\text{Lemma A.4}).
\end{aligned}$$

□